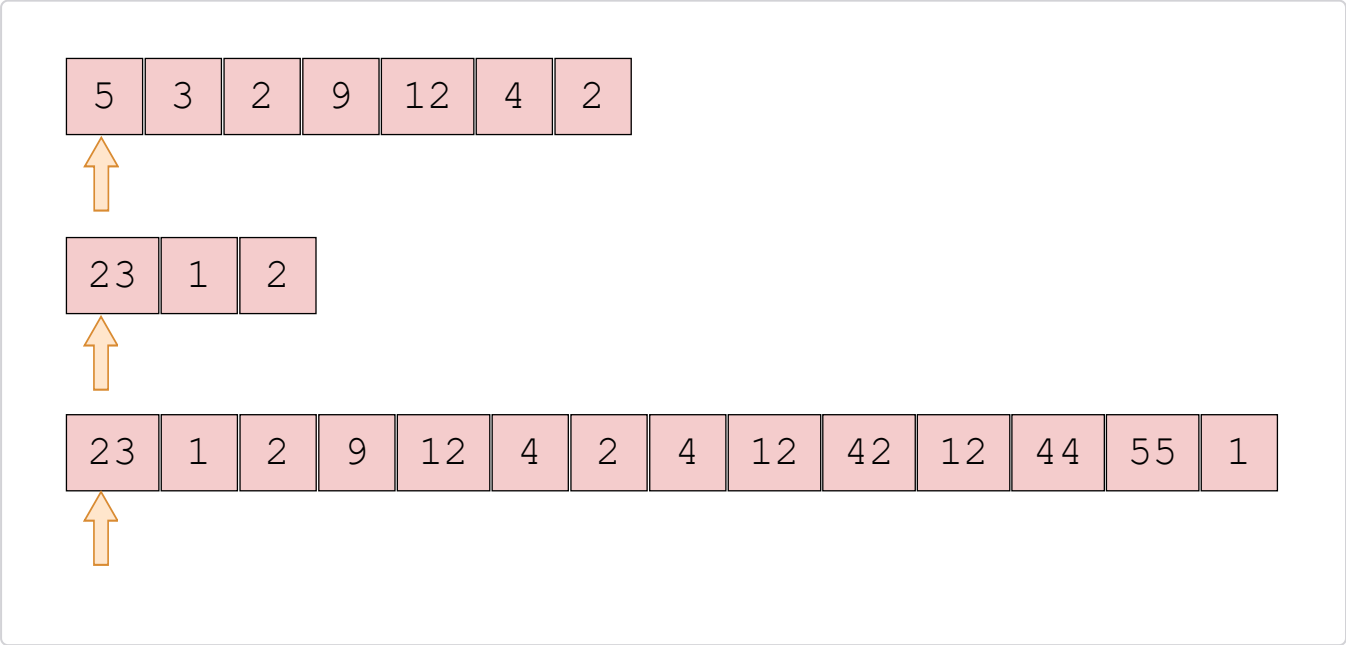
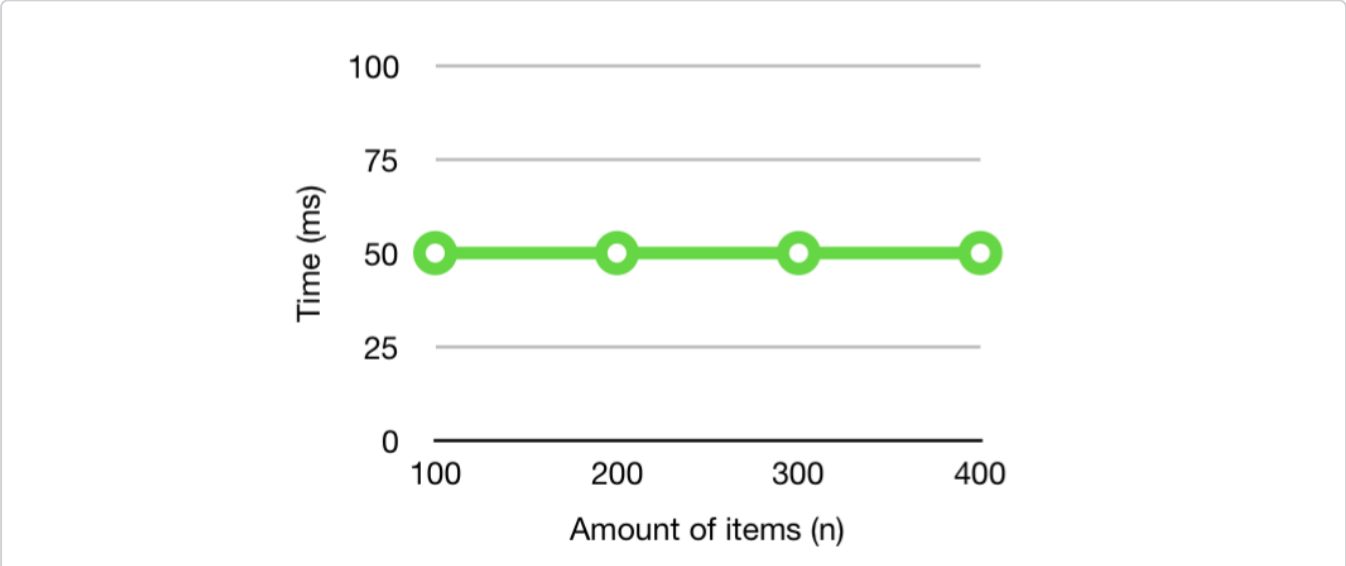
Constant Complexity - O(1)

An algorithm has constant time complexity if it takes the same time regardless of the number of inputs. (Reading time: under 1 minute)

If an algorithm's time complexity is constant, it means that it will always run in the same amount of time, no matter the input size. For example, if we want to get the first item of an array, it doesn't matter how big the input size is.



It always takes the same amount of time to find the first element in the array. In a graph, it would look like this:

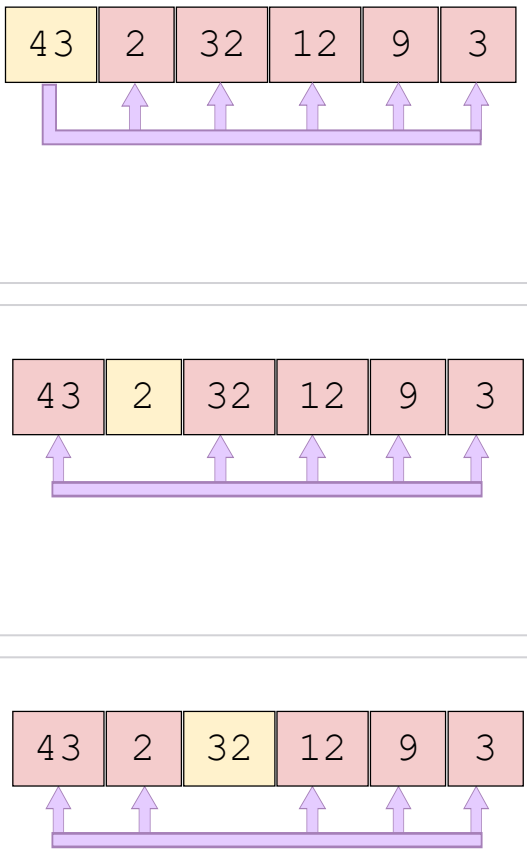


In the next lesson, I will talk about the linear time complexity of an algorithm.

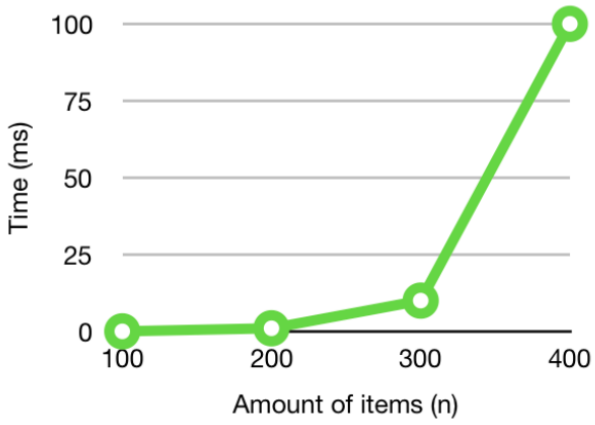
Quadratic Complexity - O(n^2)

If an algorithm's time complexity is quadratic, it means that the runtime of the algorithm is directly proportional to the square of the size of the input. (Reading time: under 1 minute)

A common example of an algorithm with quadratic time complexity is by looping over an array, and comparing the current element with all other elements in the array. We have to loop over n elements, and for every element, we again have to loop over n elements. This ends up being n * n, which is n^2.



For every element in the array, we loop over the array, and compare their values. In a graph, it would look like this:



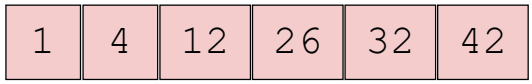
Now, let’s talk about algorithms having logarithmic time complexity.

Logarithmic Complexity - O(log(n))

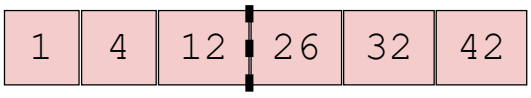
The size of the input gets split into half with each iteration of the function. (Reading time: under 2 minutes)

If an algorithm has logarithmic time complexity, it means that the size of the input we are considering gets split into half with each iteration. Let's say that we have a function that takes 1 second to execute if the input size is 100. With a logarithmic runtime, it would then take 2 seconds if the input size is 1000, and 3 seconds if the input size is 10,000. The bigger the input size gets, the smaller the difference in runtime!

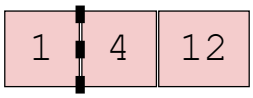
Consider the following array:



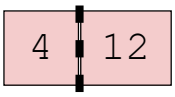
Let's say that we're looking for the value of 7 in this sorted array. With a specific searching algorithm, we split the input array on every round, and only check the side of the half (displayed as the dotted line) where the value could potentially be.



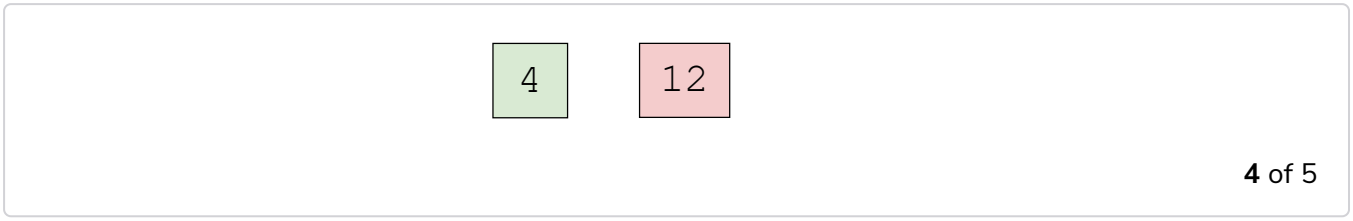
1 of 5

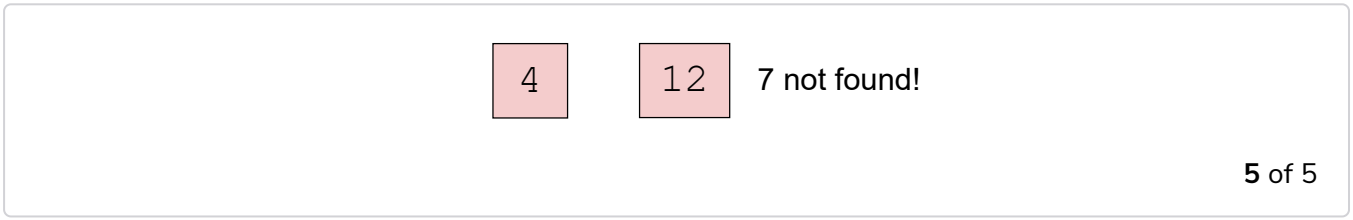


2 of 5



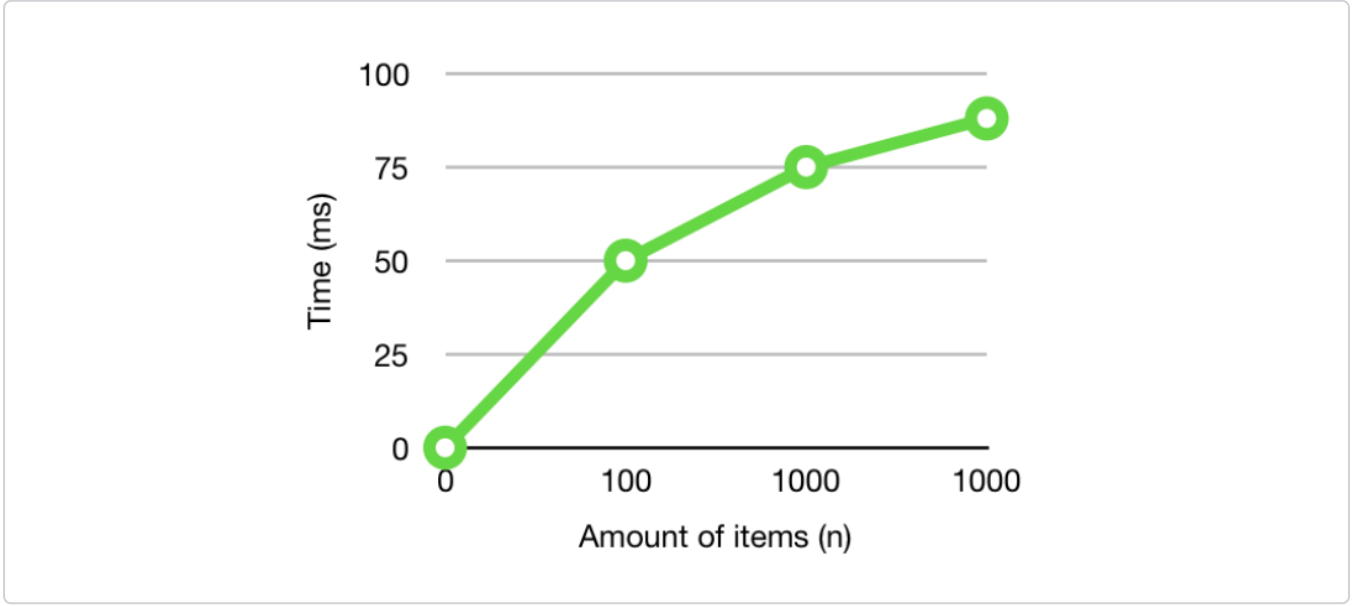
3 of 5





After every execution, the size of the array gets split (approximately) in half!

In a graph, it would look like this:



In the next lesson, I will discuss algorithms with exponential time complexity.

Exponential Complexity - O(2^n)

The runtime of the algorithm gets doubled after every addition in the input. (Reading time: 1 minute)

If an algorithm's time complexity is O(2^n), its runtime is doubled after every addition to the input size. If 5 items took 30 seconds, 6 items would take 60 seconds.

In the following example, the value of an element is either 0 or 1. The amount of possibilities with 0 and 1 that this array could have, is 2^9 = 512.

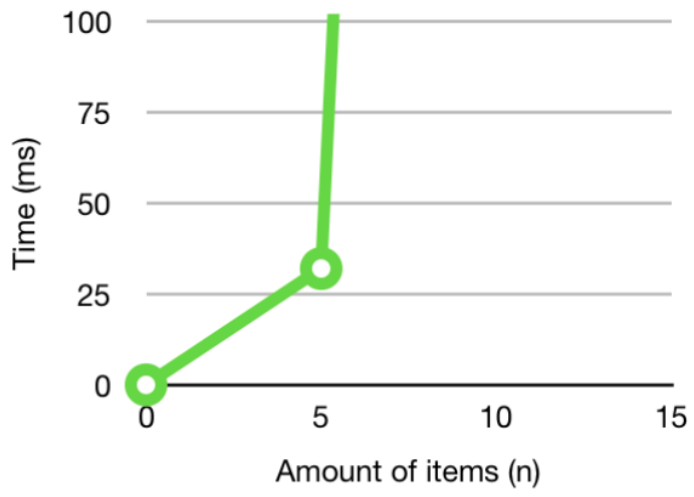
1	0	0	1	1	0	0	1	1

However, if we add just one more element, we have 2^10 = 1024 possibilities.

1	0	0	1	1	0	0	1	1	0

By just adding one element, we doubled the number of possibilities!

In a graph, it gets clear that this is extremely inefficient, as the runtime becomes double after every addition.



In the next chapter, I will discuss different sorting algorithms and their time complexities.

Introduction to Data Structures

Data structures are a collection of values, that are all connected to each other in a different way. (Reading time: under 1 minute)

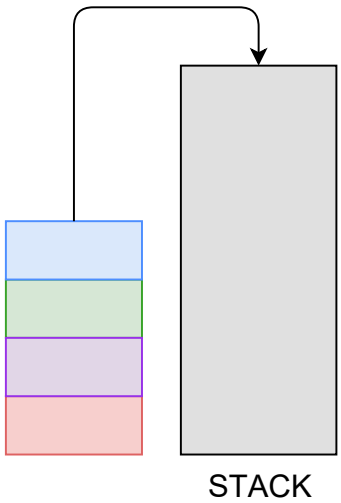
There are many different data structures. Each has its pros and cons in different scenarios. In the next few lessons, I will talk about the most common data structures:

- Stack
- Queue
- Linked List
- Binary Search Tree
- Hash Table
- Graph

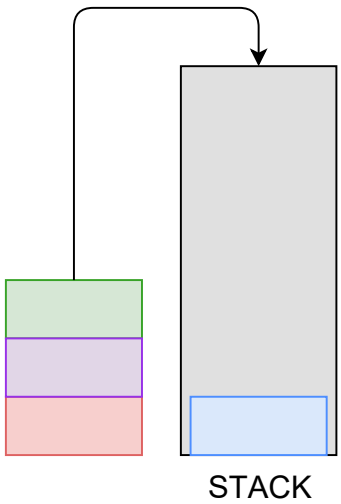
Introduction to Stacks

A stack is a collection of items that can only be stored or deleted in a particular order. (Reading time: under 1 minute)

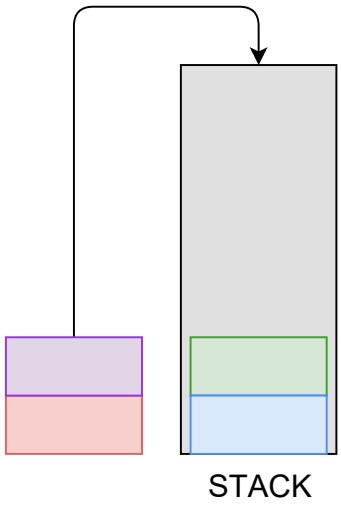
You can think of the stack as a container, to which we can add items, and remove them. Only the top of this “container” is open, so the item we put in first will be taken out last, and the items we put in last will be taken out first. This is called the **last-in-first-out principle**.

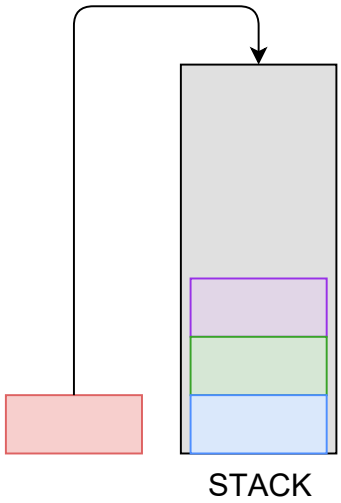


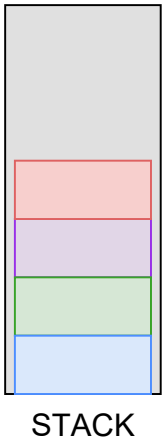
1 of 10

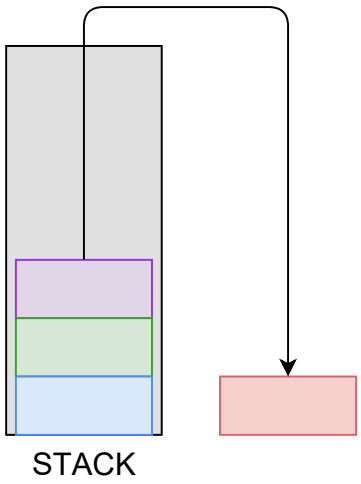


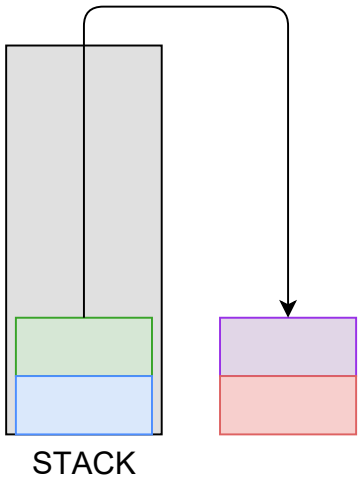
2 of 10

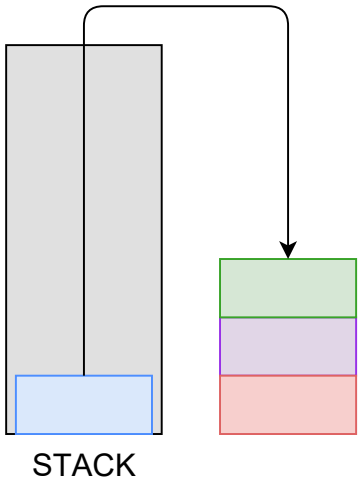

STACK

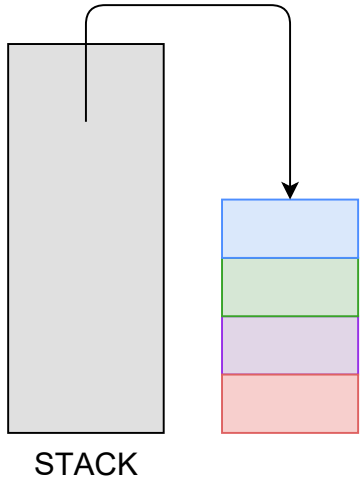

STACK


STACK

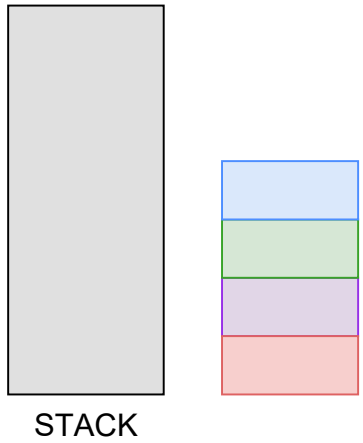

STACK


STACK


STACK



9 of 10



10 of 10

Now, let's move on to the implementation of this data structure.

Stacks (Implementation)

create a stack, add and delete data, and print the stack (Reading time: under 2 minutes)

In a stack, we should at least be able to **push** and **pop** values, or **nodes**. But first, we need to create the stack.

```
class Stack {
  constructor() {
    this.stack = [];
  }
}
```

The initial value of the stack is an empty array: there are no nodes in the stack unless we push them!

To push a node, we can use the built-in push method that JavaScript provides.

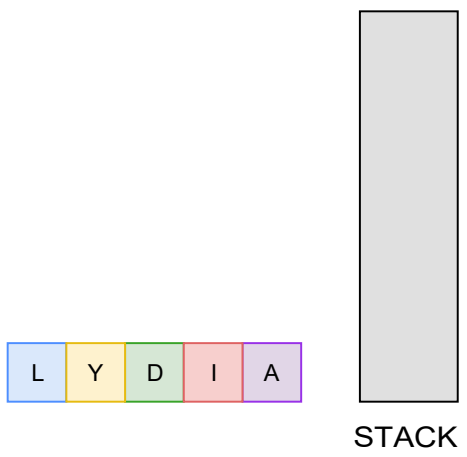
```
  push(data) {
    this.stack.push(data);
  }
}
let myStack = new Stack(); //create item of type stack
//push elements to a stack
myStack.push(2);
myStack.push(3);
myStack.push(9);
myStack.push(1);
//print stack
console.log("Your stack is:\n"+myStack.printStack()); //printStack discussed later in this le
```

The same goes for popping a node. However this time we don't need to provide any data, as the pop method always pops the last item in the list:

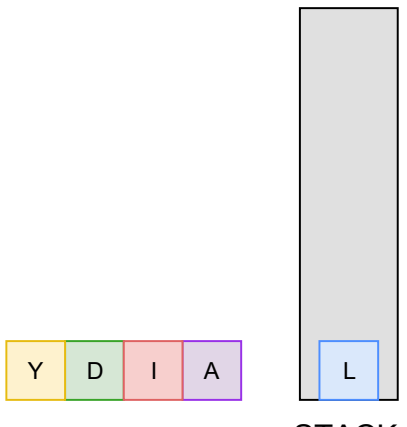
```
pop() { //pop takes no arguments and automatically removes the top elements
  return this.stack.pop();
}
}
```

```
let myStack = new Stack(); //create item of type stack
//push elements to a stack

myStack.push(2);
myStack.push(3);
myStack.push(9);
myStack.push(1);

//print stack
console.log("Your stack is:\n"+myStack.printStack()); //printStack discussed later in this le

//pop items from the stack
myStack.pop();
//print stack
console.log("Your stack after popping is:\n"+myStack.printStack()); //printStack discussed la
```

These are the two most important methods of a stack, as we're able to add values and pop them off again. Let's say that we want to reverse the word "Lydia":

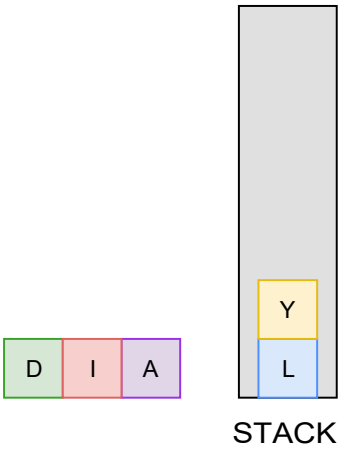


1 of 11

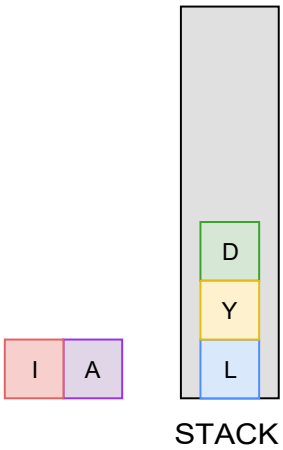


STACK

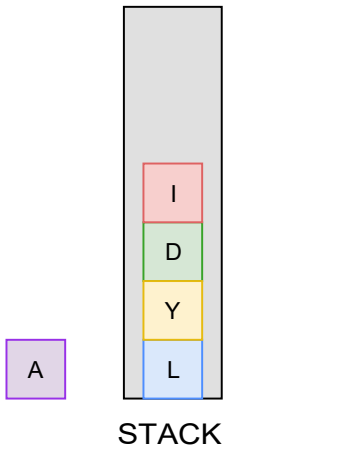
push



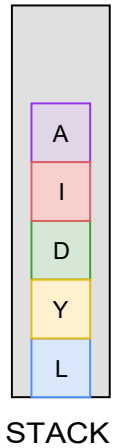
push



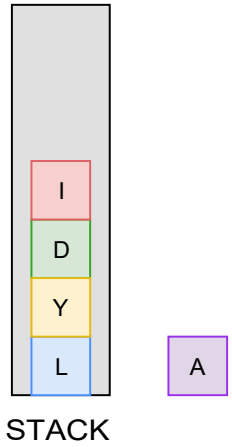
push



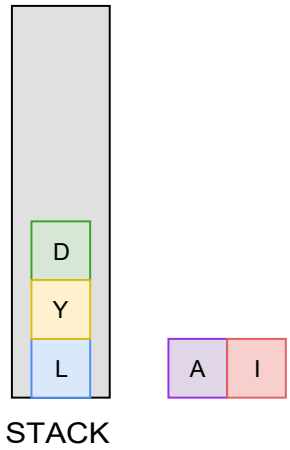
push



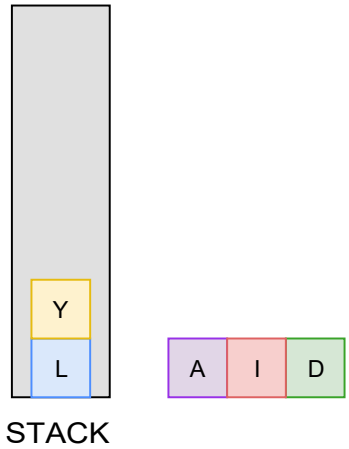
push



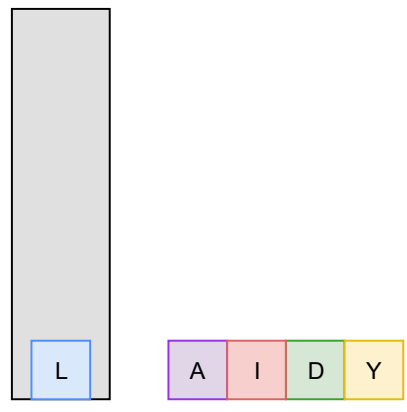
pop



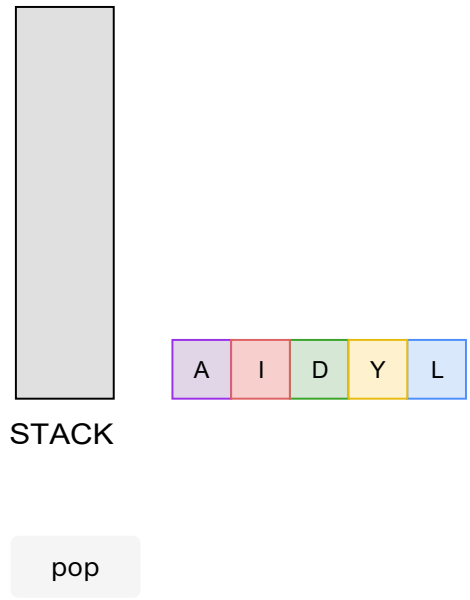
pop



pop



10 of 11



11 of 11

In the above example, we first make an array of the string “Lydia”. The values get pushed to the stack, and the value on top of the stack gets popped off first. This results in the word reversed!

Another important function is print. You can view the code below:

```
printStack()
  {
    var str = "";
    for (var i = 0; i < this.stack.length; i++)
        str += this.stack[i] + "\n";
    return str;
  }//print function ends here
}

let myStack = new Stack(); //create item of type stack
//push elements to a stack
myStack.push(2);
myStack.push(3);
myStack.push(9);
myStack.push(1);
//print stack
console.log("Your stack is:\n"+myStack.printStack());
```

In the next lesson, I will talk about the time complexity of these functions.

Stacks (Time Complexity)

best, average and worst case time complexity of the various stack functions (Reading time: under 1 minute)

	TIME		SPACE
Type	**Average**	**Worst**	**Worst**
Get, Search	O(n)	O(n)	O(n)
Insertion, Deletion	O(1)	O(1)	

Get and Search:

To get or search for a certain value, we'd have to walk over all the items in the stack. The amount of time needed is directly proportional to the number of items in the stack.

Insertion and Deletion:

When we insert new data onto the stack, we add it at the top of the stack. When we delete an item, we pop the top element off. No need to iterate through any data.

Worst space:

The more items, the bigger the **stack** array.

In the next lesson, let's talk about a data structure that has both ends open.

Introduction to Queues

A queue is an ordered data structure where items follow the first-in-first-out principle. (Reading time: under 1 minute)

You can think of a queue as a pipe, with both ends open. Objects enter from one and are removed from the other. This means that, unlike the stack that used the last-in-first-out principle, queues use the **first-in-first-out** principle.

In the next lesson, I will talk about the implementation of this data structure.

Queues (Implementation)

create a queue, add and delete items, and print the queue (Reading time: 3 minutes)

The queue will be an array, just like we did with the stack.

```
class Queue {
  constructor() {
    this.queue = [];
  }
}
```

Again, we initialize an empty array as our queue. To enqueue a node, we use the same built-in method that JavaScript provides.

```
  enqueue(data) {
    this.queue.push(data);
  }
}

//create an object of type queue
let myQueue = new Queue();

//insert items into the queue
myQueue.enqueue(2);
myQueue.enqueue(3);
myQueue.enqueue(9);
myQueue.enqueue(1);

//print the queue (I'll discuss this function in detail later in this lesson)
console.log("Your queue is:\n"+myQueue.printQueue());
```

To dequeue, you use shift instead of pop! This returns the first index of the array, which means that the first value added to the array will also be removed first. The first-in-first-out principle!

```
  dequeue() { //does not need an argumen because it deletes the elements that was inserted fi
    return this.queue.shift();
  }
```

```
}

//create an object of type queue
let myQueue = new Queue();

//insert items into the queue
myQueue.enqueue(2);
myQueue.enqueue(3);
myQueue.enqueue(9);
myQueue.enqueue(1);
//print the queue (I'll discuss this function in detail later in this lesson)
console.log("Your queue is:\n"+myQueue.printQueue());

myQueue.dequeue();
console.log("Your queue after deletion is:\n"+myQueue.printQueue());
```

Another important queue function is print. You can view the code here:

```
printQueue()
{
    var str = "";
    for (var i = 0; i < this.queue.length; i++)
        str += this.queue[i] + " ";
    return str;
}
```

In the next lesson, I will talk about the time complexity of various queue functions.

Queues (Time Complexity)

best, average and worst case time complexity of the various queue functions (Reading time: under 1 minute)

	TIME		SPACE
Type	**Average**	**Worst**	**Worst**
Get, Search, Deletion	O(n)	O(n)	O(n)
Insertion	O(1)	O(1)	

Get and Search:

To get or search for a certain value, we'd have to walk over all the items in the queue. The time needed is directly proportional to the number of items in the queue.

Insertion:

When we insert new data into the queue, we push it to the end of the queue.

Deletion:

Due to the internals of the JavaScript shift method, which walks over the entire array and returns the last item, the time complexity for deletion is linear.

Worst space:

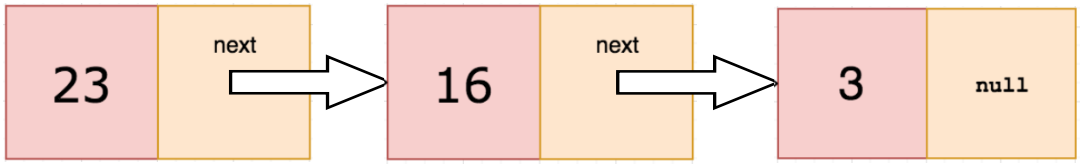
The more items, the bigger the queue array.

In the next lesson, I will discuss linked lists.

Singly Linked List

A list of elements connected using pointers. (Reading time: under 1 minute)

A singly linked list is a linear data structure. Each element, called a node, is connected to the other, by pointers (or references) to the next node.



If we log this linked list, it will look like this:

```
{
   data: 23,
   next: {
      data: 16,
      next: {
         data: 3,
         next: null
      }
   }
}
```

In the next lesson, I will discuss the implementation of this data structure.

Singly Linked List (Implementation)

create a linked list, add a node, and add and remove data to a linked list (Reading time: 4 minutes)

In a singly linked list, we should be able to find, remove, and insert nodes. Before we can make a list, we first need to create the nodes.

```
function Node(data) {
  this.data = data;
  this.next = null;
}
```

We now created a constructor function that we can use each time we create a new node. By default, the new node's next value is null, and its data is equal to the data we pass as an argument.

The linked list will be a **class**. This class will have several properties, such as the remove function, add function, and find function. However, before we can do any of that, we need to create its constructor.

```
class SinglyLinkedList {
  constructor() {
    this.head = null;
    this.tail = null;
  }
}
```

By default, the list doesn't have any nodes, and the length is equal to 0. If the list doesn't have any nodes, both the head (the first node in the list) and the tail (the last node in the list) don't exist, so their values are equal to **null**.

The function to **add** a node to the tail is as follows:

```
addNode(data) {
  const node = new Node(data);
  if (!this.head) {
    this.tail = node;
    this.head = node;
```

```
  } else {
    this.tail.next = node;
    this.tail = node;

  }
}
```

First, we need to create a new node. As the **Node** function is a constructor function, we do this by typing **new Node(data)** with the data passed as an argument to **SinglyLinkedList**. If there is no head in the list, meaning that there are no nodes at all in the list, the new node is both the head and the tail. The list would consist of only the new node!

```
const list = new SinglyLinkedList();
list.addNode(23);
```

The above code snippet would result in:

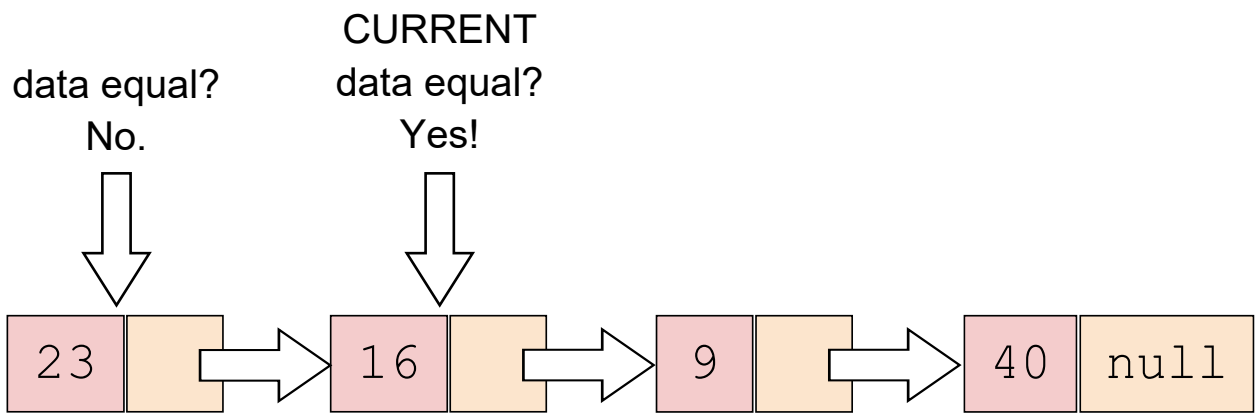
```
{
   data: 23,
   next: null
}
```

There's only one node in the list, which represents both the head and the tail.

```
insertAfter(data, toNodeData) {
  let current = this.head;
  while (current) {
    if (current.data === toNodeData) {
      const node = new Node(data);
      if (current === this.tail) {
        this.tail.next = node;
        this.tail = node;
      } else {
        node.next = current.next;
        current.next = node;
        break;
      }
    }
    current = current.next;
  }
}
```

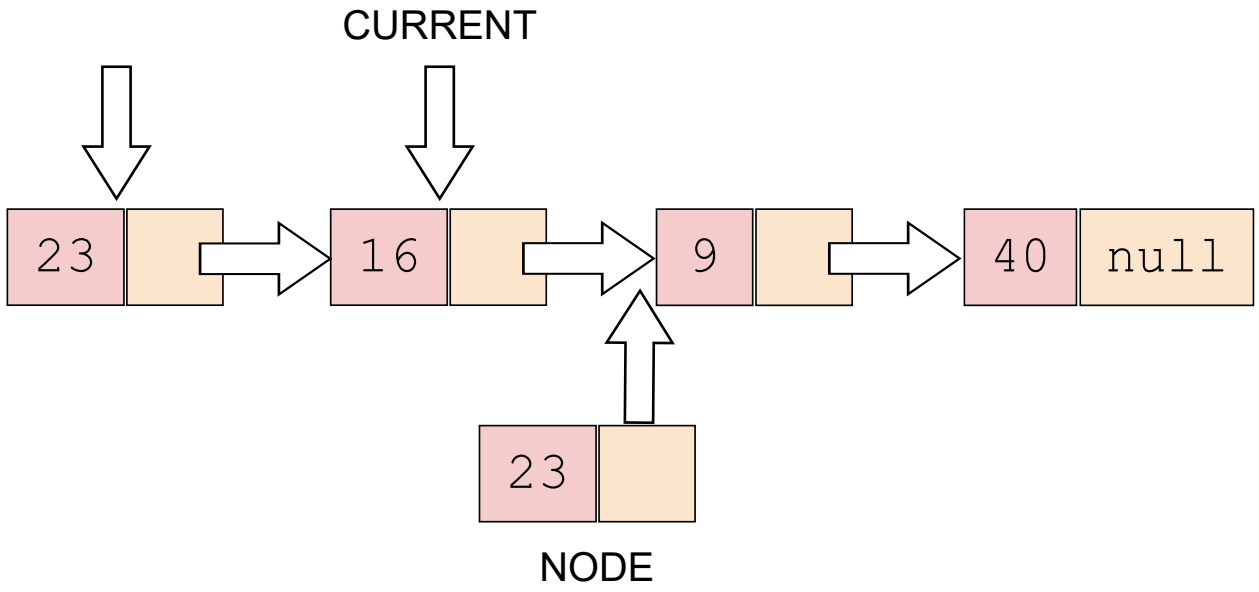
Inserting '23' after '16'

The **insertAfter** function receives two arguments: the data for the new node, and the data of the node after which we want to add the new node. As we start traversing the list again, we set the default value of the currently checked node to the head. While there is a head, meaning that the list isn't empty, we can start walking through the list. If the data of the currently checked node equals the data of the node we wanted to find, in order to add a new node after this node, we create the new node with the data we passed as an argument. If the currently checked node is the tail of the list, meaning that we're just adding a new node to the end of the list, the tail's next value is equal to the new node, and now the new node equals the tail. Else, the new node's next value equals the currently checked node's next value.

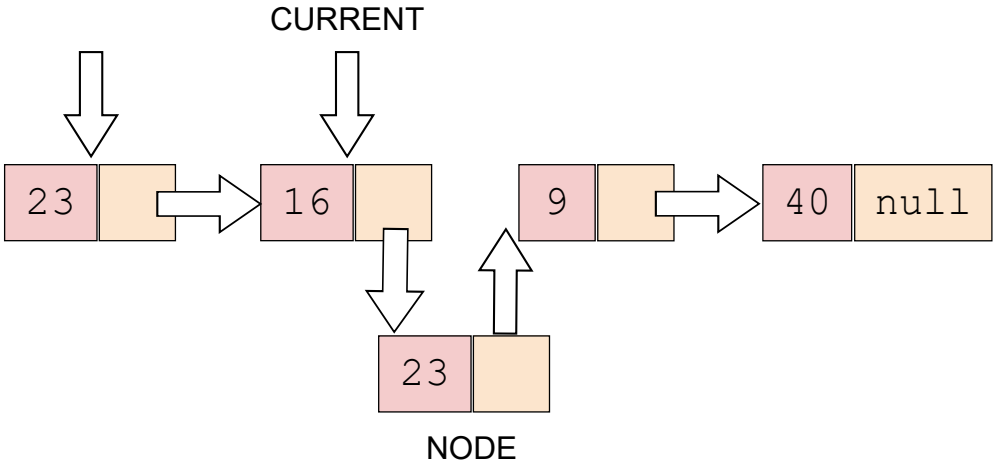


Filter through the list, until you find the node with the right data.

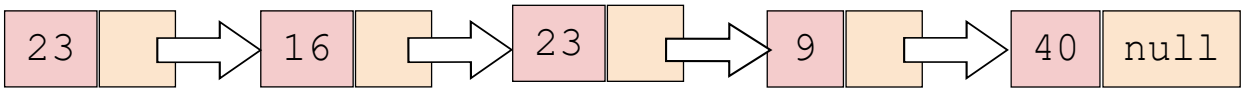
1 of 4



Create a new node, and set the new node's next value equal to the current node next value.



Set the current node's next value equal to the new node.



Now, the new node has been inserted!

4 of 4

The function to **remove** a node is as follows:

```
removeNode(data) {
  let previous = this.head;
  let current = this.head;
  while (current) {
    if (current.data === data) {
      if (current === this.head) {
        this.head = this.head.next;
      }
      if (current === this.tail) {
        this.tail = previous;
      }
      previous.next = current.next;
    } else {
      previous = current;
    }
    current = current.next;
  }
}
```

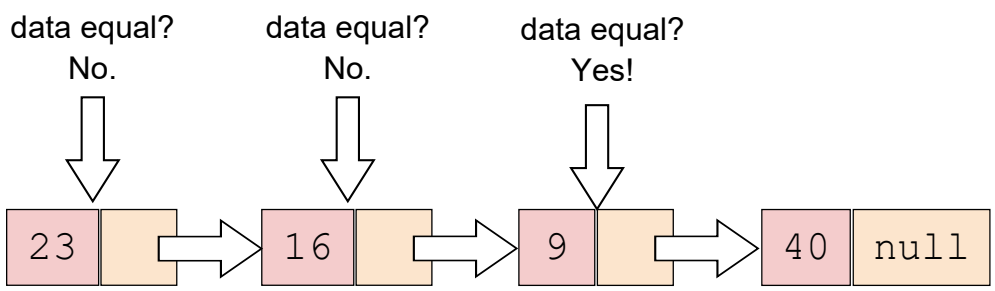
Removing '9' from the List

There are two variables, **previous** and **current**. **current** represents the

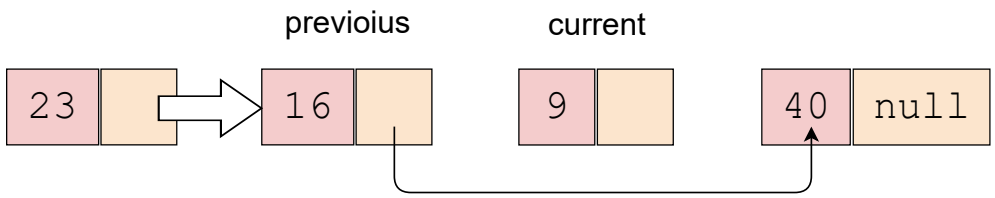
currently checked node, **previous** represents the currently checked node's previous node. To find the node we want to remove, we always start from the

beginning of the list, the head. The values of the previous and current variables are now equal to the head. If there's a current value, meaning that the list consists of at least one node, we start to traverse the list. If the currently checked node's data is equal to the data we want to delete, we found the right node! Now, we need to check where this node is placed. Let's say that we want to remove the node with the data equal to **9**. We find the node, and set the previous node's next value equal to the next node.

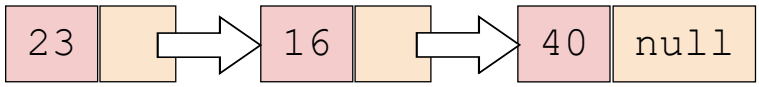
If the node we want to delete is the head of the list, we set the current head's node next value equal to **this.head**.



1 of 3



2 of 3



If we log this linked list, it would look like this:

```
{
   data: 23,
   next: {
     data: 16,
     next: {
        data: 9,
        next: {
          data: 40,
          next: null
        }
     }
  }
}
```

BEFORE DELETION

```
{
   data: 23,
   next: {
     data: 16,
     next: {
        data: 40,
        next: null
     }
  }
}
```

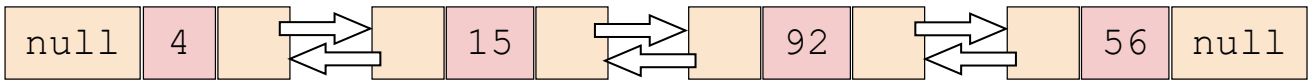
AFTER DELETION

In the next lesson, I will talk about the time complexity of the various functions of a linked list.

Doubly Linked List

A doubly linked list is the same as a singly linked list with the addition of previous pointers that point to the previous node.

In a doubly linked list, each node has two references: one to its previous node, and one to its next node (instead of only the next node in the singly linked list).



The above linked list would look like this:

```
{
   data: 4,
   previous: null
   next: {
      data: 15,
      previous: {
        data: 4,
        previous: null,
        next: { … }
      },
      next: {
         data: 92,
         previous: {
            data: 15,
            previous: { … },
            next: { … }
         },
         next: {
            data: 56,
            previous: {
               data: 95,
               previous: { … },
               next: { … },
            }
            next: null
         }
      }
   }
}
```

In the next lesson, I will talk about the implementation of this data structure.

Doubly Linked List (Implementation)

constructor, add a node at the beginning and in the center, remove a node

First of all, just like a singly linked list, we need to write a constructor function to create new nodes.

```
function Node(data) {
  this.data = data;
  this.next = null;
  this.previous = null;
}
```

By default, the node's previous and next values equal to null.

The linked list will again be a class. This class will have several properties, just like the singly linked list, such as the remove function, add function, and find function. We create a constructor with properties that every list has by default.

```
class DoublyLinkedList {
  constructor() {
    this.head = null;
    this.tail = null;
    this.length = 0;
  }
}
```

Just like the singly linked list, we start off with the tail and head value equal to null: the list doesn't have any nodes yet, until we add them.

The function to **add** a node to the tail is as follows:

```
addNode(data) {
  const node = new Node(data);
  if (!this.head) {
    this.tail = node;
    this.head = node;
  } else {
    node.previous = this.tail;
```

```
    this.tail.next = node;
    this.tail = node;
  }
  this.length++;
}
```

First, we need to create a new node. As the **Node** function is a constructor function, we do this by typing **new Node(data)** with the data passed as an argument to **DoublyLinkedList**. If there is no head in the list, meaning that there are no nodes at all, the new node is both the head and the tail. The list would consist of only the new node!

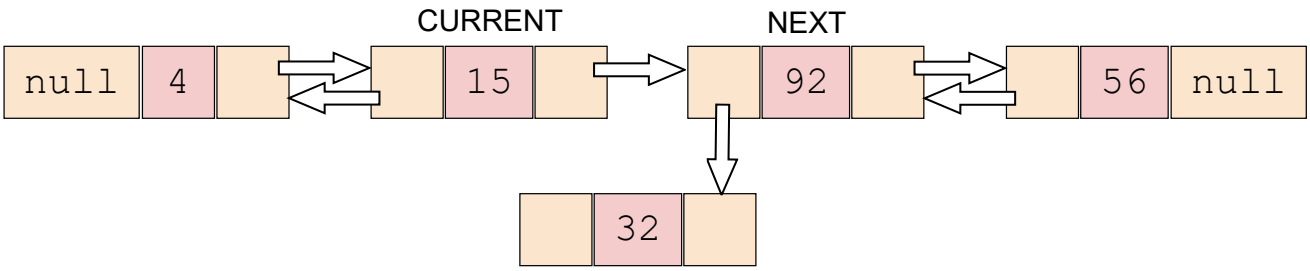
```
const list = new DoublyLinkedList();
list.addNode(23);
```

```
{
   data: 23,
   previous: null,
   next: null
}
```

There's only one node in the list, which represents both the head and the tail. It's almost the same as the singly linked list; the only difference is that we have to define the node's previous value in case the node isn't the head!

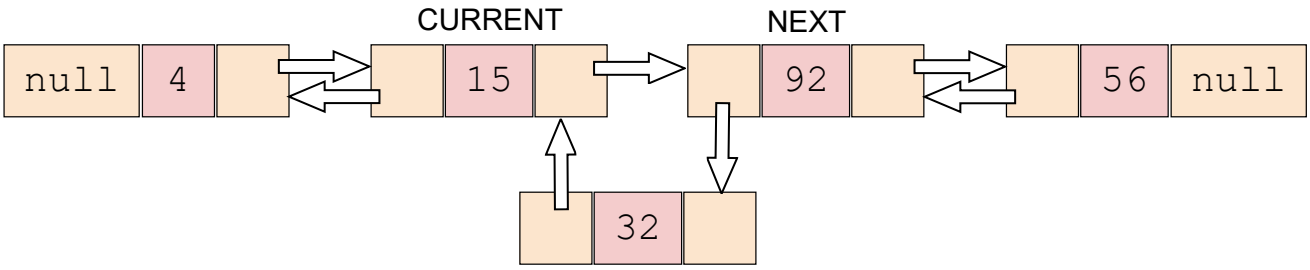
```
insertAfter(data, toNodeData) {
  let current = this.head;
  while (current) {
    if (current.data === toNodeData) {
      const node = new Node(data);
      if (current === this.tail) {
        this.addNode(data);
      } else {
        current.next.previous = node;
        node.previous = current;
        node.next = current.next;
        current.next = node;
        this.length++;
      }
    }
    current = current.next;
  }
}
```

Again, we start traversing the list from the head. While there is a head, meaning that the list isn't empty, we check whether the currently checked node's value is equal to the node after which we want to insert the new node. If we find that node and it happens to be the tail, we invoke the **addNode** function as that function adds nodes to the end of the list, so we won't have to repeat that logic. However, if the node is somewhere in the middle of the list (for example, if **data** is equal to **32** and **toNodeData** is **15**) this is how we handle the situation:



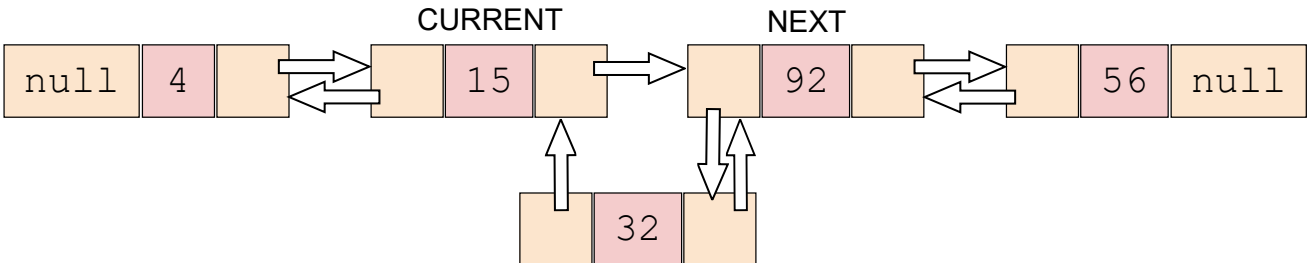
Set the currently checked node's next value's previous value (yes, bear with me) equal to the new node.

1 of 5



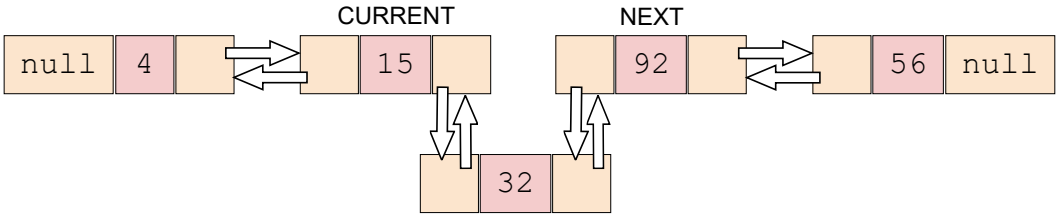
Then, we set the new node's previous value equal to the current node's next value.

2 of 5



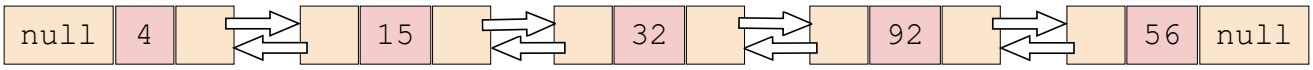
Then, we set the node's next value equal to the current next value.

3 of 5



Then, we set the node's next value equal to the current next value.

4 of 5



The node has been inserted!

5 of 5

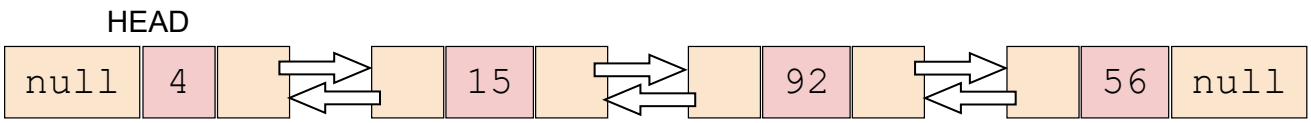
And in the end, we set the current node's next value equal to the new node. We've successfully inserted the new node in the list!

Removing a node:

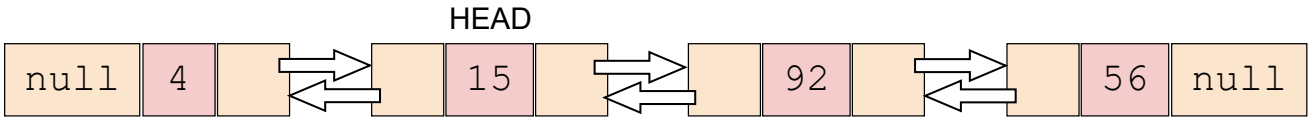
```
removeNode(data) {
  let current = this.head;
  while (current) {
    if (current.data === data) {
      if (current === this.head && current === this.tail) {
        this.head = null;
        this.tail = null;
      } else if (current === this.head) {
        this.head = this.head.next;
        this.head.previous = null;
      } else if (current === this.tail) {
        this.tail = this.tail.previous;
        this.tail.next = null;
      } else {
```

```
        current.previous.next = current.next;
        current.next.previous = current.previous;
      }

      this.length--
    }
    current = current.next;
  }
}
```

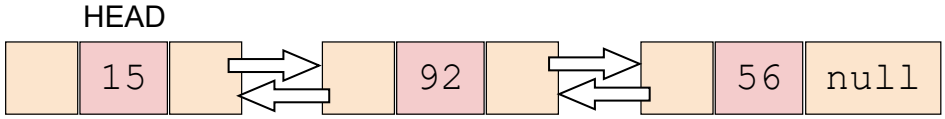
If the node we want to remove is the head, we first set the head's next node equal to the head. Then, we set the new head's previous node equal to null, which results in the deletion of the previous head node.



1 of 3



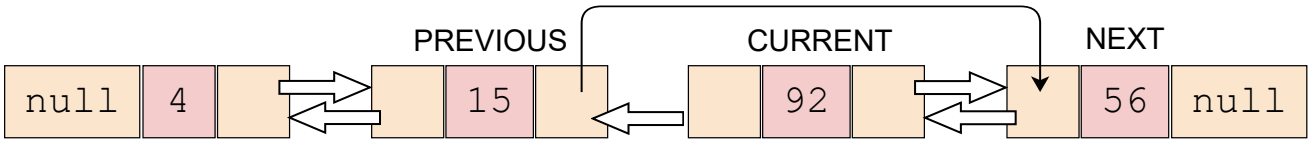
2 of 3



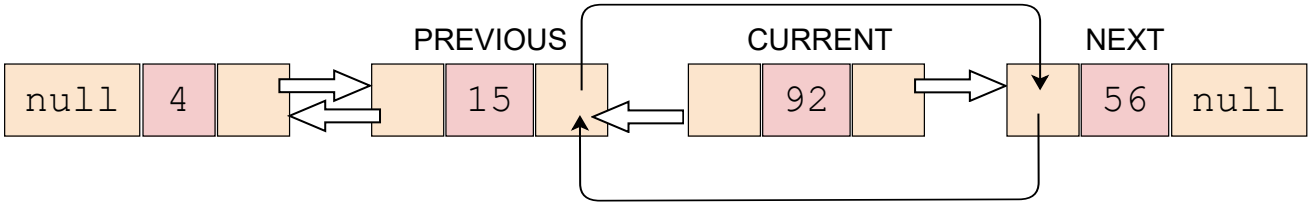
3 of 3

The same logic applies to delete the tail node, just the other way around.

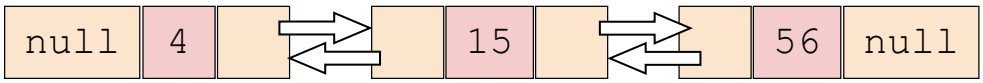
If the node we want to delete is somewhere in the middle, we set the previous node's next value equal to the current's next node. Then, we set the next node's previous value equal to the previous node's next value, which results in the deletion of the current node.



1 of 3



2 of 3



3 of 3

In the next lesson, I will talk about the time complexity of the various functions of singly and doubly linked lists.

Linked List (Time Complexity)

The time complexity for get, search, insertion, and deletion functions of the linked list (Reading time: under 1 minute)

	TIME		SPACE
Type	**Average**	**Worst**	**Worst**
Get, Search, Insertion, Deletion	O(n)	O(n)	O(n)

Get, Search, Insertion and Deletion: To get to a node in the list, we would have to walk through the list to find the node we're searching. It is possible to use pointers instead, which would be constant, but in these examples, the time complexity would be linear.

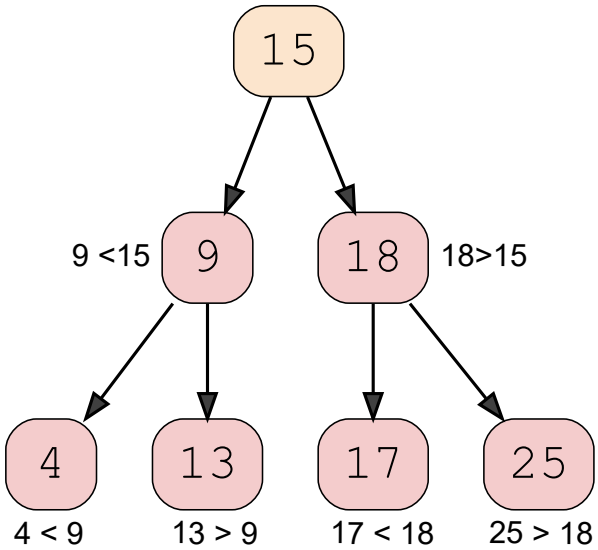
Worst space: The more items, the bigger the list.

In the next lesson, let's talk about another data structure, the binary search tree.

Introduction to Binary Search Tree

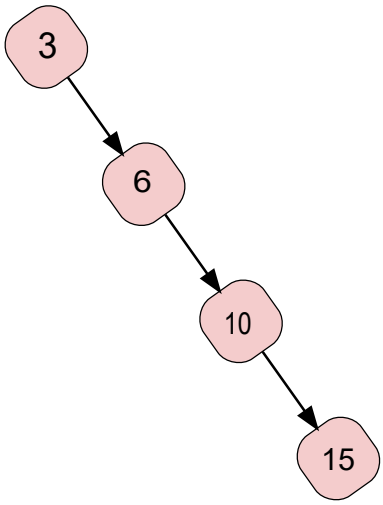
A binary search tree (BST) is a representation of data in a tree structure. (Reading time: 1 minute)

In a binary search tree, the left child node is always smaller than the parent, and the right child is always greater than the parent, i.e., **left child** < **parent node** < **right child**



The node represented with the orange background is called the **root** node. Due to the sorted nature of the tree, we can easily search through the tree: let's say we want to find the value 17. Is it bigger or smaller than 15? Bigger, so we go right. Is 17 smaller or bigger than 18? Smaller, so we go left. And we found the node!

The algorithm to **balance** the tree won't be covered in these notes, I will assume the tree is already balanced. An example of an **unbalanced tree**:



In the next lesson, I'll talk about the implementation of the binary search tree.

Insertion in Binary Search Tree

(Reading time: 4 minutes)

The Javascript function to add a node in the binary search tree is as follows:

```
addNode(data) {
  const newNode = new Node(data);
  if (!this.root) {
    this.root = newNode;
  } else {
    this.insertNode(this.root, newNode);
  }
}

insertNode(node, newNode) {
  if (newNode.data < node.data) {
    if (!node.left) {
      node.left = newNode;
    } else {
      this.insertNode(node.left, newNode);
    }
  } else {
    if (!node.right) {
       node.right = newNode;
    } else {
       this.insertNode(node.right, newNode);
    }
  }
}
```

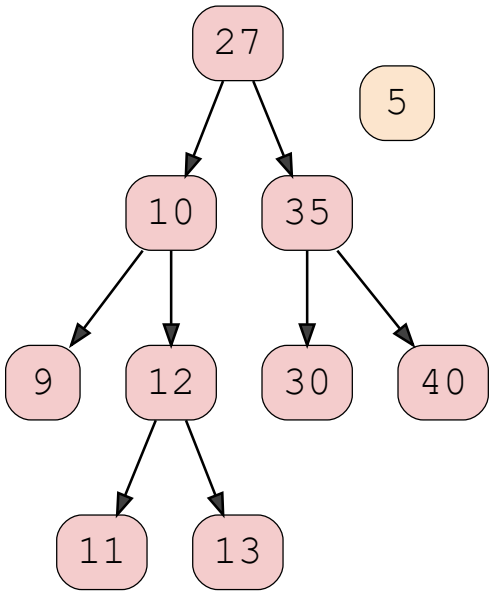
BST after inserting '5'

First, we create our new node. If there is no root in the tree, meaning that there are no nodes at all, the new node is the new root. Otherwise, if the root is already present, we invoke the **insertNode** function that receives two arguments: the root, as we need to check all values in the tree later on, and the new node.

In the **insertNode** function, we check if the currently checked node's data is lower or higher than the new node's data. If it's lower, meaning we go left, we

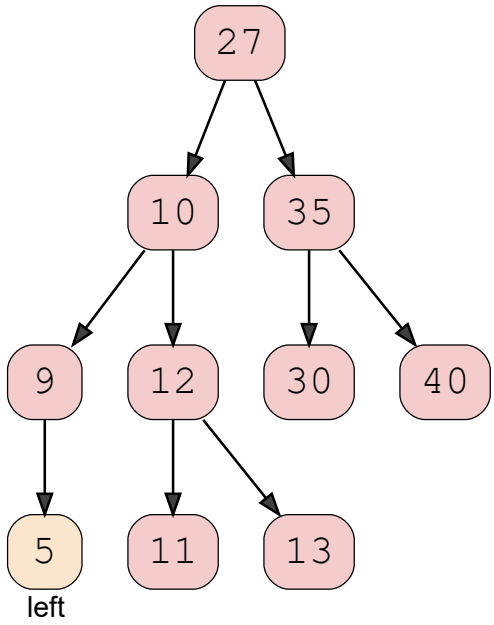
lower or higher than the new node's data. If it's lower, meaning we go left, we first check if there is a value already. If there isn't, the currently checked

node's left value is equal to the new node. Otherwise, we invoke the **insertNode** function again, to keep on walking through the tree until we find the right value. The node's left node is now passed as the first argument, and we check again whether it's value is lower or higher than the new node's value. This keeps on going, until we find the place for the new node. The same goes for the right values, but the other way around! Let's say we want to add a new node with data 5.



There is a root, so the **insertNode** function gets called as `!this.root` returns false. Within this function, we get to the first if-statement: is the data of the new node is smaller than the currently checked node? In this case, 5 is smaller than 27, so it returns true. Then, we get to the second if-statement. Is there no left node? Yes, there is, there is a node on the node's left with the value of 10, so it returns false.

This means that the **insertNode** function gets called again, only this time with the node which data is **10** as the first argument. We again get to the first if-statement, which still returns true. The node already has a left node, so the insertNode function gets called yet again, only this time with the currently checked node's left node as the first argument: the node with the value of **9**. Again, the first if-statement returns true (5 is smaller than 9), but this time, `!node.left` also returns true! The currently checked node's left value is now equal to the new node. This results in:



In the next lesson, I will talk about deleting a node from a binary search tree.

Binary Search Tree (Implementation)

create a binary search tree and create a node in a tree (Reading time: 1 minute)

Before we write the tree, we need to create the constructor function for each node.

```
function Node(data) {
  this.data = data;
  this.left = null;
  this.right = null;
}
```

Every node has a node on its left side, the node smaller than the current node's value, and on its right side, the node bigger than the current node's value.

```
class Tree {
  constructor() {
    this.root = null;
  }
}
```

The tree is a class, which constructor contains the value of the root node. By default, the root's value is equal to null: there are no nodes in the tree until we add them! We will study the function to add a node in the next lesson.

Deletion in Binary Search Tree

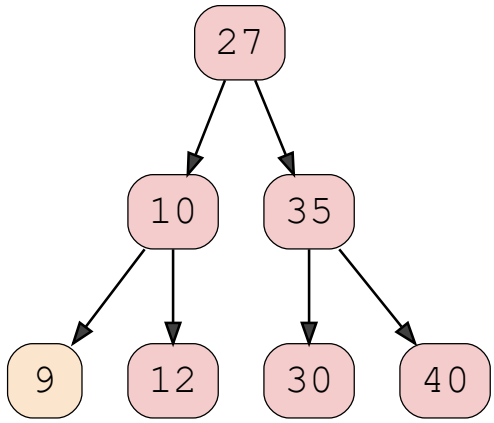
(Reading time: 5 minutes)

The function to remove a node from a binary search tree is as follows:

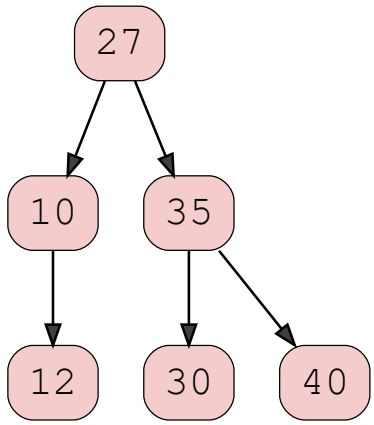
```
remove(data) {
  this.root = this.removeNode(this.root, data)
}

removeNode(node, data) {
  if (!node) {
    return null;
  }
  if (data < node.data) {
    node.left = this.removeNode(node.left, data);
    return node;
  } else if (data > node.data) {
    node.right = this.removeNode(node.right, data);
    return node;
  } else {
    if (!node.left && !node.right) {
      node = null;
      return node;
    }
    if (!node.left) {
      node = node.right;
      return node;
    }
    if (!node.right) {
      node = node.left;
      return node;
    }

    let min = this.findMinNode(node.right);
    node.data = min.data;
    node.right = this.removeNode(node.right, min.data);
    return node;
  }
}
```

BST after removing '9'

Removing a leaf



1 of 2



2 of 2

First, we invoke the **remove** function. We set the root's value equal to whatever gets returned from the **removeNode** function. We pass two arguments to this function: the root, and the node's value that we want to delete, 9 in this case.

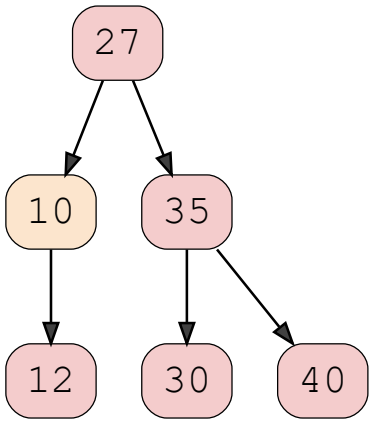
Inside the **removeNode** function, we get to the first if-statement. Nodes are present in the tree, so **!node** returns false. We get to the second if-statement, which returns true as data (9) is smaller than **node.data** (27). Now, we set the left node's value equal to whatever gets returned from the **removeNode** function, which we invoke again, only now with the value of the left node (10) as the first argument.

Again, we get to the first if-statement, which returns false. The second if-statement returns true again, as **data** (9) is smaller than **node.data** (10). Again, we set the left node's value equal to whatever gets returned from the

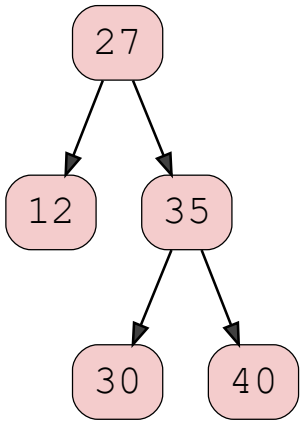
Again, we set the left node's value equal to whatever gets returned from the **removeNode** function, with the new left node (9) as the first argument.

The second if-statement now returns false, because **data** (9) is not smaller than **node.data** (9). The else-if statement also returns false, because 9 is not bigger than 9. We get into the else block, where we get our first if-statement. This one, **!node.left** && **!node.right**, returns true: the node is a leaf and doesn't have a left and right node. We set the node equal to null: the node gets deleted.

Removing a value with **one child** node:



1 of 2



2 of 2

First, we invoke the **remove** function. We set the root's value equal to whatever gets returned from the **removeNode** function. We pass two arguments to this function: the root, and the node's value that we want to delete, 10 in this case.

Inside the **removeNode** function, we get to the first if-statement. There are

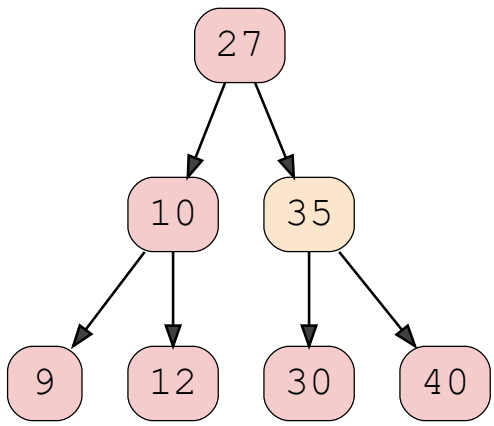
Inside the **removeNode** function, we get to the first if-statement. There are nodes, so **!node** returns false. We get to the second if-statement, which returns

true as **data** (10) is smaller than **node.data** (27). Now, we set the left node's value equal to whatever gets returned from the **removeNode** function, which we invoke again, only now with the value of the left node (10) as the first argument.

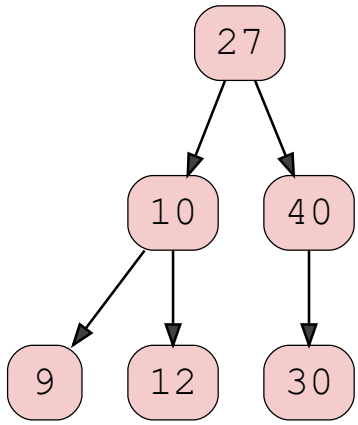
Again, we get to the first if-statement, which returns false. The second if-statement returns false, as **data** (10) is not smaller than **node.data** (10). Also, the else-if statement returns false, as 10 is not bigger than 10. We get into the else block, where we get to the first if-statement. This one, **!node.left** && **!node.right** returns false, as the node we want to delete has a node.right, namely the one with the value 12. The second if-statement in the else-block, !node.left, returns true: there is no left node! We now replace the current node with its node on the right. The node has now been deleted and replaced.

Removing a value with **two child** nodes:

EXAMPLE 1:



1 of 2



2 of 2

First, we invoke the **remove** function. We set the root's value equal to whatever gets returned from the **removeNode** function. We pass two arguments to this function: the root, and the node's value that we want to delete, 35 in this case.

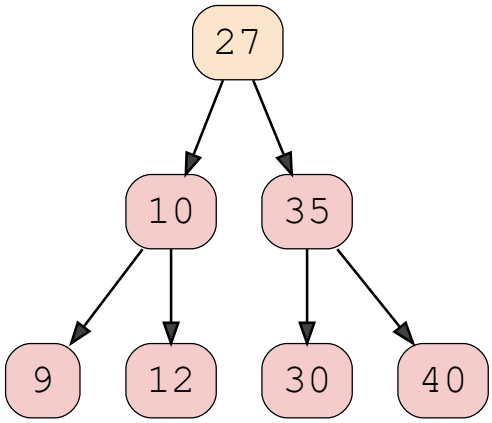
Inside the **removeNode** function, we get to the first if-statement. There are nodes, so **!node** returns false. We get to the second if-statement, which returns false as **data** (35) is bigger than **node.data** (27). Now, we set the right node's value equal to whatever gets returned from the **removeNode** function, which we invoke again, only now with the value of the right node (35) as the first argument.

Again, we get to the first if-statement, which returns false. The second if-statement returns false as well, as **data** (35) is not smaller than **node.data** (35). Also, the else-if statement returns false, as 35 is not bigger than 35. We get into the else block, where all if-statements return false: there is a left node and a right node. This means that we get to the last part:

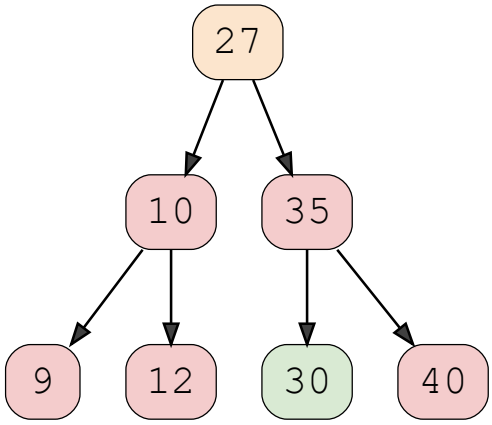
```
let min = this.findMinNode(node.right);
node.data = min.data;
node.right = this.removeNode(node.right, min.data);
return node;
```

The **findMinNode** is a function we haven't implemented yet: however it finds the minimum value of the right subtree. It needs to be the right subtree because the minimum value means that it's at least always bigger than every value on the left subtree, and always smaller than any value on the right subtree! After we find that minimum node, we replace the current node with the minimum node, by setting its value equal to the minimum node's value. We then remove the node that's on the right, as we replaced the current node with that node (otherwise they would be duplicated). The node has now been deleted and replaced!

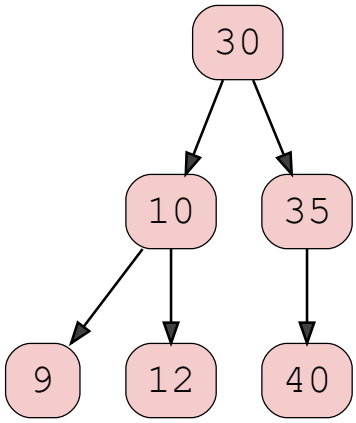
EXAMPLE 2:



1 of 3



2 of 3



3 of 3

As the node we want to delete is the root node, we immediately go to the last part. The minimum value on the right subtree is 30, so the current node gets replaced with the minimum node's value.

In the next lesson, I'll discuss traversal of a binary search tree.

Traversing a Binary Search Tree

inorder, preorder and postorder traversal (Reading time: 2 minutes)

There are three ways to traverse (go through) a binary search tree:

Inorder

- Left subtree
- Root node
- Right subtree

The inorder way is important if you want to flatten the tree back into its original sequence.

```
inOrder(node) {
   if (node) {
      this.inOrder(node.left);
      console.log(node.data);
      this.inOrder(node.right);
   }
}
```

Preorder

- Root node
- Left subtree
- Right subtree

The preorder way is important if you need to inspect roots before inspecting the leaves.

```
preOrder(node) {
   if (node) {
      console.log(node.data);
      this.preOrder(node.left);
```

```
        this.preOrder(node.right);
    }
}
```

Postorder

- Left subtree
- Right subtree
- Root node

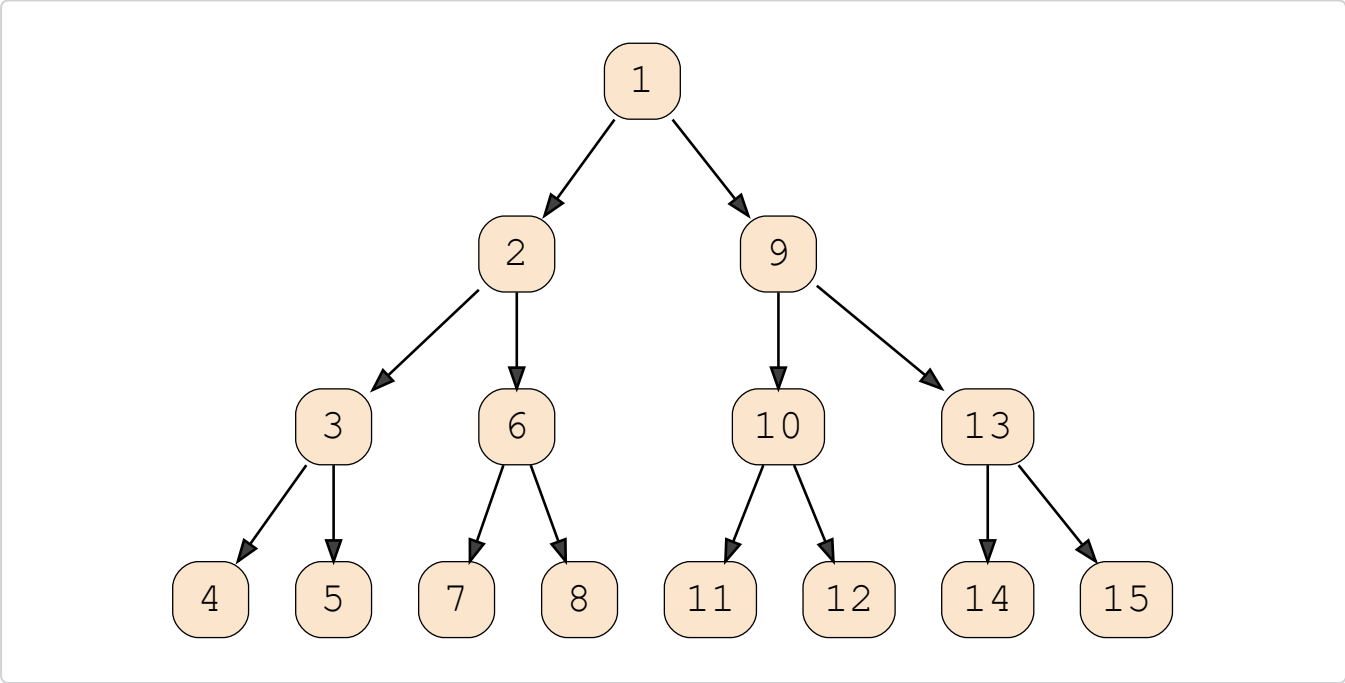
The postorder way is important if you want to delete an entire tree, or simply want to inspect the leaves before inspecting the nodes. If you deleted the root node, you wouldn't be able to delete the nodes in the right subtree!

```
postOrder(node) {
    if (node) {
        this.postOrder(node.left);
        this.postOrder(node.right);
        console.log(node.data);
    }
}
```

Now, lets move on to the breadt-first and depth-first traversals of a BST.

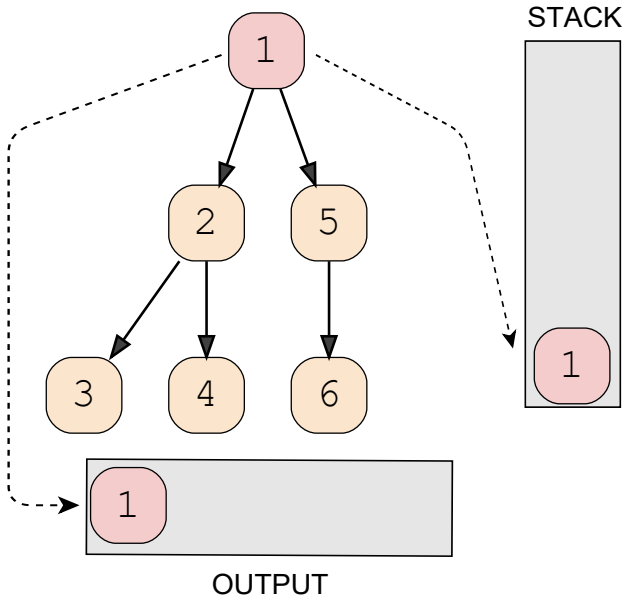
Depth First Traversal

As the name suggests, depth-first traversal involves traversing a tree from top to bottom. (Reading time: 2 minutes)

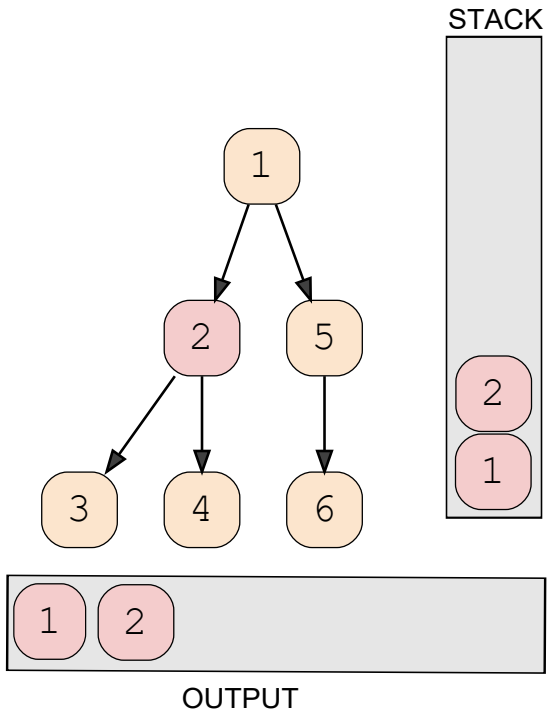


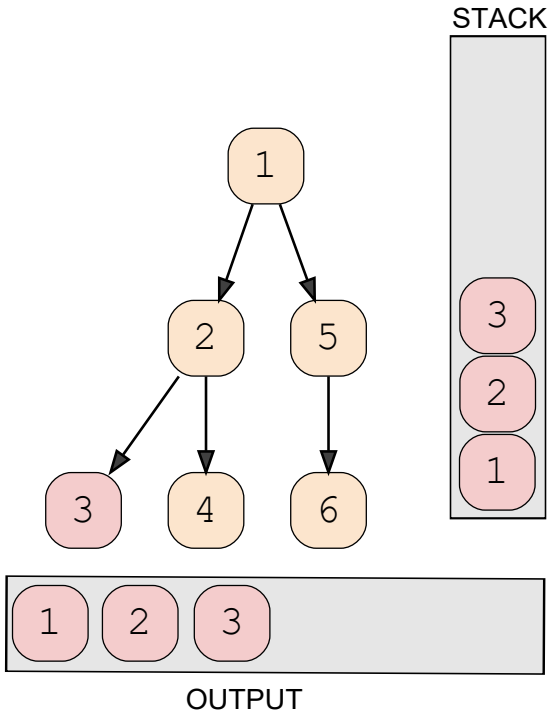
In depth-first traversal, we first traverse through the **left subtrees**, and then the **right subtrees**! The values within the nodes in this example show the order, not their values!

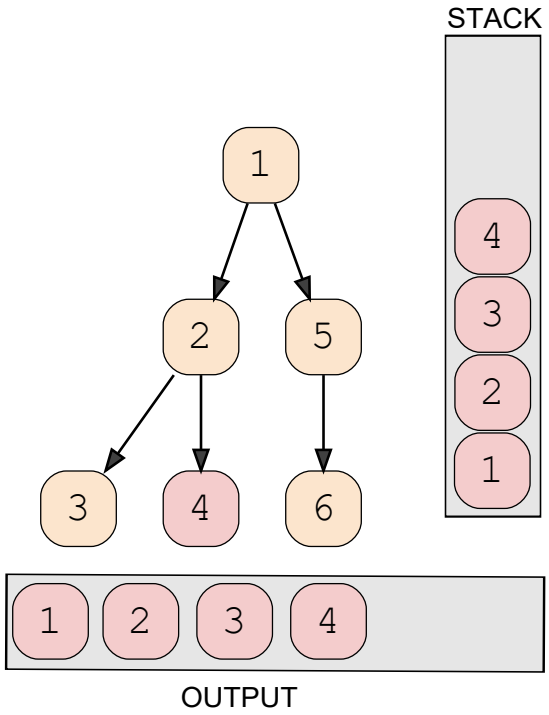
Depth-first traversal uses a **stack** data structure. The stack keeps track of all the visited nodes! However, the stack is implemented implicitly.

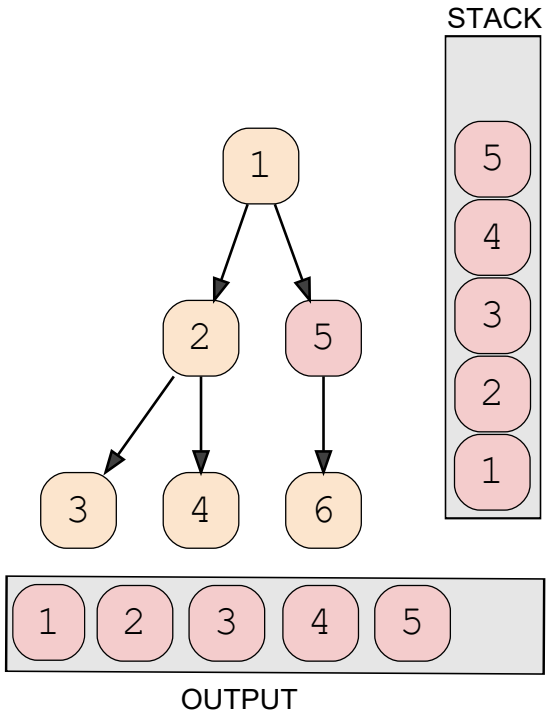


We visit the first node. The node has the value **1**, so that value gets pushed to the stack and added to the output sequence. This node has other nodes, so that value doesn't get popped off the stack just yet. Instead, we go to the next value.

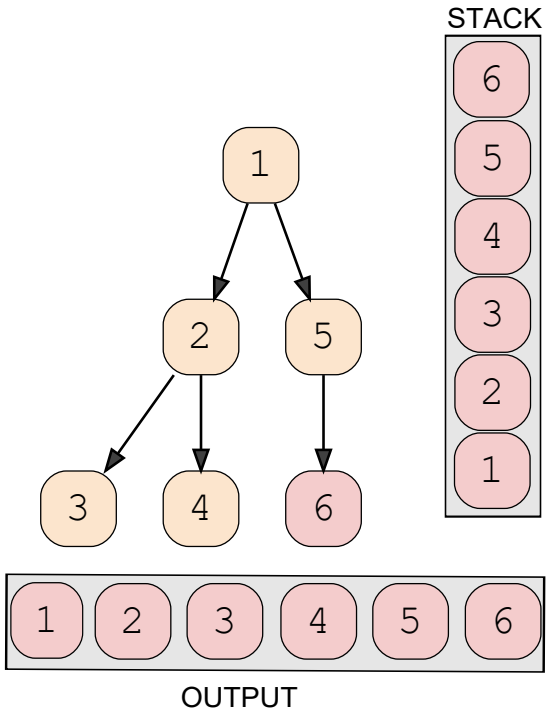



STACK
1
2
5
3
4
6
3
2
1
1
2
3
OUTPUT


STACK
1
2
5
3
4
6
4
3
2
1
1
2
3
4
OUTPUT



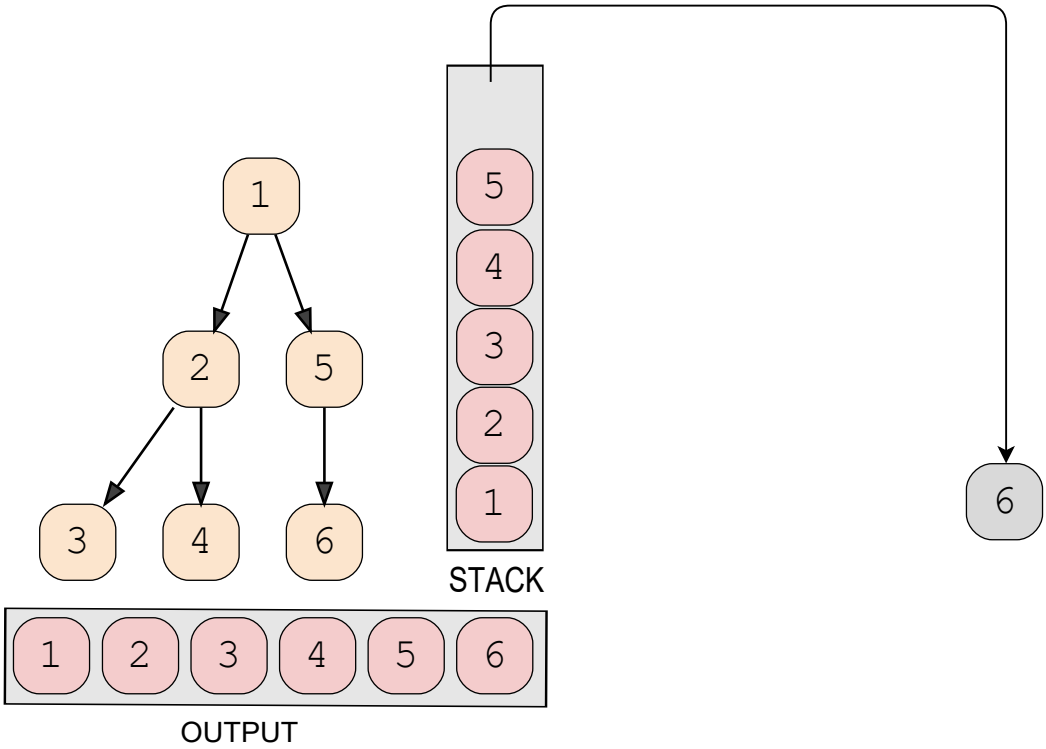
4 of 5



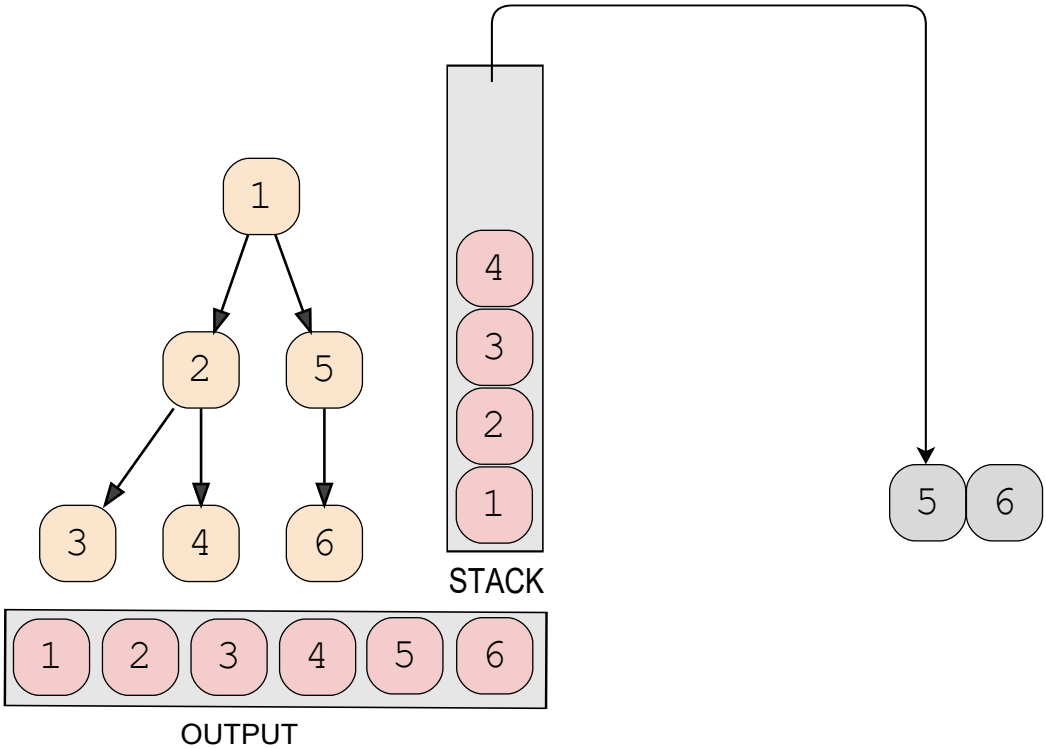
5 of 5

We reached the end of the tree: there are no other nodes to visit! Now, the values will be popped off the stack until the stack is **empty**. This is the sign

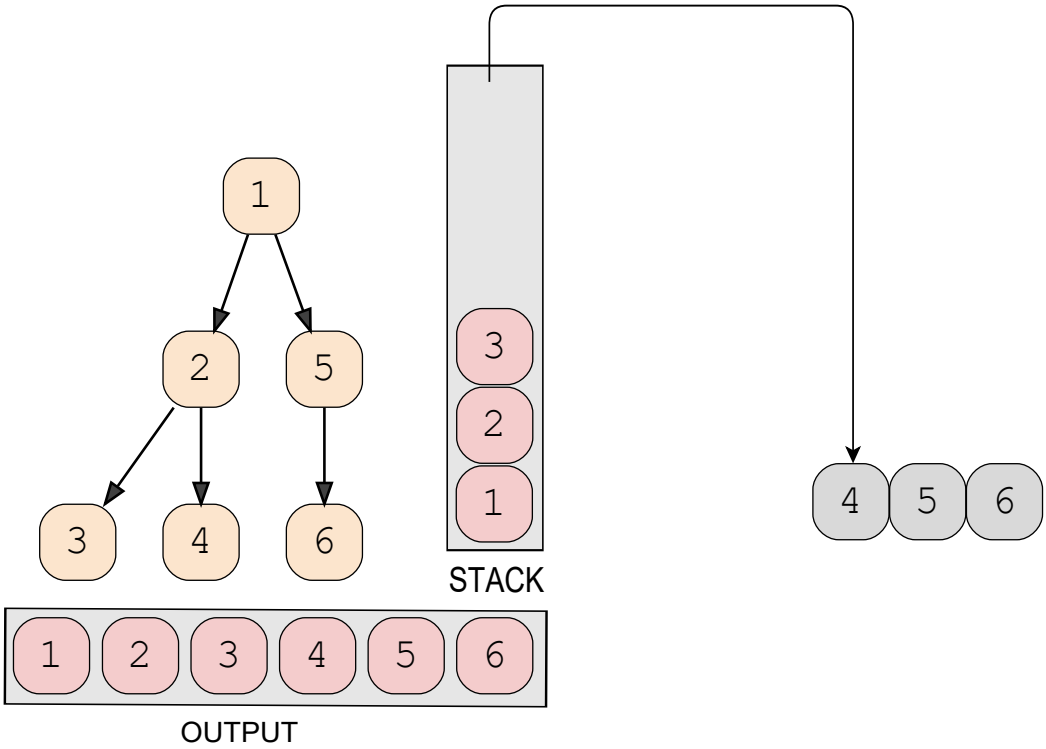
that we have finished traversing through the tree, and we have the complete output!

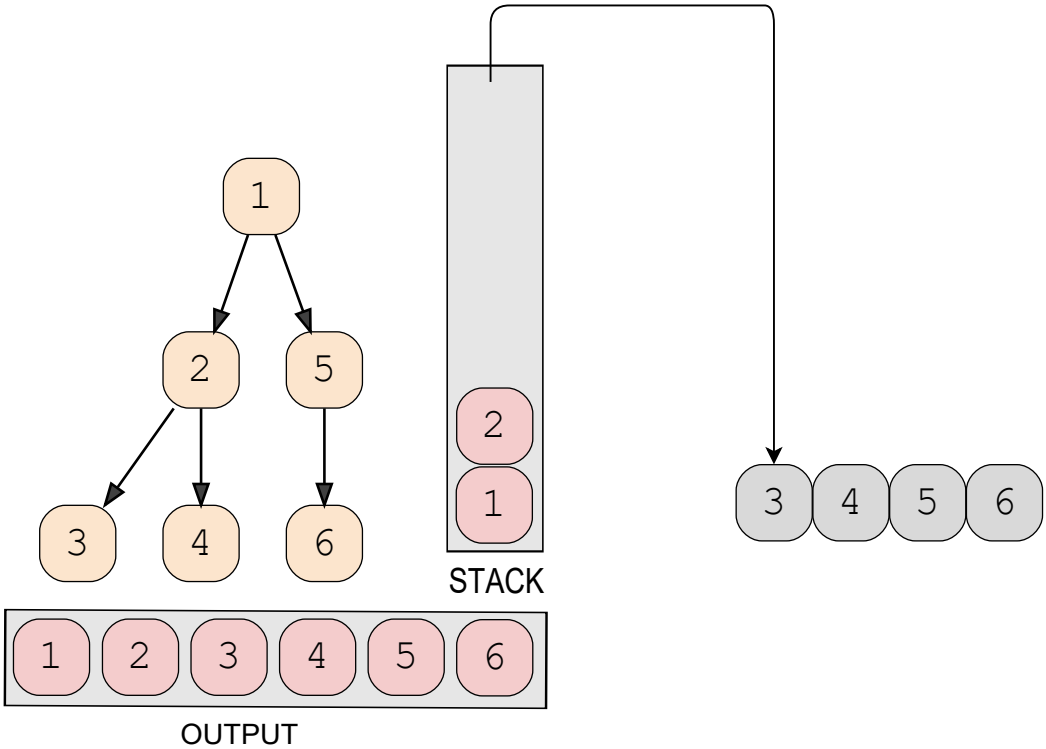


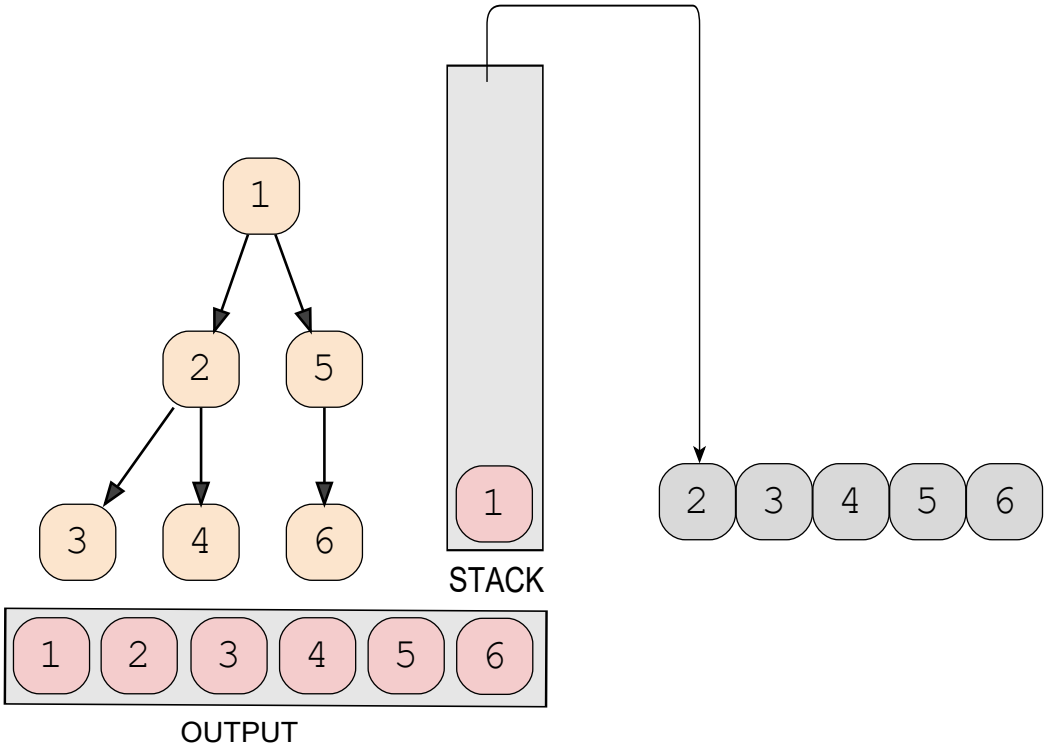
1 of 6



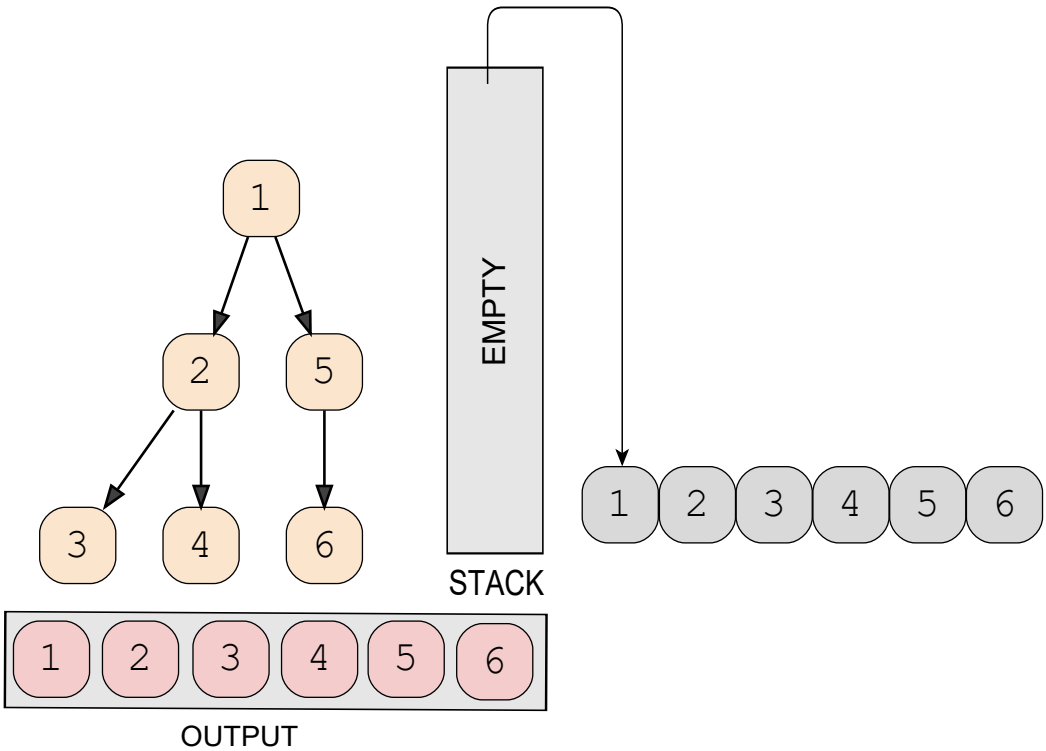
2 of 6


1
2
5
3
4
6
3
2
1
STACK
4
5
6
1
2
3
4
5
6
OUTPUT


1
2
5
3
4
6
2
1
STACK
3
4
5
6
1
2
3
4
5
6
OUTPUT



5 of 6

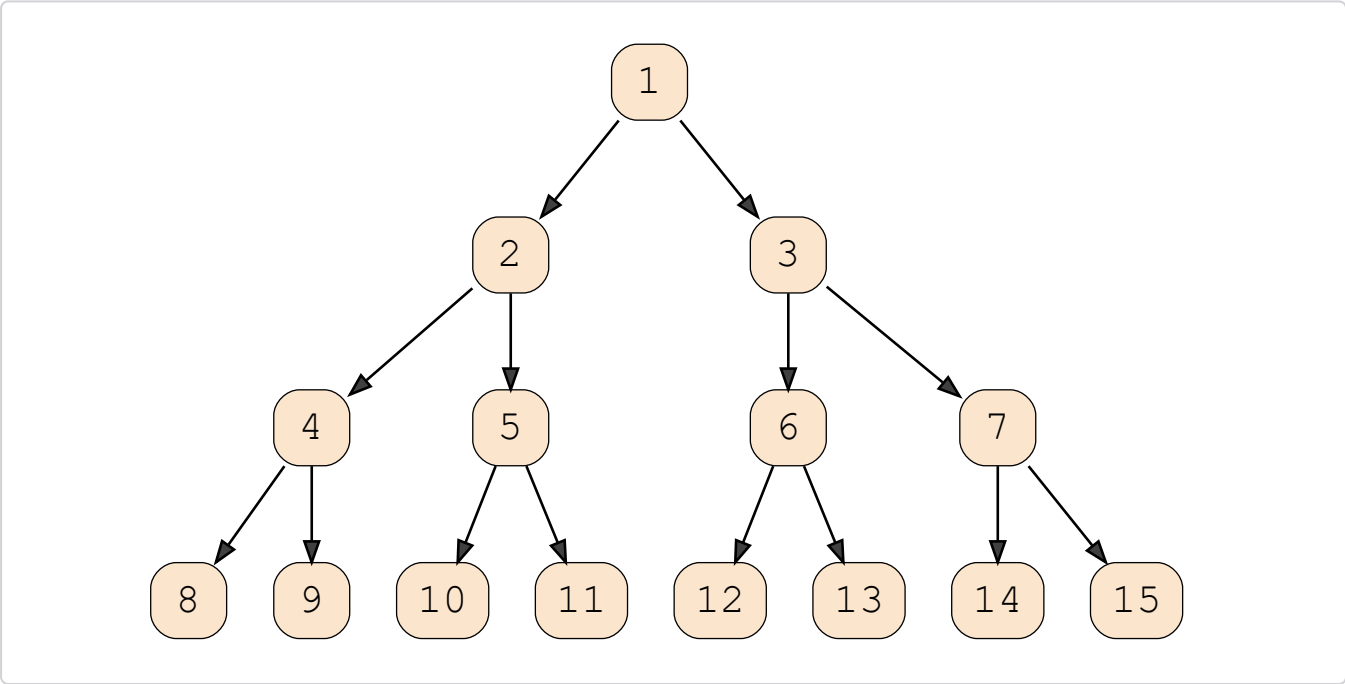


6 of 6

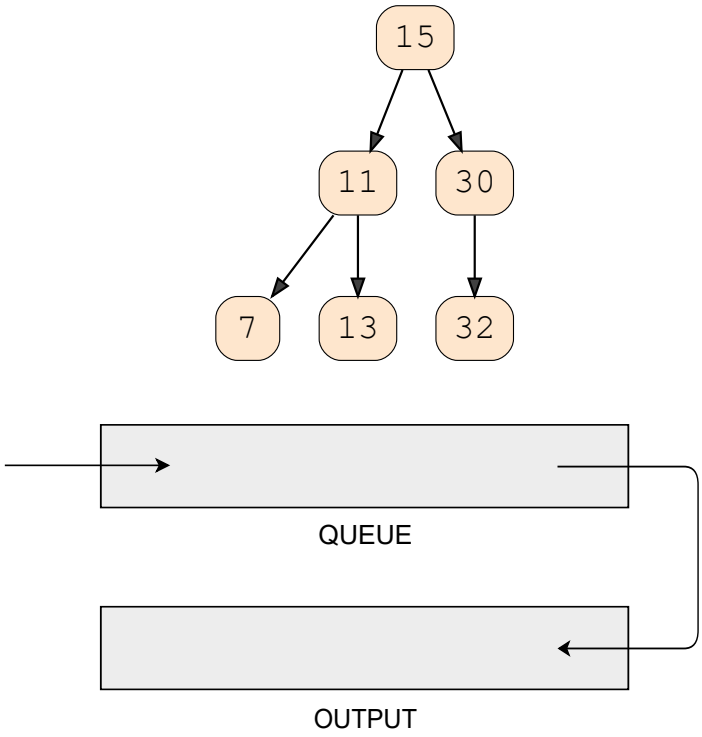
Now, let's move on to breadth-first traversal.

Breadth-first Traversal

In breadth-first traversal, we traverse the binary search tree from left to right. (Reading time: under 5 minutes)

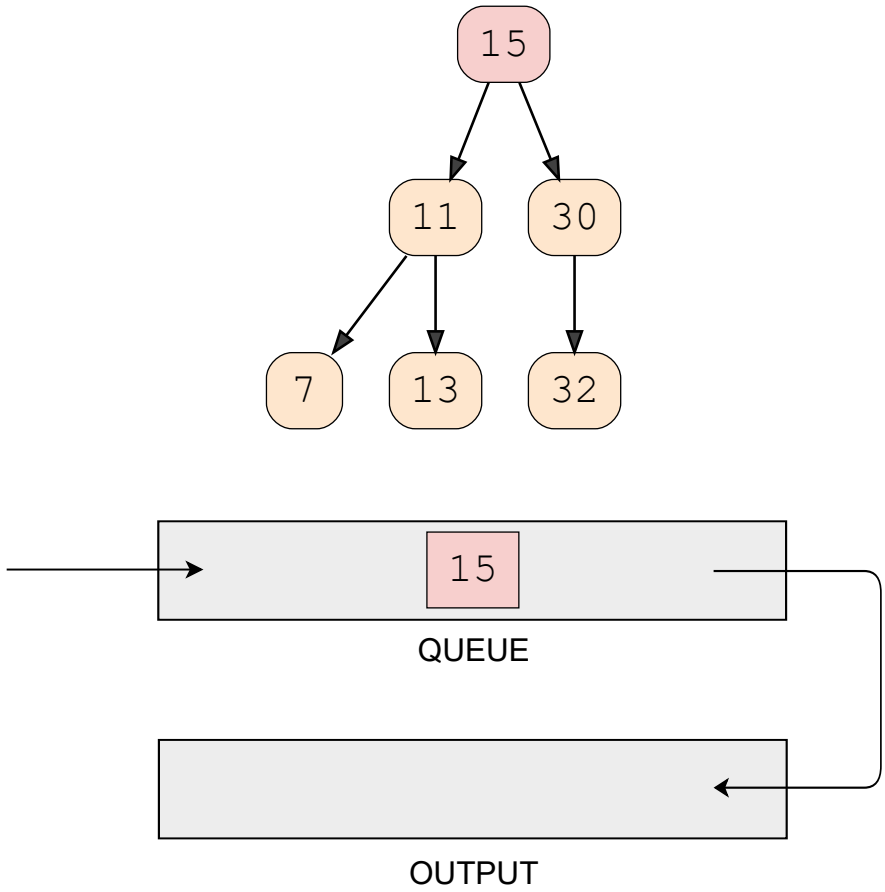


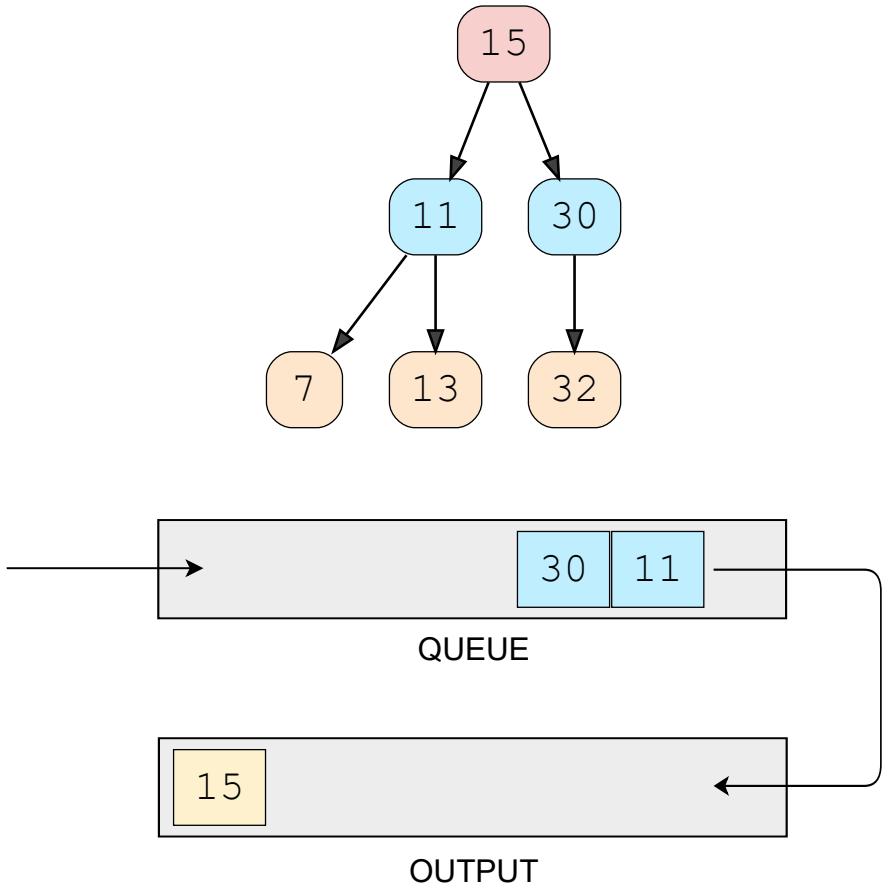
With breadth-first traversal, we first traverse through one **level of children nodes**, then through one level of grandchildren nodes, then through one level of grand-grandchildren nodes, etc. In the above tree, the numbers show the sequence in which the nodes will be traversed, not the data! With breadth-first search, we can't use a stack like we could with depth-first search: when we go from node 1 to node 2, node 2 doesn't have any connection to node 3! Instead of a stack, breadth-first traversal uses a **queue**. Queues allow us to store a reference to the nodes that we want to visit in the future, but haven't visited yet!



With breadth-first search, we want to go from 15, to 11, to 30, to 7, to 13, to 32 in this case. The currently visited node will be pushed to the queue. Once that node has been pushed to the output, its child nodes are also pushed to the queue!

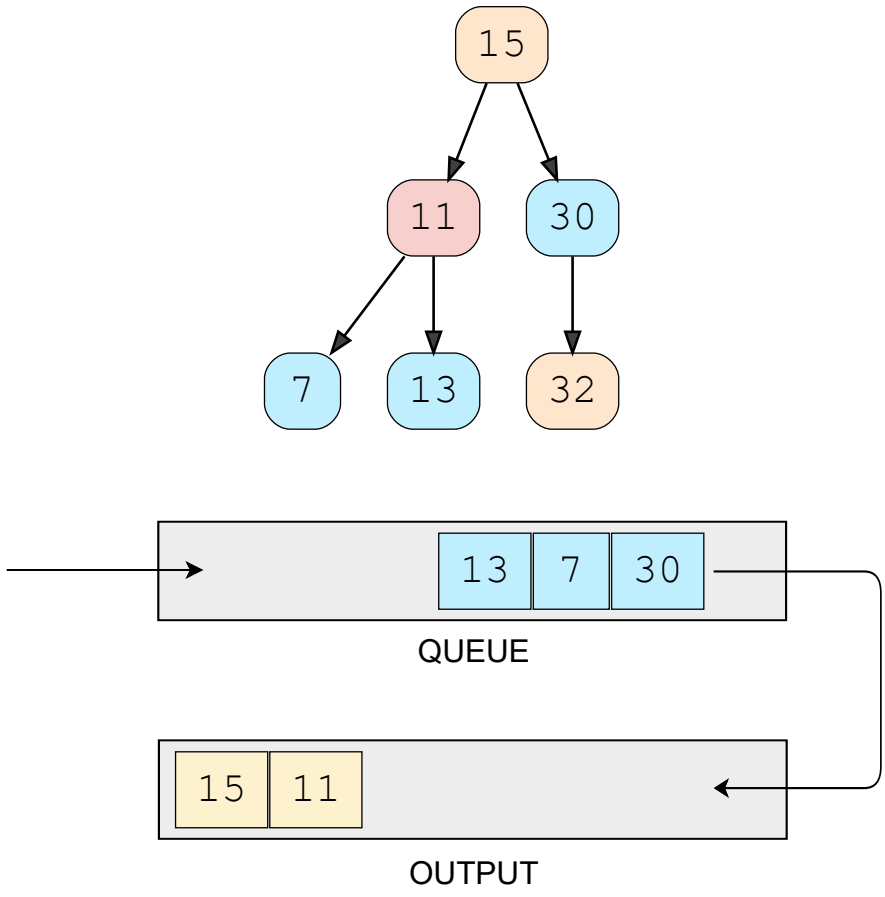
Now let's consider the above example in a bit more detail.

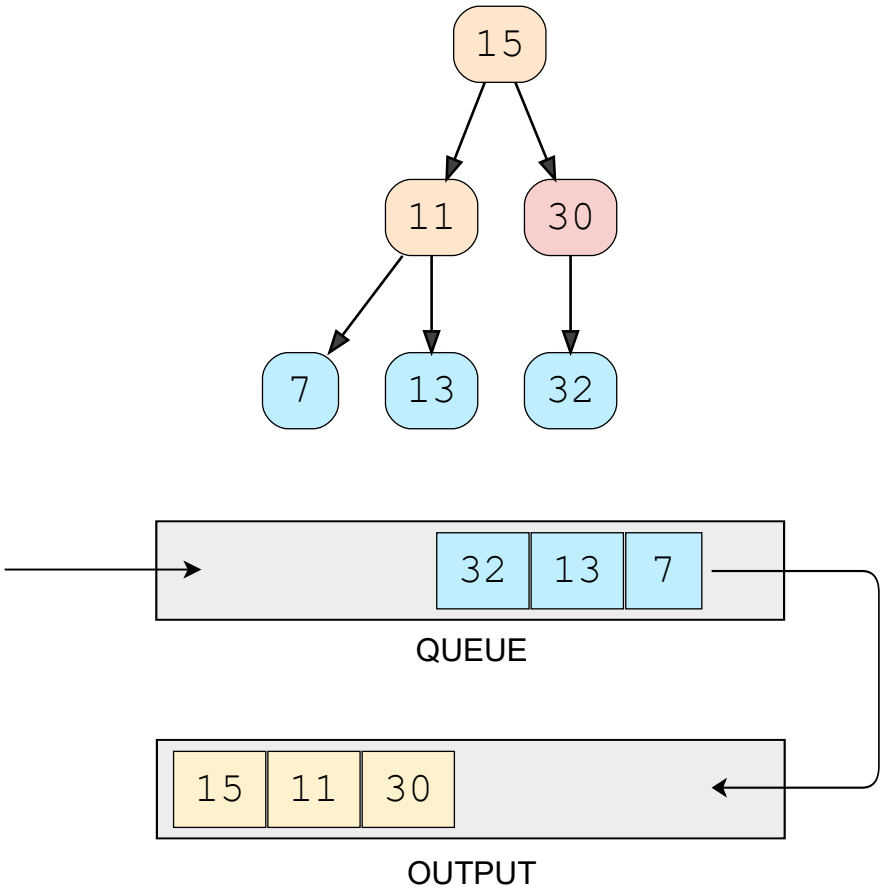

15
11
30
7
13
32
15
QUEUE
OUTPUT

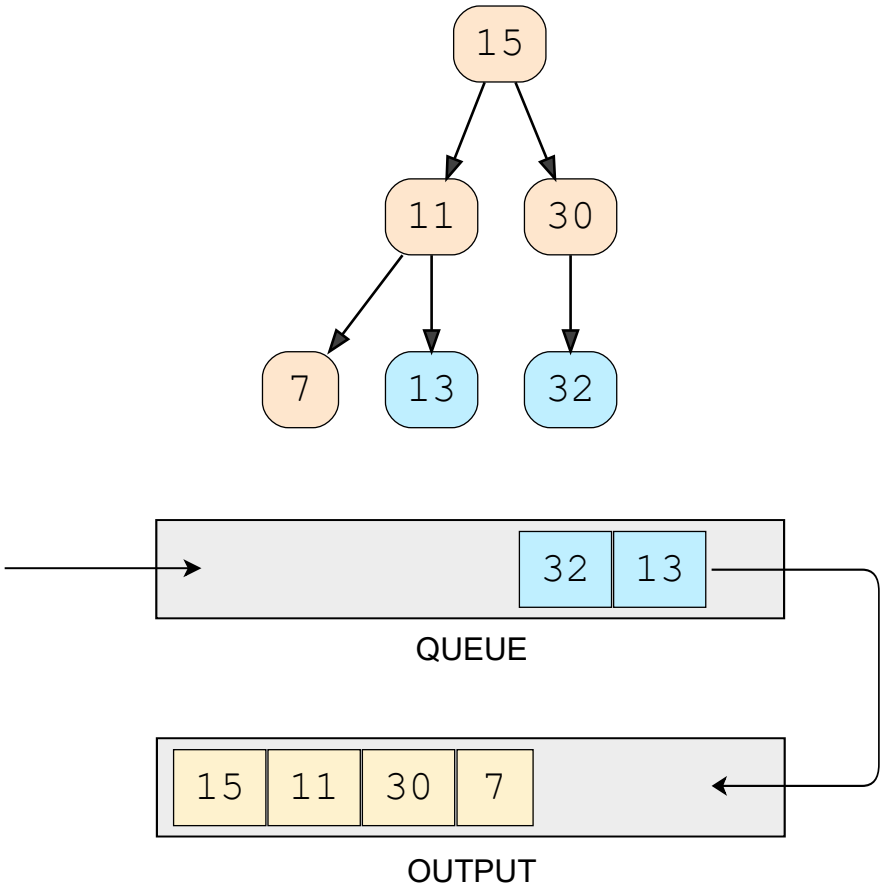

15
11
30
7
13
32
30
11
QUEUE
15
OUTPUT

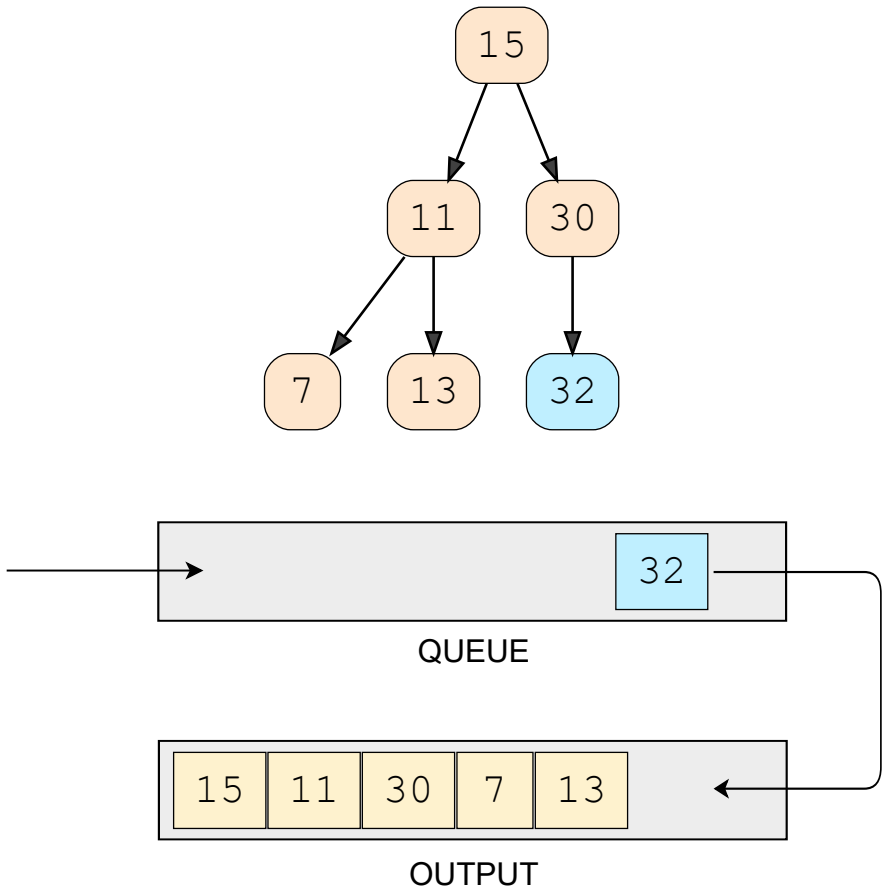
The node with the value **15** is first put in the queue, but immediately goes to the output. Now that it is in output, its child nodes get pushed to the queue.

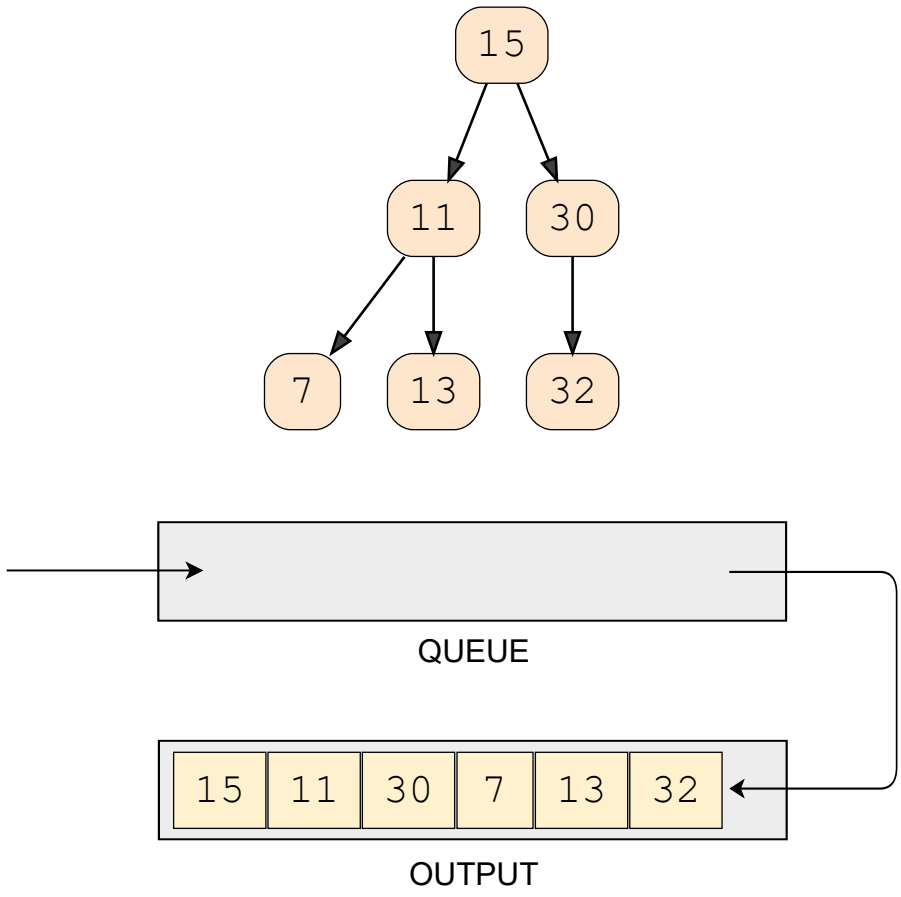
The first one in the queue gets added to the output sequence, and its child nodes get pushed to the queue. This keeps on going until we've reached the end of the tree!




15
11
30
7
13
32
32
13
7
QUEUE
15
11
30
OUTPUT


15
11
30
7
13
32
32
13
QUEUE
15
11
30
7
OUTPUT


15
11
30
7
13
32
32
QUEUE
15
11
30
7
13
OUTPUT


15
11
30
7
13
32
QUEUE
15
11
30
7
13
32
OUTPUT

Now let's write the code for breadth-first traversal of a tree:

```
traverseBFS() {
  if (!this.root) return;
  this.queue = [];
  this.queue.push(this.root);
  this.output = [];

  while (this.queue.length) {
    const node = this.queue.shift();
    if (node.left) {
       this.queue.push(node.left);
    }
    if (node.right) {
       this.queue.push(node.right);
    }
    this.output.push(node.data);
  }
  return this.output;
}
```

First, we need to check whether the tree has nodes at all. If that's not the case, we can't traverse anything, so we return from the function. Then, we initialize the queue. The first node that needs to be pushed to the queue, is the root. This means that the queue has a length, and the while condition returns true. We declare a variable called node, and set it equal to the last item in the queue, which now gets removed from the queue. Does this item have a node on the left? If yes, then that item gets pushed to the queue, the same logic gets repeated for its right node. The node that got removed from the queue gets returned!

```
15
11
30
7
13
32
```

A successful breadth-first traverse!

Finding Minimum and Maximum Value

Finding the minimum and maximum value in a binary search tree is rather easy, as you always know that the left subtree's values are lower than the current node, and the right subtree's values are higher than the current node.

```
getMin() {
  let node = this.root;
  while (node.left) {
      node = node.left;
  }
  return node.data;
}
```

Find the minimum

```
getMax() {
  let node = this.root;
  while (node.right) {
      node = node.right;
  }
  return node.data;
}
```

Find the maximum

From the above codes, it is evident that we keep on going left/right to find the absolute minimum/maximum value, and return that node's value!

In the next lesson, I will discuss the time complexity of the various binary search tree functions.

Binary Search Tree (Time Complexity)

time complexity for the get, search, insertion and deletion functions of the binary search tree (Reading time: under 1 minute)

	TIME		SPACE
Type	**Average**	**Worst**	**Worst**
Get, Search, Insertion, Deletion	O(log(n))	O(n)	O(n)

Average:

The number of items is split in half, as we decide whether to go for the left subtree, or right subtree.

Worst:

If the tree is very unbalanced, it resembles a linked list.

Worst space:

The more items, the bigger the data list.

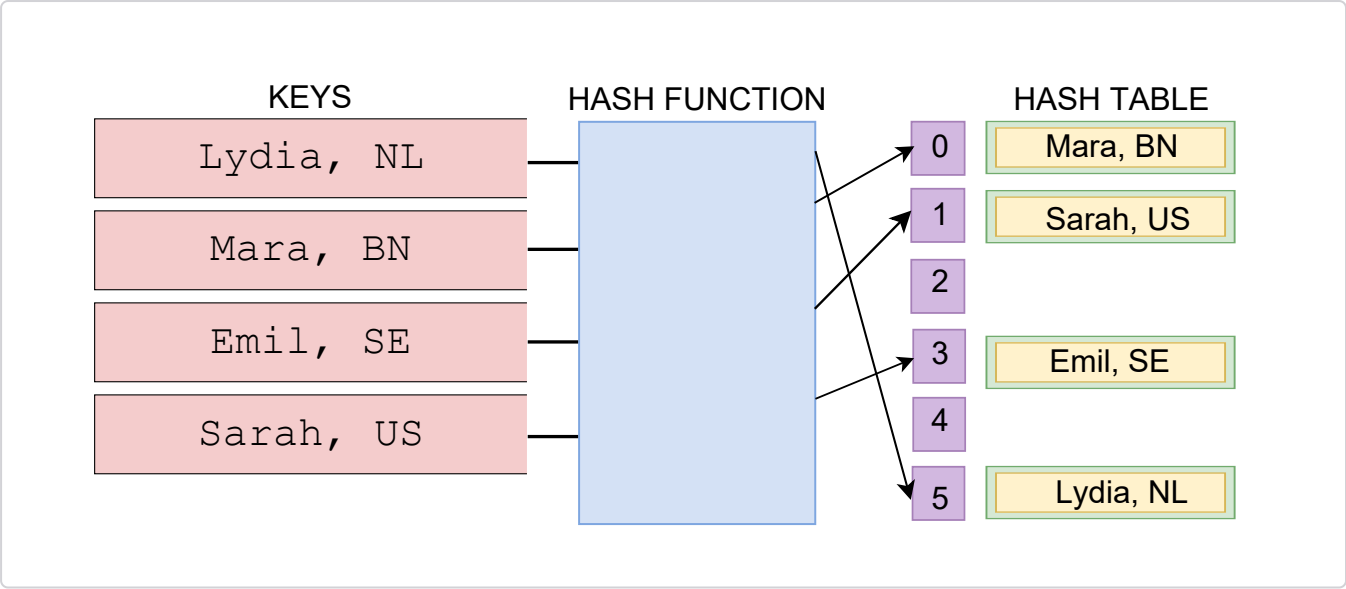
Now, let's move on to another data structure, the hash table.

Introduction to Hash Table

A hash table is a data type that can map keys to values. (Reading time: under 3 minutes)

Hash tables are very efficient. Let's say that we want to look up a specific person in an array: we would have to walk through every item to look for that person! The space complexity would be O(n), as space depends on the size of the array.

To look things up way more efficiently, you can use hash tables! Hash tables are made up of two parts: an **object** with the table where the data will be stored, and a **hash function** (or mapping function). This function creates a mapping by assigning the input data to a very specific index within the array! This function takes a key, and **always returns the same index for the same key**! If we would run the same key through the function twice, it gives us the same index.



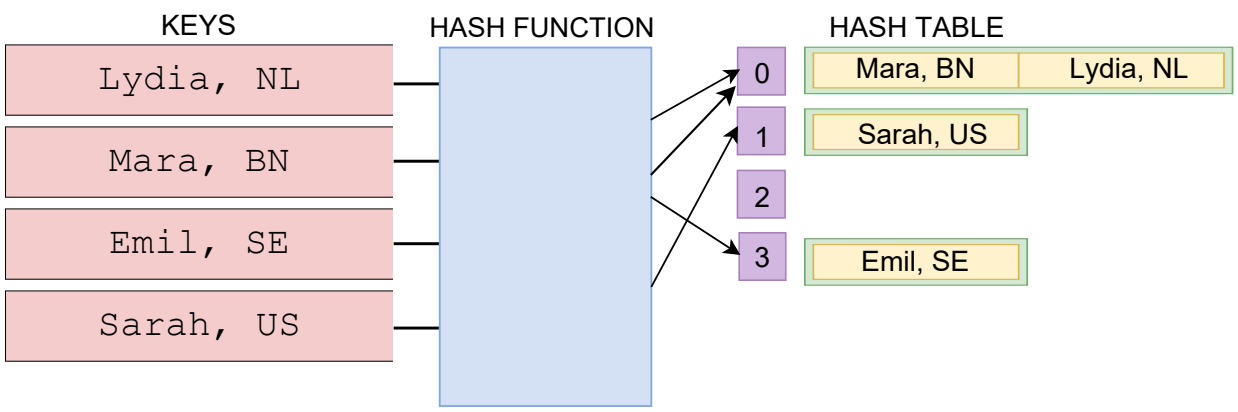
If we log this hash table (after implementing it), we get:

```
{
   values: {
      0: { "Mara": "BN" },
      1: { "Sarah": "US" },
      3: { "Emil": "SE" },
      5: { "Lydia": "NL" }
```

```
      5: { "Lydia" : "NL" },
    },
    length: 4,

    size: 6,
}
```

As the hash function always gives the same hash for every key, we can easily look up key/value pairs. Instead of having to map over an entire object, we just pass the key we want to the hash function, which then returns the index where this key/value pair is stored!

However, sometimes the hash function returns the same index for different keys. This is called **collision**.



There are now two key/value pairs at hash 0. If we now want to find the key/value pair with the key “Lydia”, we first have to iterate through this bucket, until we find the right key (the bucket is shown in green).

In the next lesson, I will talk about the implementation of this data structure.

Hash Table (Implementation)

create a hash table, add a hash, search for a key (Reading time: 2 minutes)

To implement a hash table, we create a class.

```
class HashTable {
  constructor() {
    this.values = {};
    this.length =  0;
    this.size =  0;
  }
}
```

The constructor contains an object in which we're going to store the values, the length of the values, and the entire size of the hash table: meaning how many buckets the hash table contains. Somewhere in these buckets, we can store our data.

Next, before we can do anything else, we need to implement our hashing function. One example of a hashing function:

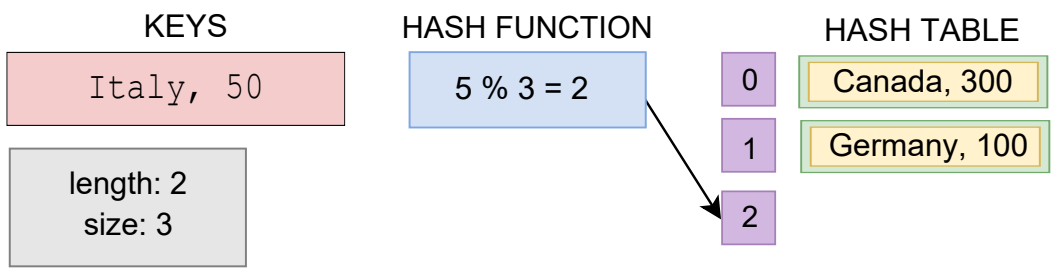
```
calculateHash(key) {
  return key.toString().length % this.size;
}
```

Let's say we have the string "Hello" and we create a hash table with the size of 10. "Hello" has the size of 5, so 5 % 10 becomes **5**! The string "Hello" is now stored in the bucket with hash 5. If later on, we want to see where the key/value pair with the key "Hello" has been stored, it would return the same hash as the length of "Hello" hasn't changed, and neither has the size of the hash table!

```
add(key, value) {
   const hash = this.calculateHash(key);
   if (!this.values.hasOwnProperty(hash)) {
      this.values[hash] = {};
   }
```

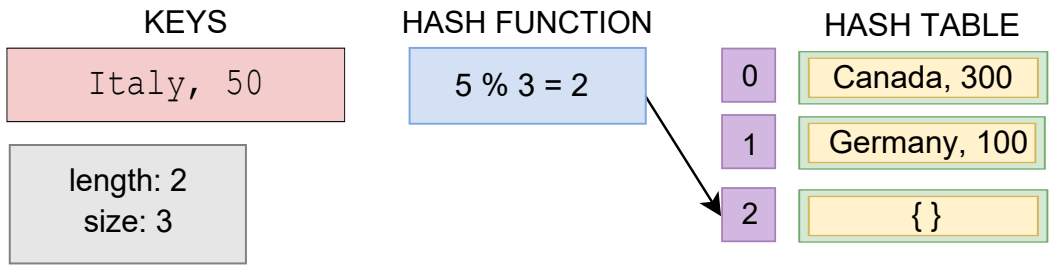
```
    if (!this.values[hash].hasOwnProperty(key)) {
        this.length++;
    }
    this.values[hash][key] = value;
}
```

To add a key/value pair, we first need to calculate the hash with the key provided. If this hash is brand new (meaning that no other key/value pair used it yet and it's not in the values object), we initialize an empty object for that hash. Next, we check whether the hash has a property with the same key name! If that's not the case, it means we will add a new key/value pair, and the length of the hash table grows. Lastly, we add the key/value pair to the right hash.



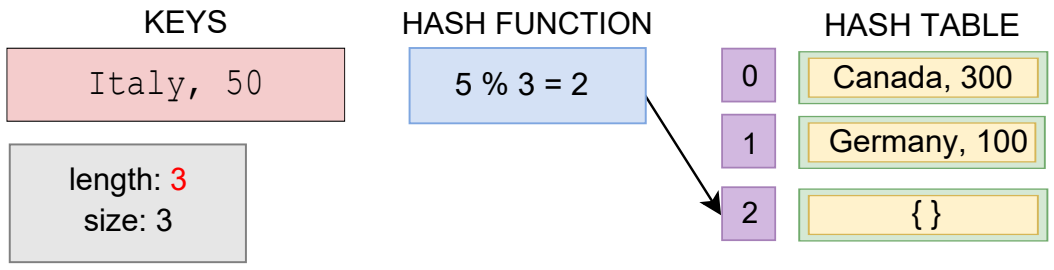
The length of "Italy" is 5, so the hash function returns the hash 2.

1 of 4



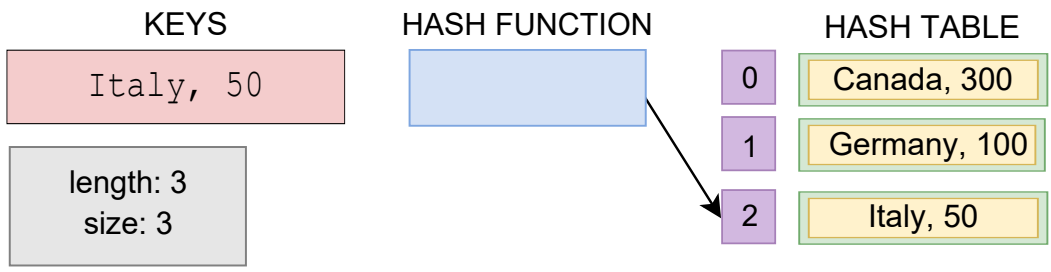
!this.values.hasOwnProperty(hash) returns true, so we initialize an empty object.

2 of 4



The hash's key value on the object doesn't have the value yet, so we increment the length by one.

3 of 4



We set the key equal to the value in the hash table, which adds it.

4 of 4

Searching in a hash table goes very fast. As with an array we have to go through all of the elements until we find it, with a hash table we simply get the index. This means that its runtime is constant, O(1).

```
search(key) {
  const hash = this.calculateHash(key);
  if (this.values.hasOwnProperty(hash) && this.values[hash].hasOwnProperty(key)) {
    return this.values[hash][key];
  } else {
    return null;
  }
}
}

//create object of type hash table
const ht = new HashTable();
//add data to the hash table ht
ht.add("Canada", "300");
ht.add("Germany", "100");
ht.add("Italy", "50");

//search
console.log(ht.search("Italy"));
```

Searching for the key "Italy" in the above hash table

First, we calculate the hash again. As the length of the string and the size of the hash table haven't changed, the hash remains the same. Then, we check

whether the hash is within the object, and whether that hash points to the key

we're looking for. If that's the case, return the value that that pair stores, else nothing gets returned.

In the next lesson, I will talk about the time complexity of the various functions of the hash table.

Hash Table (Time Complexity)

time complexity of the search, insertion and deletion functions of the hash table (Reading time: under 1 minute)

	TIME		SPACE
Type	**Average**	**Worst**	**Worst**
Search	O(1)	O(n)	O(n)
Insertion	O(1)	O(n)	
Deletion	O(1)	O(n)	

Average:

As we get the right location directly from the hash, we don't need to iterate over anything. The data is directly accessible.

Worst:

It is possible that the user stores all items in one bucket, which we would then have to iterate through.

Worst space:

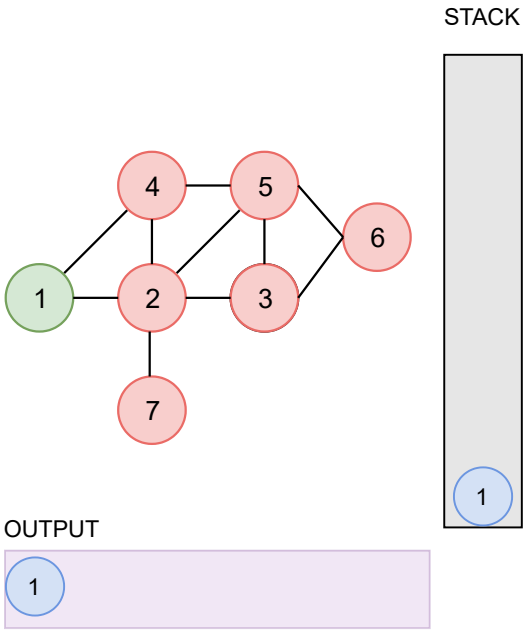
The more items, the bigger the hash table.

Now, let's move on to another data structure - the graphs.

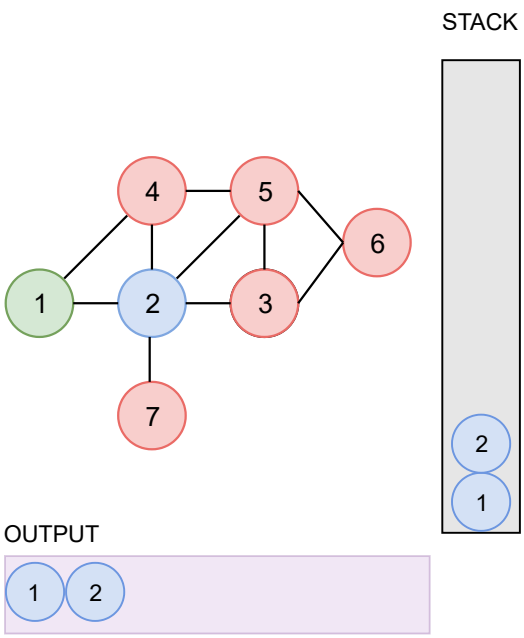
Graphs (Depth-first traversal)

The nodes are traversed from top to bottom. (Reading time: under 2 minutes)

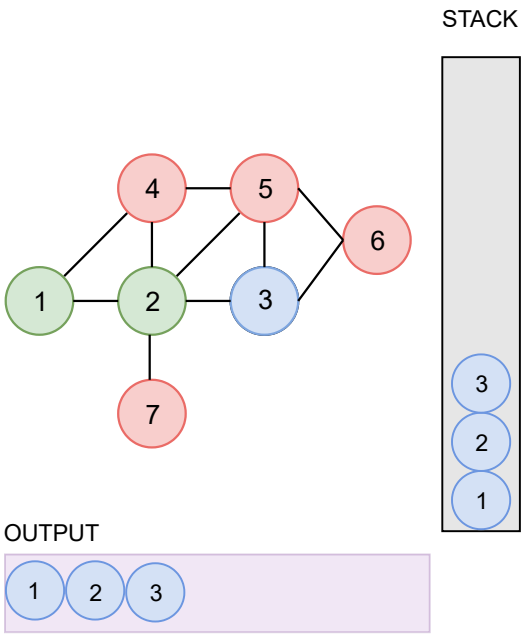
To traverse a graph depth-first, we need to use a **stack**. We manually pick one node to start with, as there is no specific root node like you have with a binary search tree, and go down each of the children nodes. In this example, **green nodes are marked as visited nodes, blue nodes are currently being visited**.



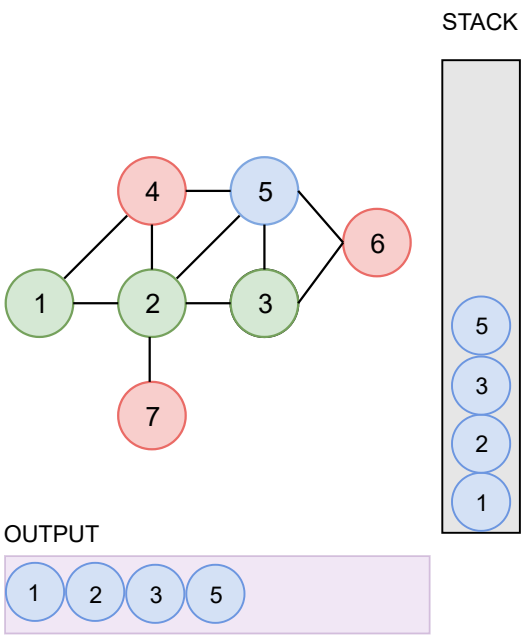
Let's say we used the node with the value 1 as our starting node. Right now, the stack contains the node with the value 1, and gets marked as visited as it gets added to the output sequence.



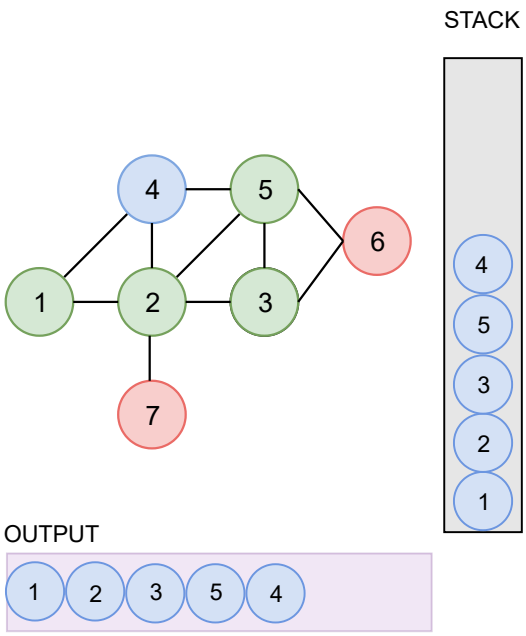
We go past its unvisited child nodes: 2 and 4. We choose the nodes numerically, and 2 is lower than 4, so we go to node 2. Node 2 is now added to the stack, pushed to the output sequence, and marked as visited.



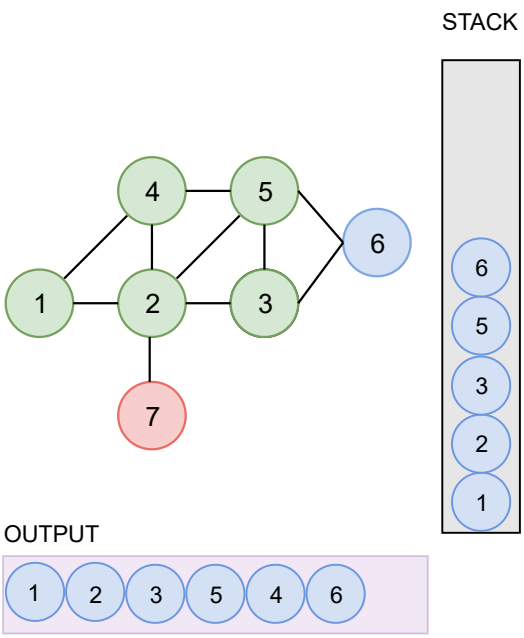
Node 2 has 3 unvisited child nodes: 7, 3 and 4. 3 is the smallest value, so the node with the value 3 gets added to the stack, pushed to the output sequence, and marked as visited.



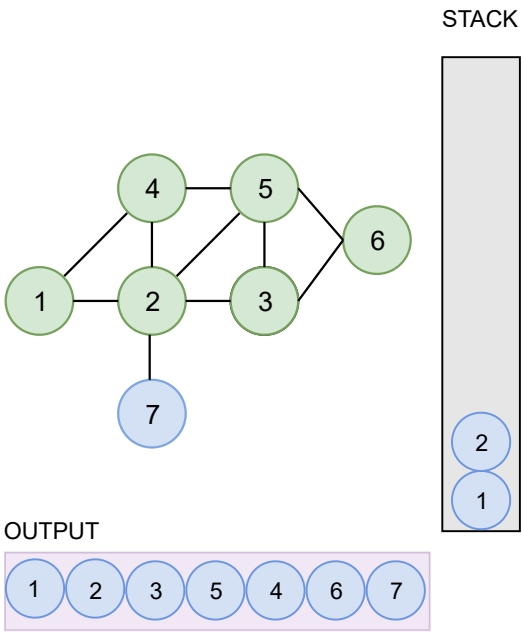
5 is the smallest unvisited node's value, so it gets pushed to stack and output sequence, and marked as visited.



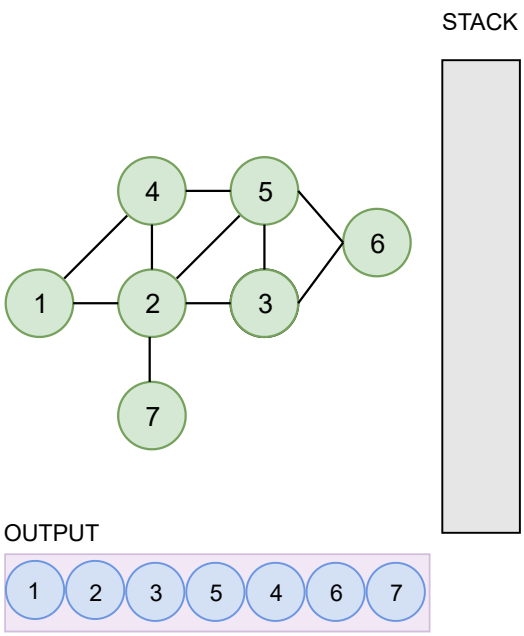
4 is the smallest unvisited node: it gets pushed to the stack and output array, and marked as visited. However, 4 doesn't have any unvisited child nodes! Now, the nodes get popped off the stack until it reaches a node that has unvisited child nodes.



After 4 has been popped off the stack, 5 is the node on top. 5 has an unvisited child node, node 6, so 6 gets added. This node again doesn't have any unvisited nodes. We pop nodes off the stack, until we find a node with unvisited child nodes, node 2!



The stack is empty: this is the sign that we have successfully traversed the entire graph!



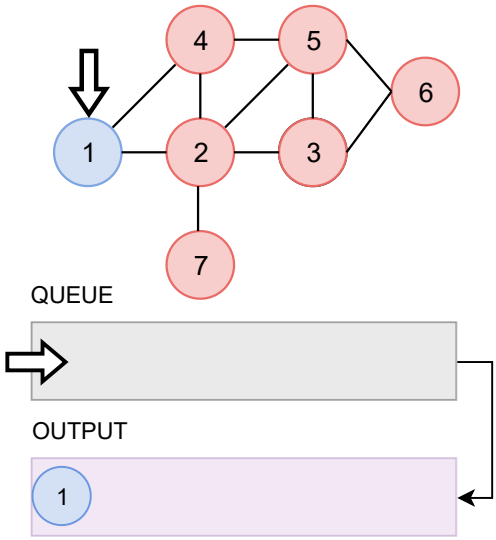
The stack is empty: this is the sign that we have successfully traversed the entire graph!

In the next lesson, I will talk about the breadth-first traversal of a graph.

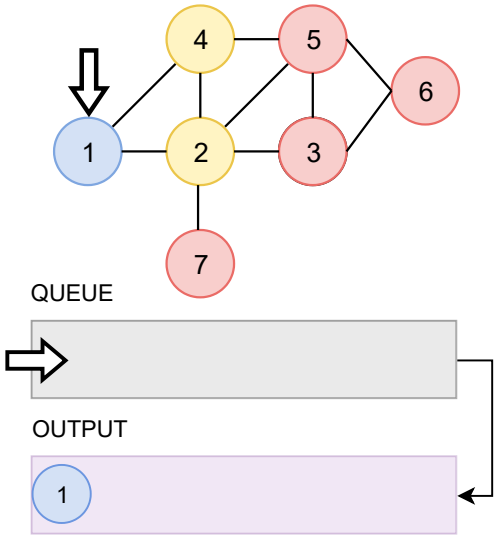
Graphs (Breadth-first traversal)

The nodes are traversed from left to right. (Reading time: under 2 minutes)

Like always, we use a queue with breadth-first traversal.

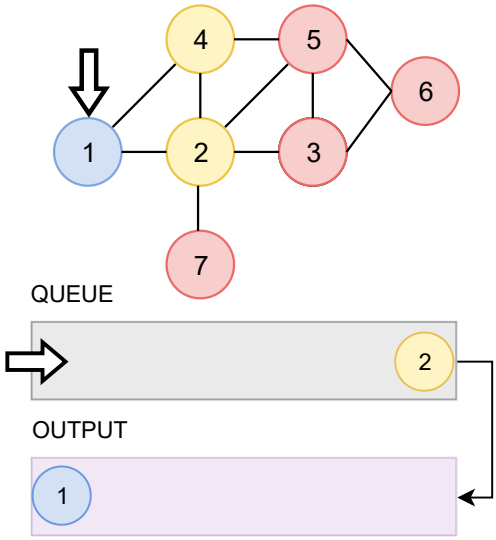


With breadth-first search, we use an “active node” which is displayed with an arrow in the examples. We set node 1 to be the active node when we start, so it directly gets pushed to the output sequence.

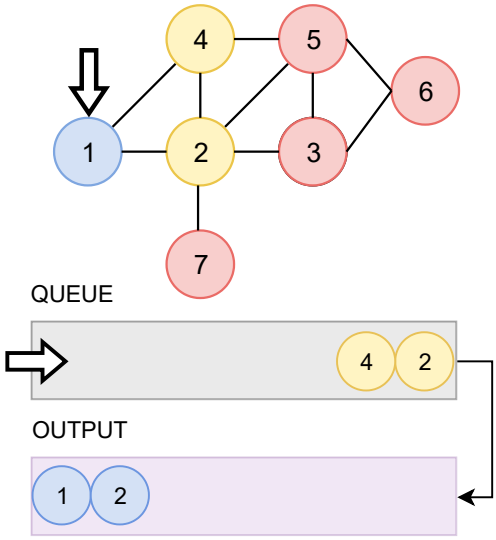


We visit the active node’s child nodes, mark them as visited, push them to the queue numerically, and add them to the output sequence! Node 1 doesn’t have any other child nodes, so we set the

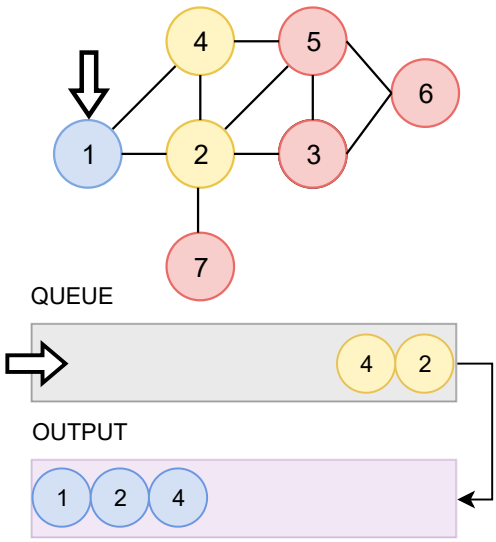
and add them to the output sequence! Node 1 doesn't have any other child nodes, so we set the first node in the queue to be the active node now, which is node 2.



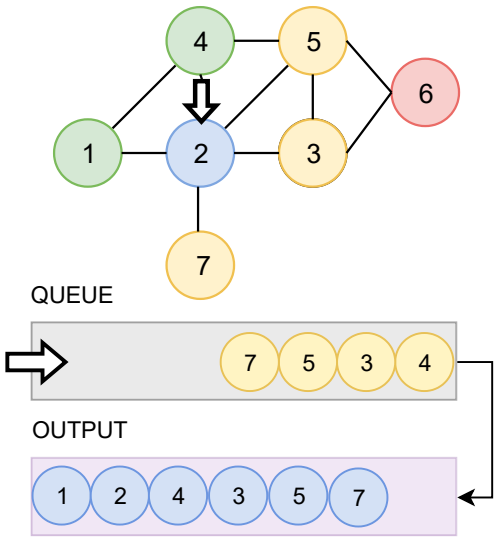
We visit the active node's child nodes, mark them as visited, push them to the queue numerically, and add them to the output sequence! Node 1 doesn't have any other child nodes, so we set the first node in the queue to be the active node now, which is node 2.



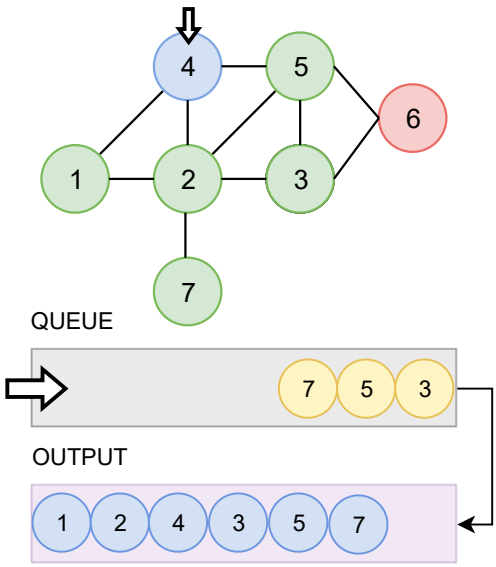
We visit the active node's child nodes, mark them as visited, push them to the queue numerically, and add them to the output sequence! Node 1 doesn't have any other child nodes, so we set the first node in the queue to be the active node now, which is node 2.



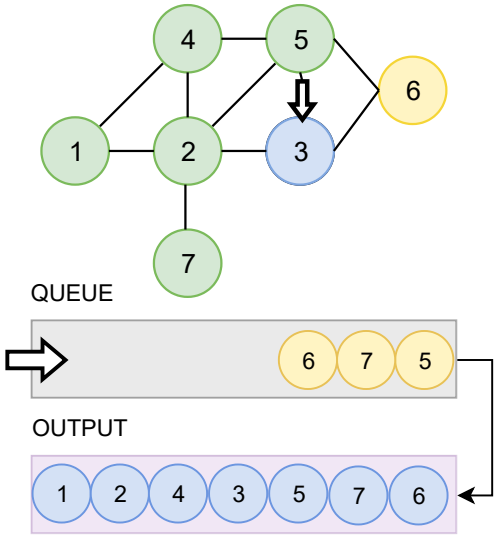
We visit the active node's child nodes, mark them as visited, push them to the queue numerically, and add them to the output sequence! Node 1 doesn't have any other child nodes, so we set the first node in the queue to be the active node now, which is node 2.



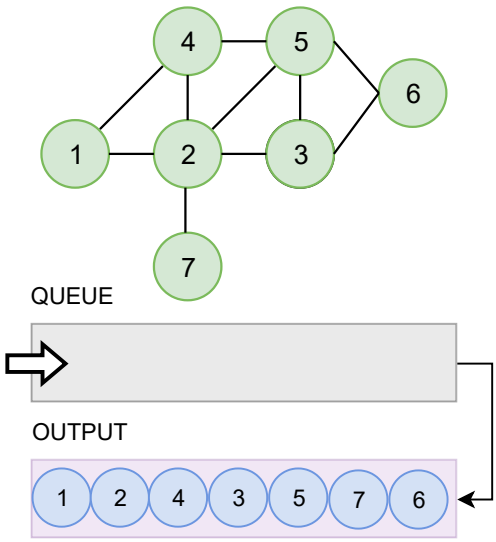
Node 2 gets removed from the queue, and its child nodes get pushed to the queue and output sequence in numerical order.



The next node in the queue is 4, however 4 doesn't have any unvisited child nodes. We keep on removing nodes from the queue, until we find a node that has unvisited child nodes.



The next in the queue is node 3, which has an unvisited child node 6! 6 gets pushed to the queue and sequence array.



Right now, the graph doesn't have any unvisited nodes anymore! The queue is empty, which is a sign that traversing has been successfully completed.

In the next chapter, I will talk about various algorithms and their time complexity.

Introduction to Sorting Algorithms

A brief introduction to algorithms used for sorting a list (Reading time: under 1 minute)

With sorting algorithms, we can put items of a list in the right order. There are many different ways to sort a list, which all have different time complexities! Sorting a list can be extremely efficient before using other algorithms, such as search or merge algorithms.

In this course, I will be covering **9** popular sorting algorithms:

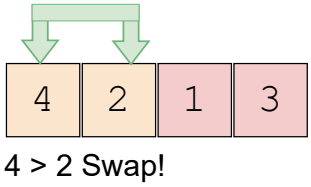
- Bubble sort
- Insertion sort
- Merge sort
- Quicksort
- Selection sort
- Counting sort
- Bucket sort
- Radix sort
- Heapsort

The next few lessons talk about each of the algorithms in detail.

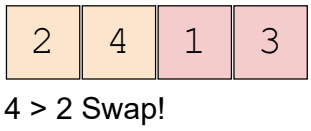
Introduction to Bubble Sort

Bubble sort sorts an array, by swapping elements that are in the wrong order.

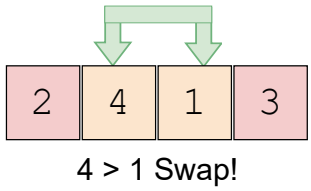
The algorithm starts with the first element in the array until the last element in the array, and keeps on doing so until there is one complete pass without swapping! This creates a **“bubble” effect**, hence the name.



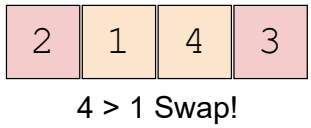
1 of 11



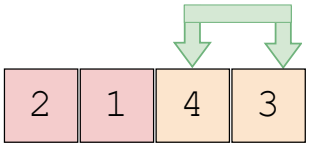
2 of 11



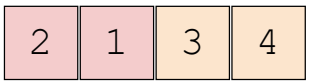
3 of 11



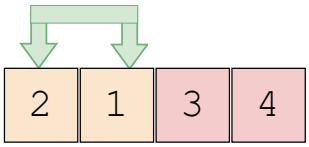
4 of 11



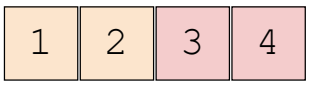
4 > 3 Swap!



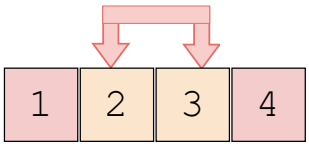
4 > 3 Swap!



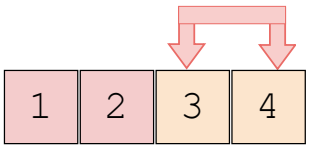
2 > 1 Swap!



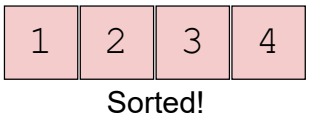
2 > 1 Swap!



2 < 3 No Swap!



3 < 4 No Swap!



We want the array `[4, 2, 1, 3]` to be sorted **in descending order**.

1. We go to the first element in the array, which is **4**. The next element, **2**, is smaller than **4**, so they are in the wrong position. They swap, and we go to the next element.

2. As the elements swapped, the current element is still **4**. We compare it to its next element, which is **1**. **1** is smaller than **4**, so they swap.

3. Again, the current element is **4**, as the elements swapped. We compare it to its next element **3**, which is smaller. They swap, and we've reached the end of the array. As this wasn't a pass without swapping, it means that the array hasn't been sorted yet, so we go back to the first element.

4. Current element **2** is bigger than **1**, they swap.

5. Current element **2** is smaller than **3**, no swapping.

6. Current element **3** is smaller than **4**, no swapping. Although we can see in the example that the array has been sorted, **the algorithm only returns once there has been an entire pass without swapping!**

7. Current element **1** is smaller than **2**, no swapping.

8. Current element **2** is smaller than **3**, no swapping.

9. Current element **3** is smaller than **4**, no swapping.

10. An entire pass without swapping! We've successfully sorted the array, and the bubble sort algorithm returns.

Now let's move on to the implementation of a bubble sort algorithm.

Now let's move on to the implementation of a bubble sort algorithm.

Bubble Sort (Implementation)

(Reading time: 4 minutes)

We need to create a function that receives the array as an argument.

```
function bubbleSort(array) {
}
```

Next, it's important to know whether we should swap at all. If the array has already been swapped, meaning an entire pass has gone without swapping, we won't have to go through the array again. We need to create a variable that holds a boolean value, and only if that value is true, we want to swap values.

```
let swapped;
do {
  swapped = false;
  // Swap logic here! In here, swapped will be set to true.
} while (swapped);
```

Right now, if swapped keeps on being false, the do-while will stop. We need to create the swapping logic. First, we need to loop over the array. Then, we check whether the current value, if it exists, is bigger than the next item's value, if that exists, meaning that they should swap.

```
for (let i = 0; i < array.length; i++) {
  if (array[i] && array[i + 1] && array[i] > array[i + 1]) {
     const temp = array[i];
     array[i] = array[i + 1];
     array[i + 1] = temp;
     swapped = true;
  }
}
```

If the if-statement returns true, we swap the items. We create a temporary variable that holds the value of the current element. It is necessary not to override this value in the second step! After the elements swapped, we set the

swapped variable to true, as at least two items swapped in this pass. The array is now sorted! We only need to return it.

```
function bubbleSort(array) {
  let swapped;
  do {
     swapped = false;
     for (let i = 0; i < array.length; i++) {
        if (array[i] && array[i + 1] && array[i] > array[i + 1]) {
          const temp = array[i];
          array[i] = array[i + 1];
          array[i + 1] = temp;
          swapped = true;
       }
     }
   } while (swapped);
  return array;
}
```

Now, let's discuss the time complexity of this algorithm.

Bubble Sort (Time Complexity)

best, average and worst case time complexity of the bubble sort algorithm (Reading time: under 1 minute)

TIME			SPACE
Best	**Average**	**Worst**	**Worst**
O(n)	$O(n^2)$	$O(n^2)$	O(1)

Best:

The array is already sorted.

Average and worst:

For every item in the array, we have to loop over the entire array.

Worst space:

We have constant variables, swapped, temp and i. However, you might argue that you can't sort the array, if you don't have it in memory. In that case, the worst case space complexity would be **O(n)**.

In the next lesson, I will discuss the insertion sorting algorithm.

Insertion Sort (Implementation)

(Reading time: under 3 minutes)

We need to create a function that receives the array that we want to sort as an argument.

```
function insertionSort(array) {
}
```

Just like with bubble sort, we need to loop over every item in the array. We declare a temporary variable to store the value of the current item, and declare a new variable `j`, which is equal to the index of the element that we will compare our current element to. The initial value of this index is one index lower than the currently checked item's index.

```
function insertionSort(array) {
   for (let i = 0; i < array.length; i++) {
     const temp = array[i];
     let j = i - 1;
    }
}
```

We need to check whether `j` is not a negative number, as we don't want to compare elements with negative indexes. If the index of `j` is smaller than **0**, we have reached the end of the array. Next, we need to check whether the value of the element on that index in the array, is higher than the current value. If that's the case, the elements are not in the correct position, and need to swap!

```
while (j >= 0 && array[j] > temp) {
  array[j + 1] = array[j];
  j--;
}
```

Let's say that we were working with the array `[4, 5, 1, 2, 3]`, and **1** was our

active value. The index of the currently active element is then **2**, so `temp` is equal to `array[2]` which is **5**, and `j` is equal to **1**. Then, **1** gets replaced with **5**. The array is now `[4, 5, 5, 2, 3]`; `array[2]` is now equal to **5**, and `j` is equal to **0**. We still need to swap **5** with **1**! As we know that the index of the new position of the currently active value is always one index higher than the value of `j`, we set `array[j + 1]` equal to the value that temp holds, which is the currently active value. Now, the array is `[4, 1, 5, 2, 3]`!

```
array[j + 1] = temp;
```

```
function insertionSort(array) {
   for (let i = 0; i < array.length; i++) {
     let temp = array[i];
     let j = i - 1;
     while (j >= 0 && array[j] > temp) {
       array[j + 1] = array[j];
       j--;
     }
     array[j + 1] = temp;
    }
  return array;
}
```

In the next lesson, I will talk about the time complexity of this algorithm.

Insertion Sort (Time Complexity)

best, average and worst case time complexity of the insertion sort algorithm (Reading time: under 1 minute)

TIME			SPACE
Best	**Average**	**Worst**	**Worst**
O(n)	$O(n^2)$	$O(n^2)$	O(1)

Best:

The array is already sorted.

Average and worst:

For every item in the array, we have to loop over the entire array.

Worst space:

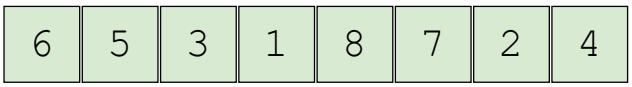
We have three constant variables, `temp`, `j` and `i`. One could argue that you would have to store the array in memory, which gives **O(n)**.

In the next lesson, I will talk about merge sort.

Introduction to Merge Sort

Merge sort sorts lists by breaking them down, sorting them and then merging the sorted lists. (Reading time: under 1 minute)

It divides the given array into halves, calls itself for the two halves, and then merges these halves.



Unsorted array

Split into half

Split left array into half

Split left array into half

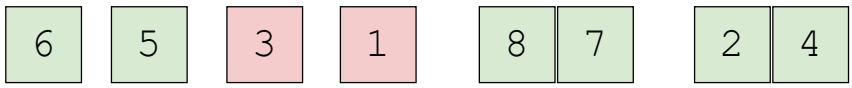
Split left array into half

Split left array into half

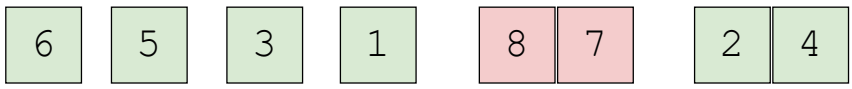
Split [6, 5]

Split [6, 5]

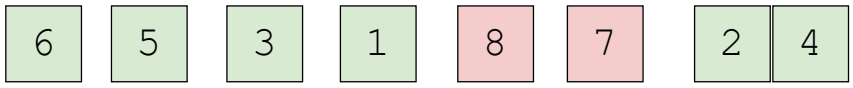
Split [3, 1]



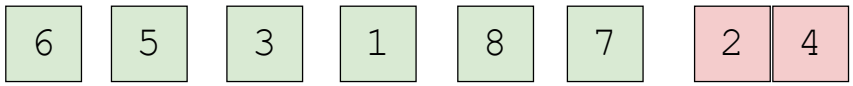
Split [3, 1]



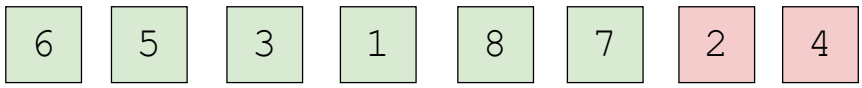
Split [8, 7]



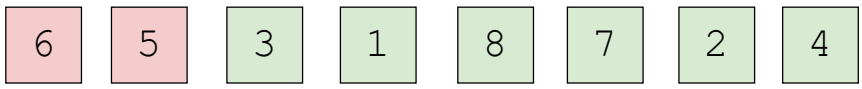
Split [8, 7]



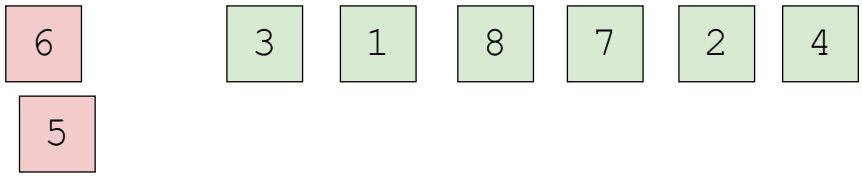
Split [2, 4]



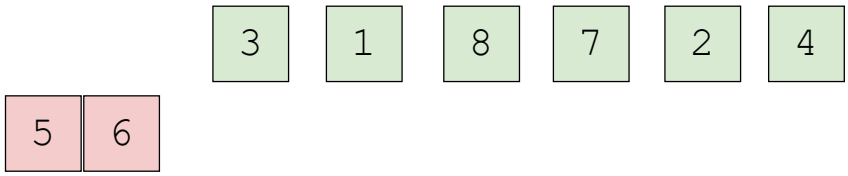
Split [2, 4]



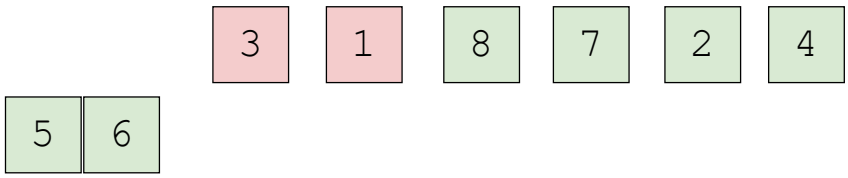
6>5



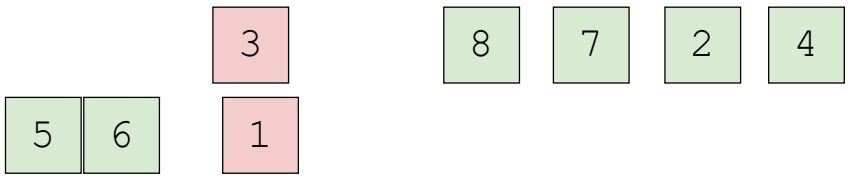
Swap!



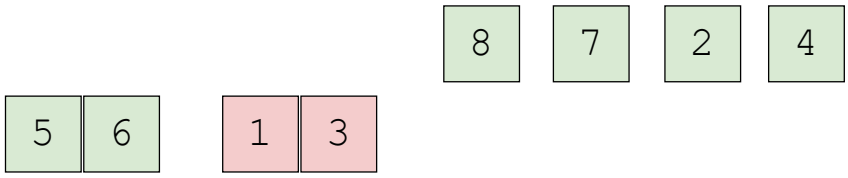
Swap!



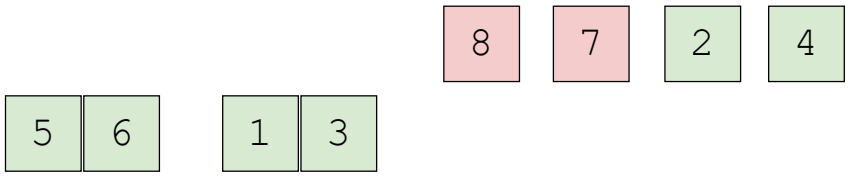
3>1



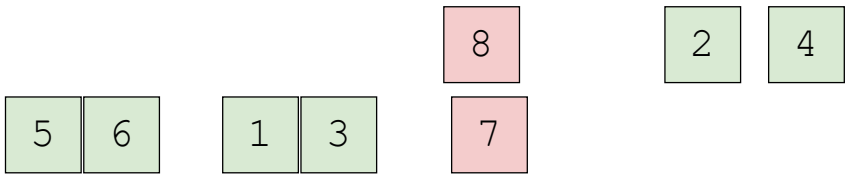
Swap!



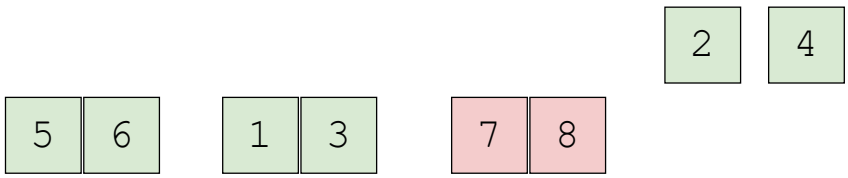
Swap!



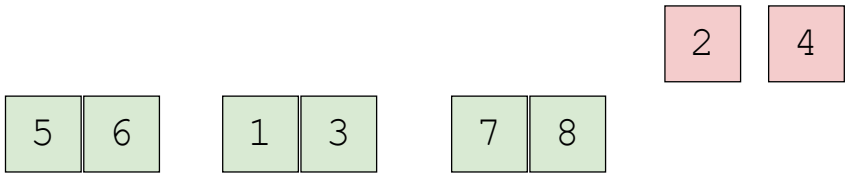
8>7



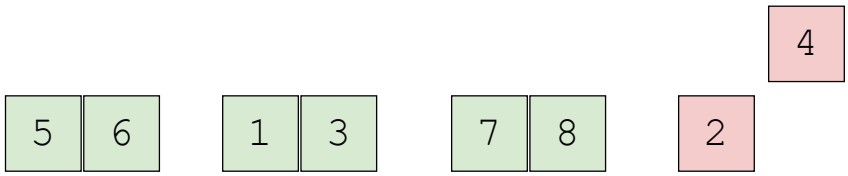
Swap!



Swap!



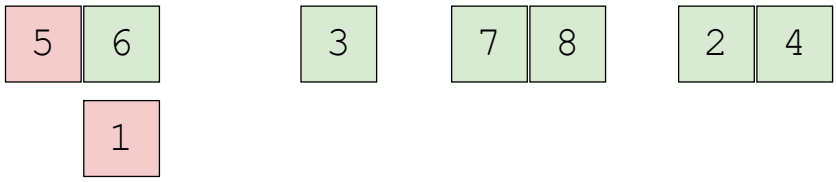
2<4



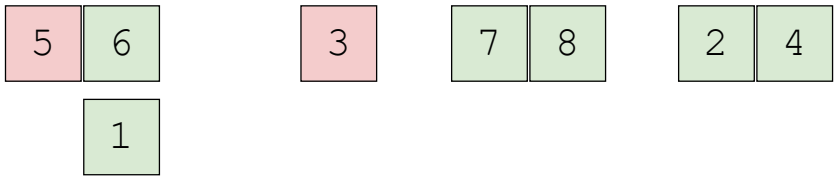
Do not swap!

Do not swap!

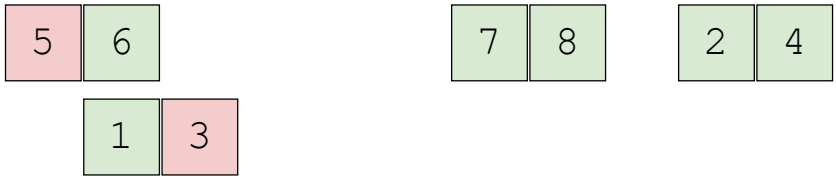
1<5



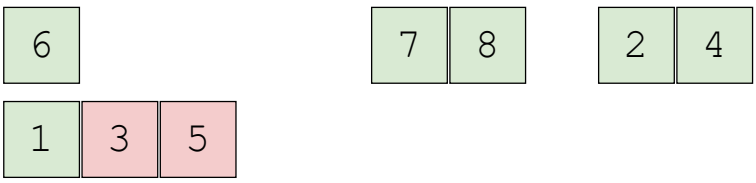
1<5



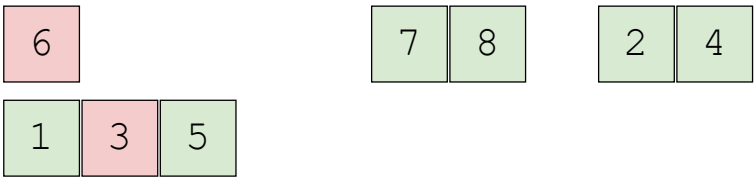
3<5



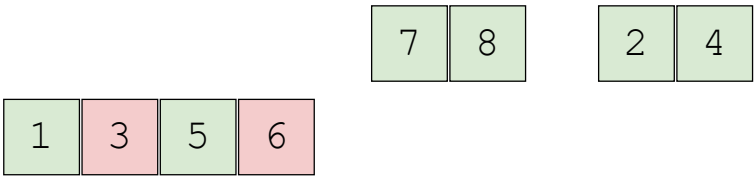
3<5



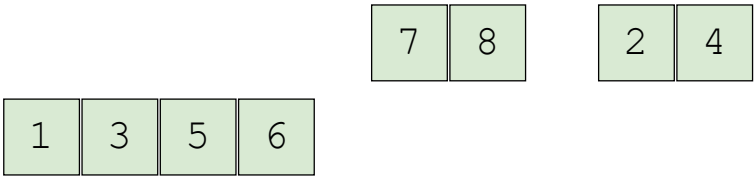
3<5



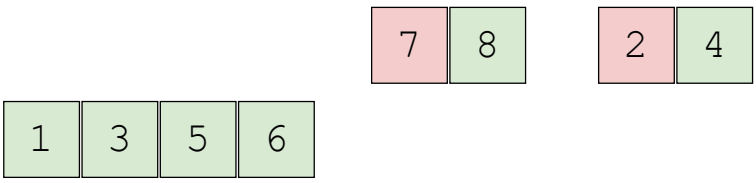
3<6



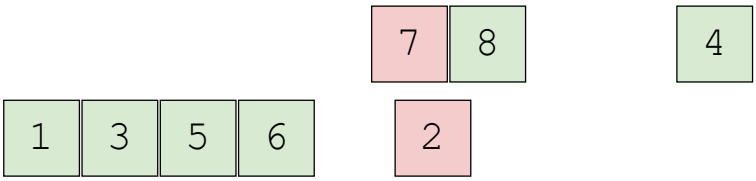
3<6



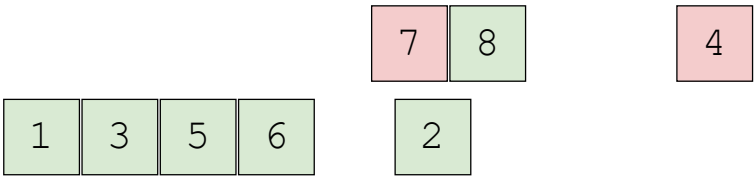
Left half of the array has been sorted.



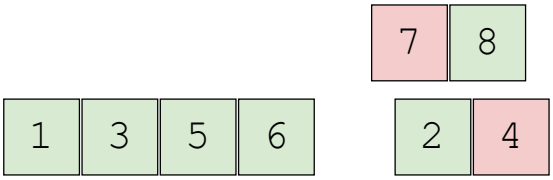
2<7



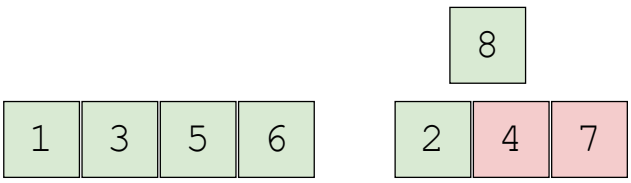
2<7



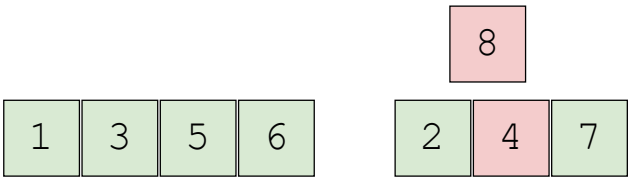
4<7



4<7



4<7

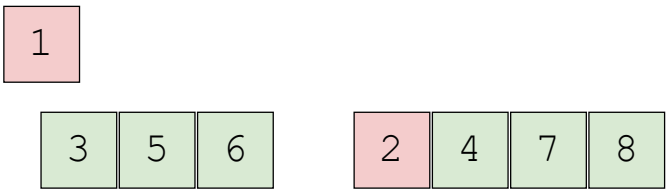


4<8

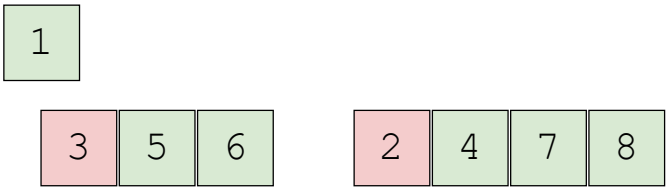
4<8

Right half of the array has been sorted.

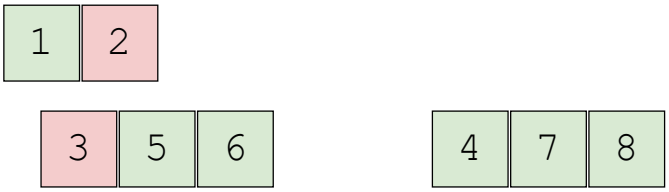
1<2



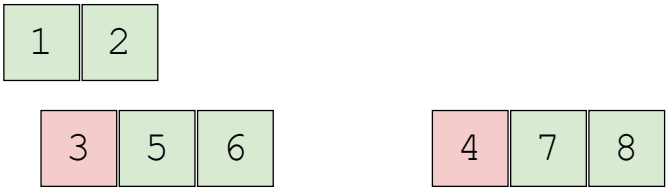
1<2



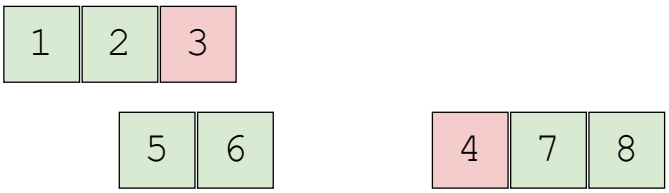
2<3



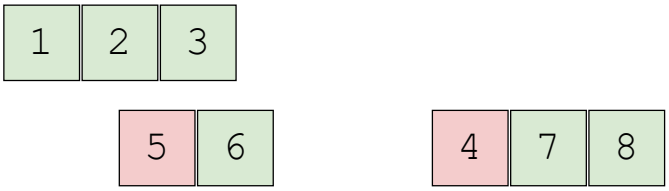
2<3



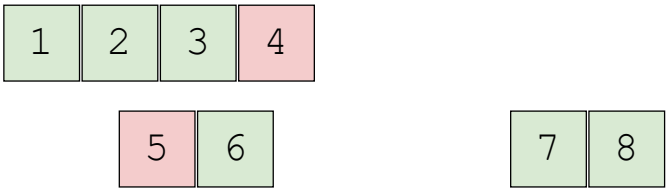
3<4



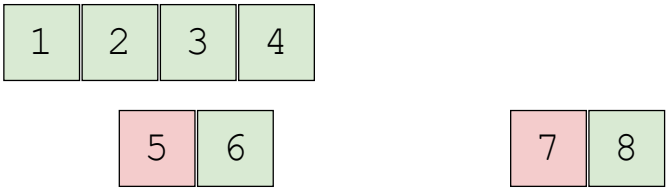
3<4



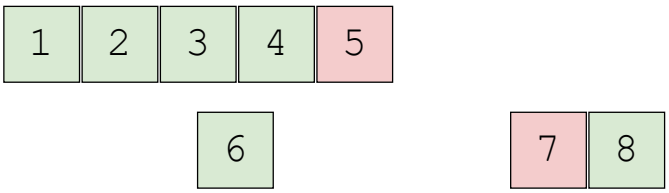
4<5



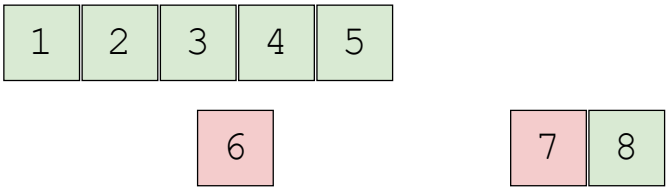
4<5



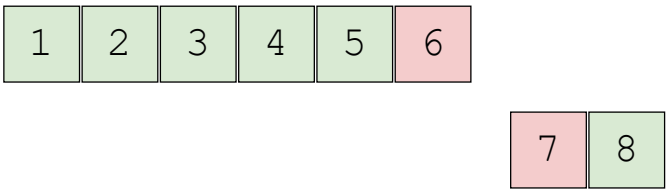
5<7



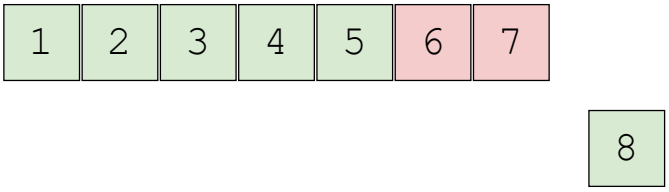
5<7



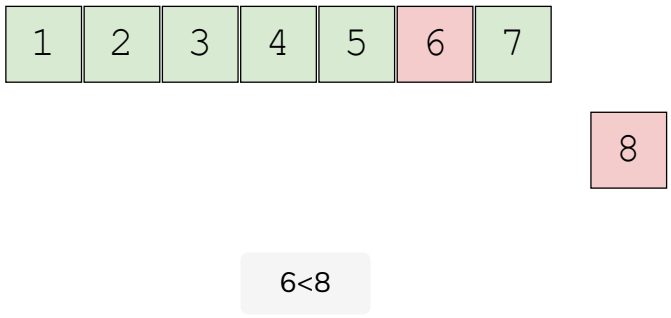
6<7



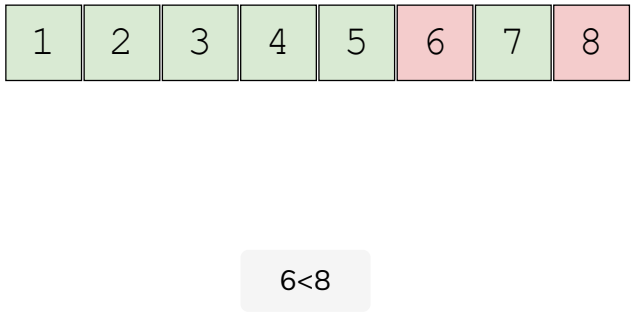
6<7



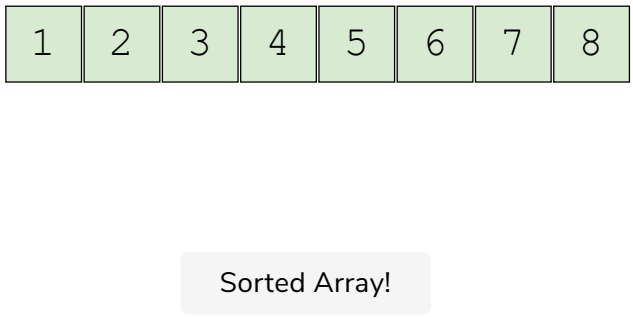
6<7



56 of 58



57 of 58



58 of 58

In the next lesson, I will discuss the working of this algorithm in a little more detail along with it's code implementation.

Merge Sort (Implementation)

(Reading time: 6 minutes)

The merge algorithm consists of two functions:

- The mergeSort function, which takes care of partitioning the arrays.
- The merge function, which merges the separate arrays.

```
function mergeSort(array) {
   if (array.length === 1) {
     return array;
   }
   const middle = Math.floor(array.length / 2);
   const left = array.slice(0, middle);
   const right = array.slice(middle);
   return merge(
      mergeSort(left),
      mergeSort(right)
   );
}

function merge(left, right) {
  let result = [];
  let leftIndex = 0;
  let rightIndex = 0;

  while (leftIndex < left.length && rightIndex < right.length) {
    if (left[leftIndex] < right[rightIndex]) {
       result.push(left[leftIndex]);
       leftIndex++;
    } else {
       result.push(right[rightIndex]);
       rightIndex++;
    }
  }

  return result.concat(left.slice(leftIndex)).concat(right.slice(rightIndex));
}
```

In the next lesson, I will discuss the runtime of this algorithm.

Merge Sort (Time Complexity)

best, average and worst case time complexity of the merge sort algorithm (Reading time: under 1 minute)

TIME			SPACE
Best	**Average**	**Worst**	**Worst**
O(n log(n))	O(n log(n))	O(n log(n))	O(n)

Best, average and worst:

Each partitioning takes **O(n)** operations, and every partitioning splits the array **O(log(n))**. This results in **O(n log(n))**.

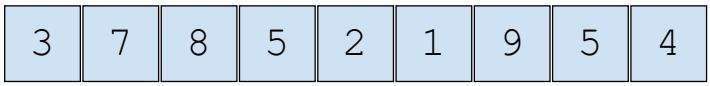
Worst space:

We save three variables for each element in the array.

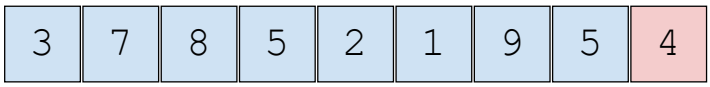
Introduction to Quicksort

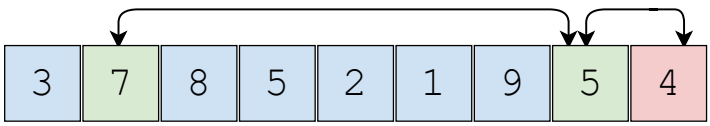
This algorithm sorts a list by choosing a pivot and comparing each element to that pivot. (Reading time:)

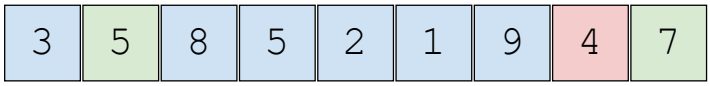
As quicksort doesn't use any space, it's a **in-place sorting algorithm**. The way quicksort works is by choosing a **pivot** (an element in the array, often random), and check whether values in the array are higher or lower than that pivot. The values lower than the pivot should be on the left side, and the values higher than the pivot should be on the right side!

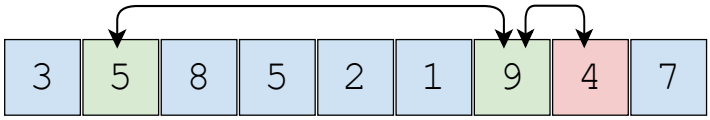


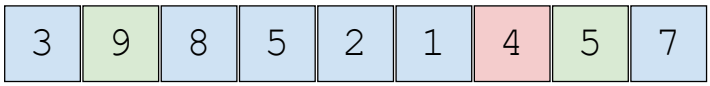
Unsorted array

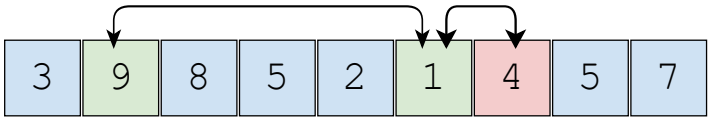


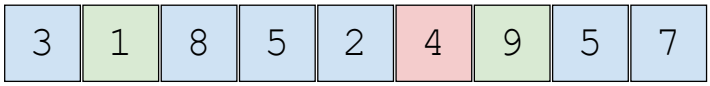


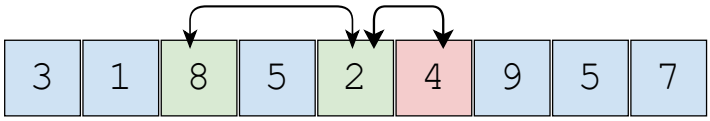

3 5 8 5 2 1 9 4 7


3 5 8 5 2 1 9 4 7

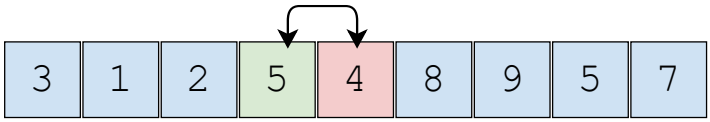

3 9 8 5 2 1 4 5 7


3 9 8 5 2 1 4 5 7

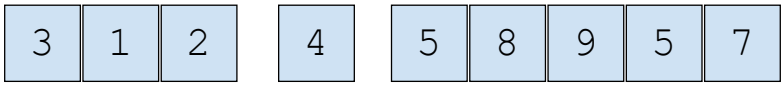

3 1 8 5 2 4 9 5 7

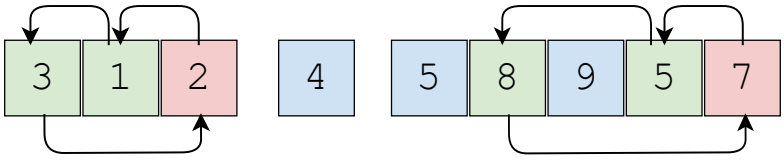

3 1 8 5 2 4 9 5 7

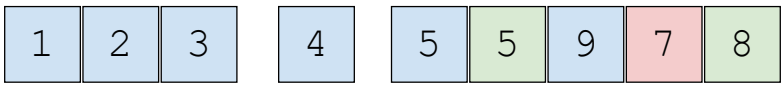
3 1 2 5 4 8 9 5 7

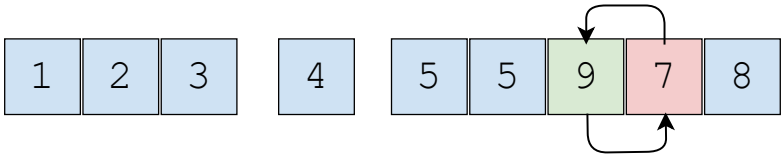

3 1 2 5 4 8 9 5 7

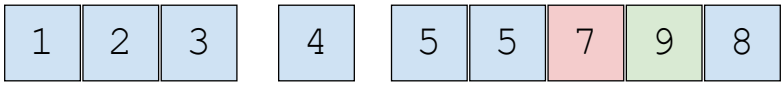
3 1 2 4 5 8 9 5 7

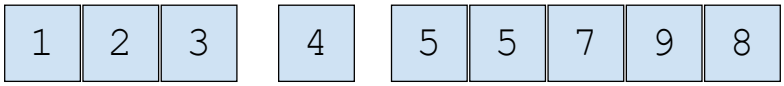

3 1 2 4 5 8 9 5 7

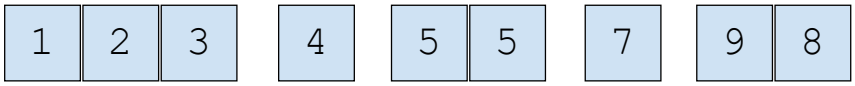

3 1 2 4 5 8 9 5 7

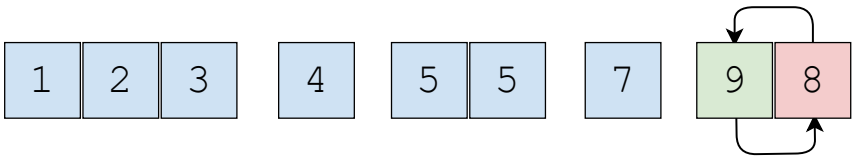

1 2 3 4 5 5 9 7 8

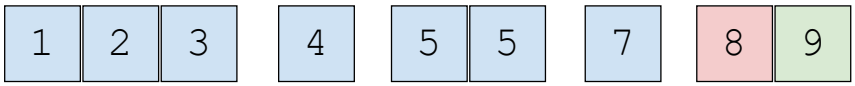

1 2 3 4 5 5 9 7 8

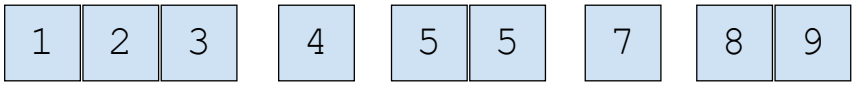

1 2 3 4 5 5 7 9 8


1 2 3 4 5 5 7 9 8


1 2 3 4 5 5 7 9 8


1 2 3 4 5 5 7 9 8


1 2 3 4 5 5 7 8 9


1 2 3 4 5 5 7 8 9

1 2 3 4 5 5 7 8 9

In the next lesson, I will talk about the implementation of this algorithm.

Quicksort (Implementation)

(Reading time: under 3 minutes)

First, we create a function that receives the array we want to sort as an argument. It should also be able to gwt the *left* and *right* value. However, as we don't pass these values at the beginning, it should have a default value as well.

```
function quicksort(array, left, right) {
   left = left || 0;
   right = right || array.length - 1;
}
```

Next, we need to create a function that chooses the pivot. In this example, I use the **Hoare partition scheme**. Again, this function should receive the array we want to sort, the *left* value, and the *right* value.

```
function partitionHoare(array, left, right) {
}
```

Here, we first choose our random pivot.

```
const pivot = Math.floor(Math.random() * (right - left + 1) + left);
```

Then, we need to check whether the left and right values are in the correct position. If the left is bigger or equal to right, they need to swap.

```
while (left <= right) {
 //Swap logic
}
```

Now, we need to check whether it's necessary to swap. If the left value is smaller than the pivot, we increase the value of the left by one, so that we move on to the next element. If the right is bigger than the pivot, we decrease

move on to the next element. If the right is bigger than the pivot, we decrease the value of right by one, so we move on to the previous element. No

swapping is needed: they're already in the correct place! If we finally find elements for which the conditions return false, we swap the elements.

```
while (array[left] < array[pivot]) {
  left++;
}
while (array[right] > array[pivot]) {
  right--;
}
if (left <= right) {
  [array[left], array[right]] = [array[right], array[left]];
}
```

We return the left value, as this will be our pivot. Right now, the entire function looks like:

```
function partitionHoare(array, left, right) {
   const pivot = Math.floor(Math.random() * (right - left + 1) + left);
   while (left <= right) {
     while (array[left] < array[pivot]) {
        left++;
     }
     while (array[right] > array[pivot]) {
       right--;
     }
     if (left <= right) {
       [array[left], array[right]] = [array[right], array[left]];
     }
   }
   return left;
}
```

Back to the quicksort function. Here, we now set the value of the pivot equal to whatever gets returned from the `partitionHoare` function.

```
const pivot = partitionHoare(array, left, right);
```

We need to invoke the quicksort function recursively, to keep on sorting every *left* and *right* side. We invoke the quicksort function with the new *left* and *right* values every time. For the left side, the *left* value is just the left element, and the right value is one element before the pivot. For the right side, the left value is the pivot, and the right value is the right element. After that, we return the sorted array.

```
if (left < pivot - 1) {
   quicksort(array, left, pivot - 1);
}
if (right > pivot) {
   quicksort(array, pivot, right);
}
```

The quicksort function now looks like:

```
function quicksort(array, left, right) {
   left = left || 0;
   right = right || array.length - 1;

   const pivot = partitionHoare(array, left, right);

   if (left < pivot - 1) {
      quicksort(array, left, pivot - 1);
   }
   if (right > pivot) {
      quicksort(array, pivot, right);
   }
   return array;
}
```

Now, let's move on to the time complexity of this algorithm.

Quicksort (Time Complexity)

best, average and worst case time complexity of the quicksort algorithm. (Reading time: under 1 minute)

TIME			SPACE
Best	**Average**	**Worst**	**Worst**
O(n log(n))	O(n log(n))	$O(n^2)$	O(log (n))

Best and average:

Each partitioning takes **O(n)** operations, and every partitioning splits the array **O(log(n))**. This results in **O(n log(n))**.

Worst:

If you always pick a pivot that is the highest or lowest value, you need to iterate through the entire array.

Worst space:

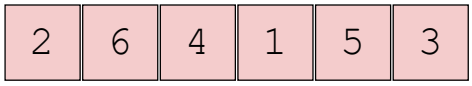
The number of variables that are stored

In the next lesson, I will discuss the selection sort algorithm.

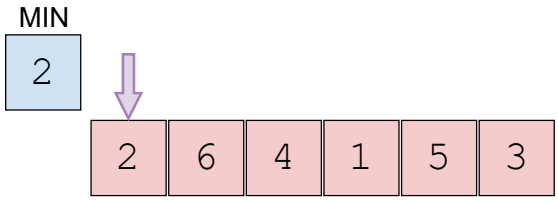
Introduction to Selection Sort

Selection sort works by finding the minimum, and then comparing each element to that minimum to decide its correct position. (Reading time: under 3 minutes)

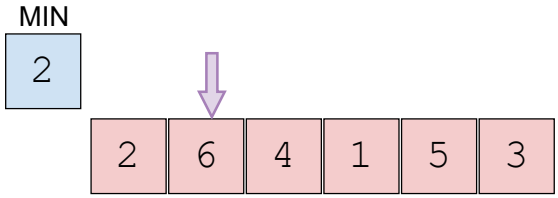
Selection sort is a simple sorting algorithm, that loops over the array and saves the absolute lowest value. The lowest value is then swapped with the first item in the unsorted array.



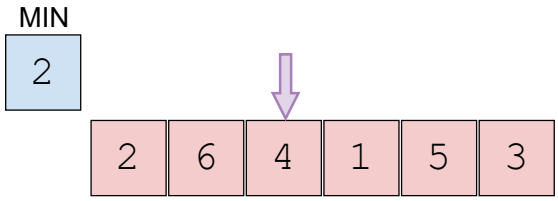
We want to sort the array with values [2, 6, 4, 1, 5, 3] in ascending order.



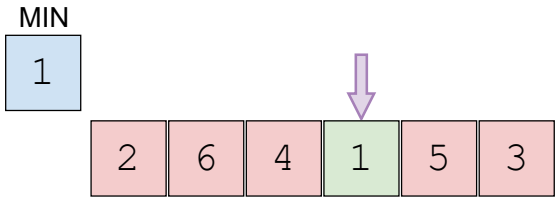
We store the current minimum value, and loop over the array. If we find an item that's lower than the current minimum value, the minimum value will be replaced with that value!



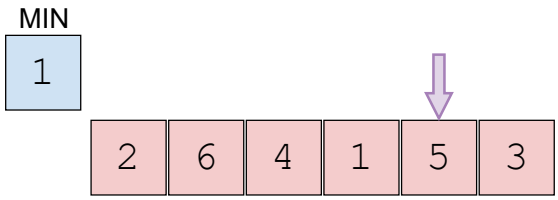
6 > 2, MIN remains the same.



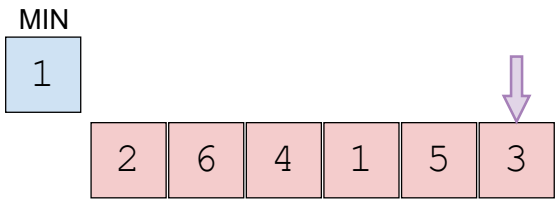
4 > 2, MIN remains the same.



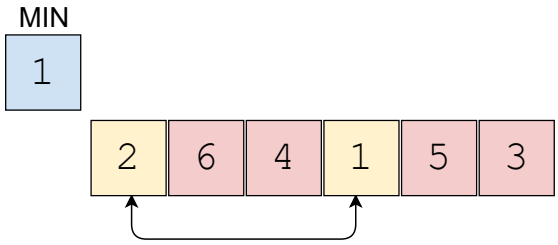
1 < 2, MIN is updated



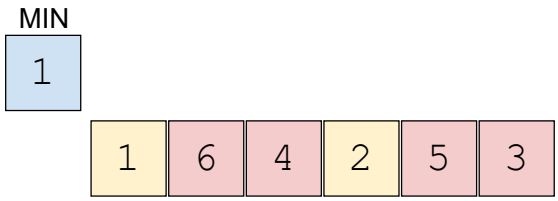
5 > 1, MIN remains the same.



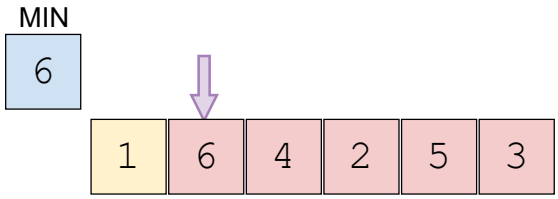
3 > 1, MIN remains the same. We're now at the end of the array.



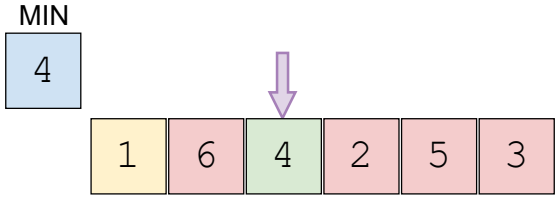
We swap the first item of the unsorted array with the minimum value, and move the sorted array one element.



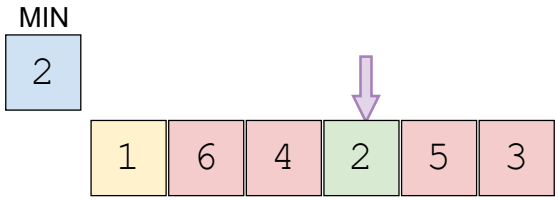
We swap the first item of the unsorted array with the minimum value, and move the sorted array one element.



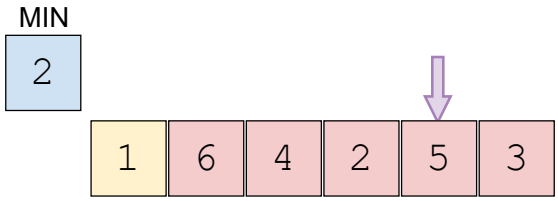
The yellow elements show the elements of the sorted array. Loop starts again. MIN = 6



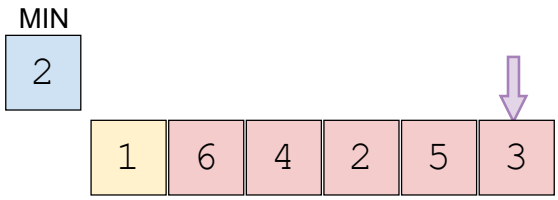
4 < 6, MIN is updated



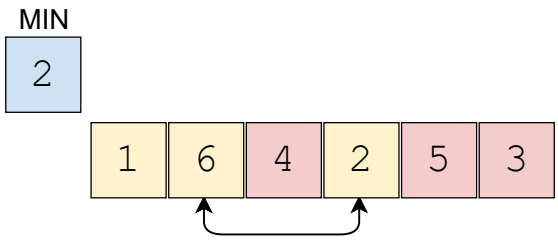
2 < 4, MIN is updated



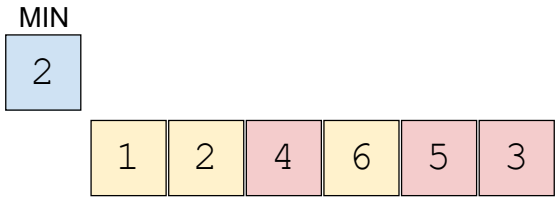
5 > 2, MIN remains the same



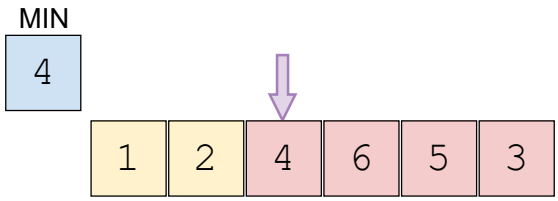
3 > 2, MIN remains the same



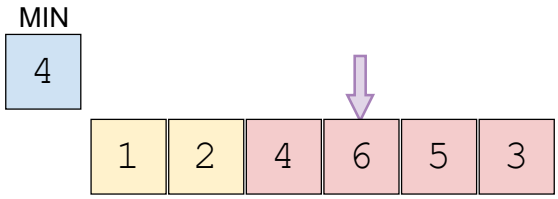
We swap the first item of the unsorted array with the minimum value, and move the sorted array one element.



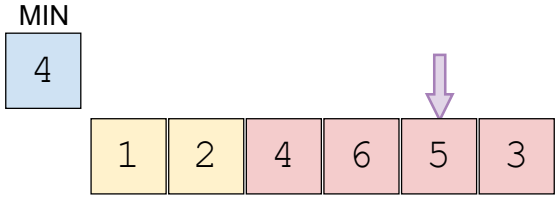
We swap the first item of the unsorted array with the minimum value, and move the sorted array one element.



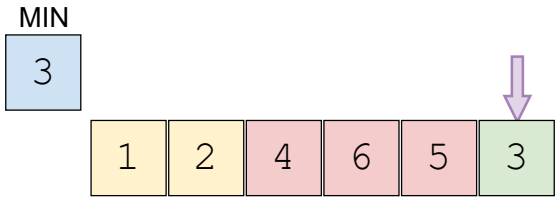
The yellow elements show the elements of the sorted array. We now loop over the unsorted array again. MIN = 4



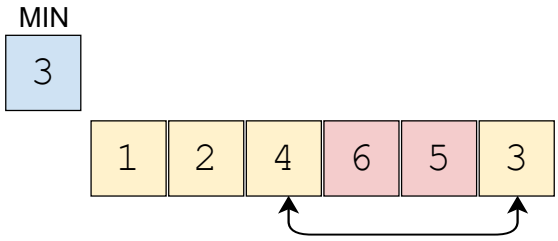
6 > 4, MIN remains the same



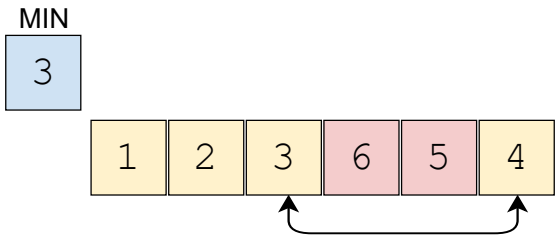
5 > 4, MIN remains the same



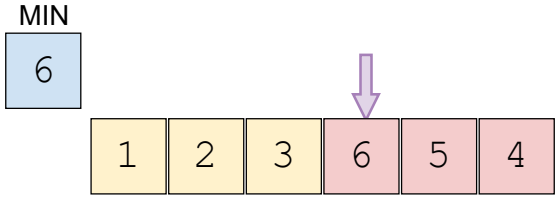
3 < 4, MIN is updated



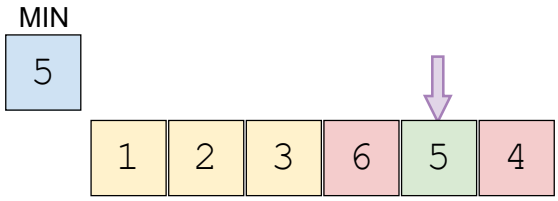
We swap the first item of the unsorted array with the minimum value, and move the sorted array one element.



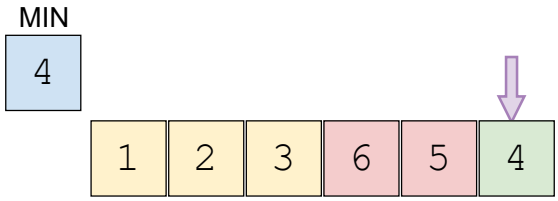
We swap the first item of the unsorted array with the minimum value, and move the sorted array one element.



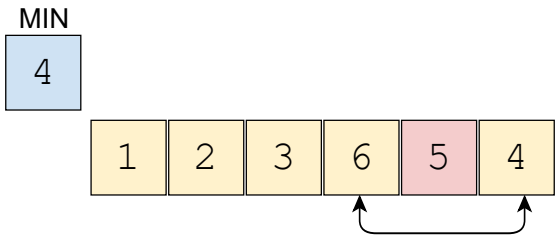
The yellow elements show the elements of the sorted array. We loop over the unsorted array again. MIN = 6



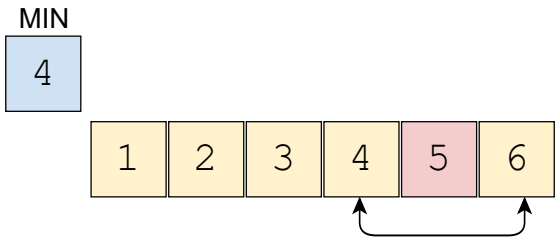
5 < 6, MIN is updated



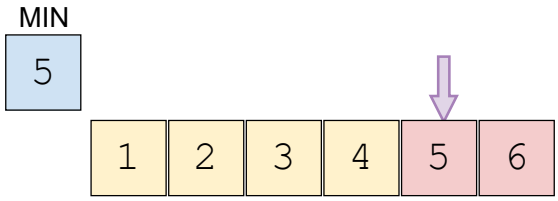
4 < 5, MIN is updated



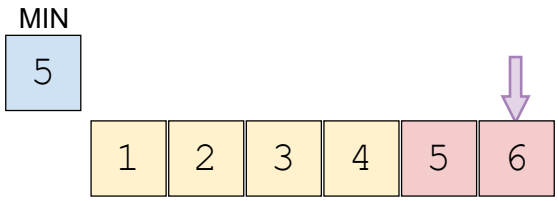
We swap the first item of the unsorted array with the minimum value, and move the sorted array one element.



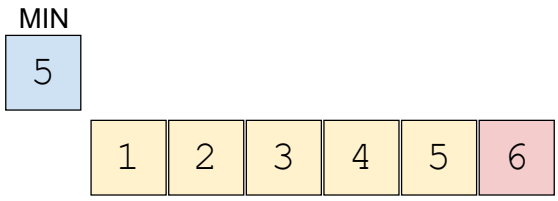
We swap the first item of the unsorted array with the minimum value, and move the sorted array one element.



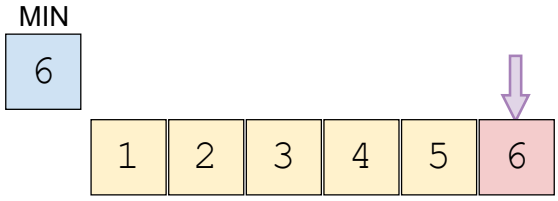
The yellow elements show the elements of the sorted array. We now loop over the unsorted array again. MIN = 5



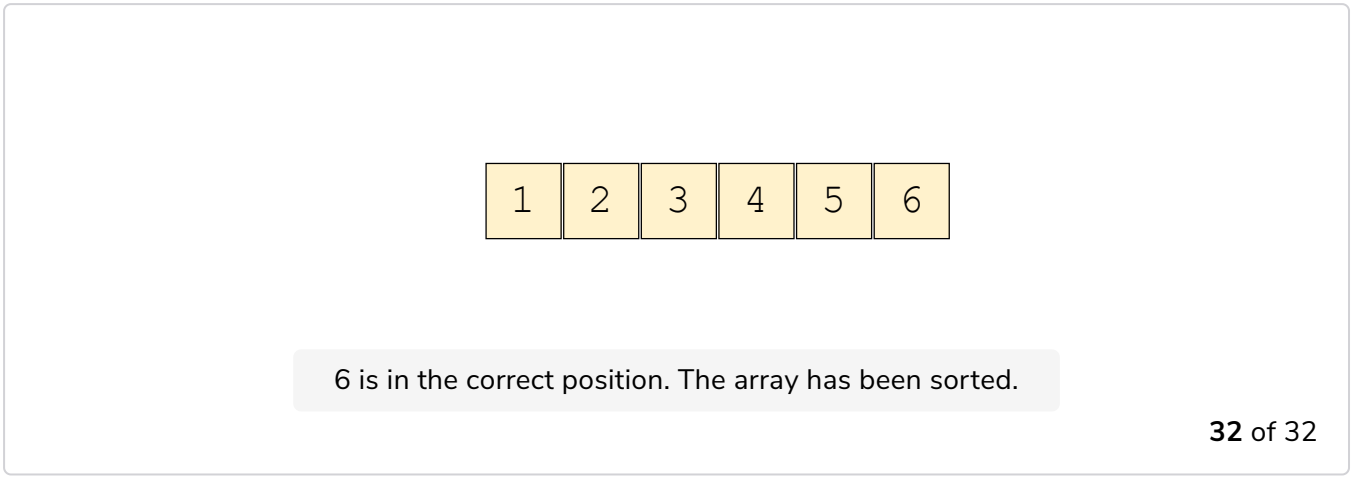
6 > 5, MIN remains the same



5 is at the correct position. The yellow elements show the elements of the sorted array.



We loop over the unsorted array again. MIN = 6



In the next lesson, I will talk about the implementation of this algorithm.

Selection Sort (Implementation)

(Reading time: 3 minutes)

First, we create a function that receives the array we want to sort as an argument.

```
function selectionSort(array) {
}
```

Then, we loop over our array. We create a variable that holds the minimum value.

```
for (let i = 0; i < array.length; i++) {
  let min = i;
}
```

For every item, we loop over the array from the element after the minimum value. We need to create another loop, which checks whether there is an item that's lower than the minimum value. If that is the case, then we set the min variable equal to that value!

```
for (let j = i + 1; j < array.length; j++) {
  if (array[j] < array[min]) {
     min = j;
  }
}
```

However, we haven't swapped anything yet. We need to check whether the minimum value has been updated, meaning that we should swap values!

```
if (i !== min) {
   [array[i], array[min]] = [array[min], array[i]];
}
```

In the end, we return the sorted array. The entire function looks like:

```
function selectionSort(array) {
   for (let i = 0; i < array.length; i++) {
      let min = i;
      for (let j = i + 1; j < array.length; j++) {
         if (array[j] < array[min]) {
            min = j;
         }
      }
      if (i !== min) {
        [array[i], array[min]] = [array[min], array[i]];
      }
    }
  return array;
}
```

Now, let’s talk about the time complexity of this algorithm.

Selection Sort (Time Complexity)

best, average and worst case time complexity of the selection sort algorithm. (Reading time: under 1 minute)

TIME			SPACE
Best	**Average**	**Worst**	**Worst**
$O(n^2)$	$O(n^2)$	$O(n^2)$	$O(1)$

Best, average and worst:

For every element in the array, we loop over the array. This means that for n elements, we have to loop over n elements.

Worst space:

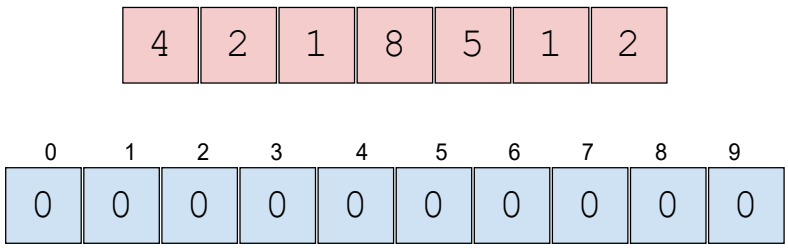
We have one variable in the for-loop.

Now, let's move on to the counting sort algorithm.

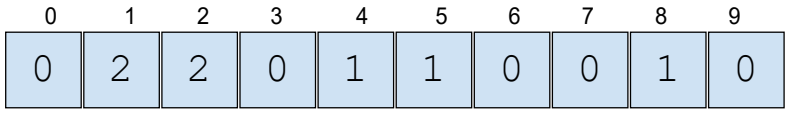
Introduction to Counting Sort

Counting sort is a sorting algorithm that works by calculating the positions of each element in the output sequence. (Reading time: 4 minutes)

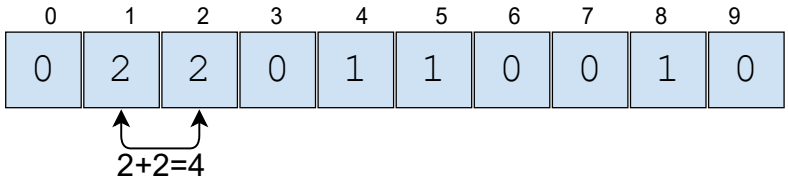
This algorithm first calculates how many times the same item occurs, and then adds all the values. We start off with the array we want to sort and initialize an index array. In this index array, we first store the number of times a particular value occurs in the array we want to sort.



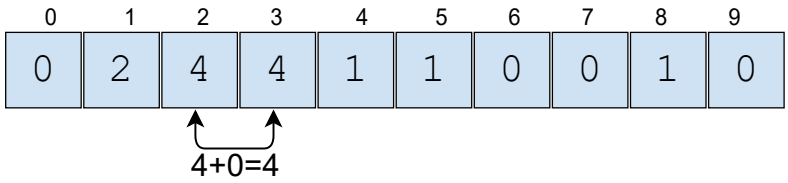
Let's say that we know that all values will be between 0 and 9. We initialize the index array that is 10 elements long. In here, we will store the occurences of the elements.



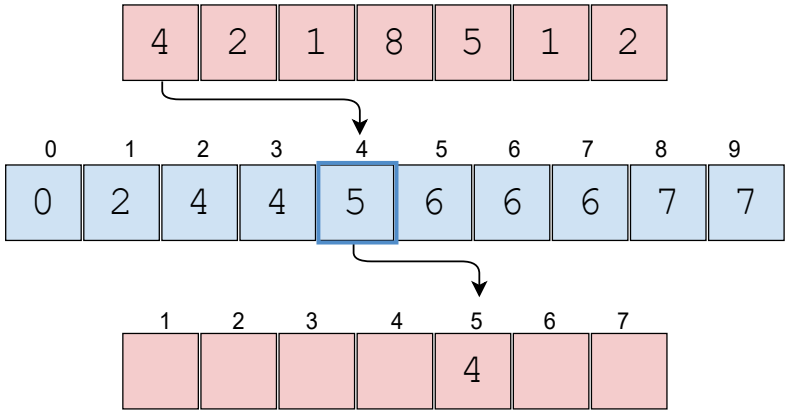
After looping over the array, we fill the index array with the occurrences of the elements. In the array we want to sort, we have two 1, two 2, one 4, one 5, and one 8!



When we have the entire array with all the occurrences, we now add all values with their previous values. The element on index 1 stays the same, because 0+2 is still 2. However, the element in the second index becomes 4, as 2+2 is 4.

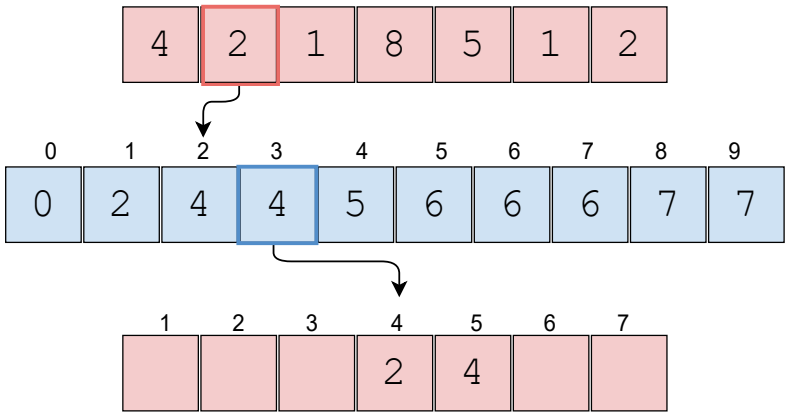


We move on to the next element, the element on index 3, which is 0. 4+0 is 4, so this becomes 4, and so on. The reason you do this, is to figure out how big the length of our sorted array will be.

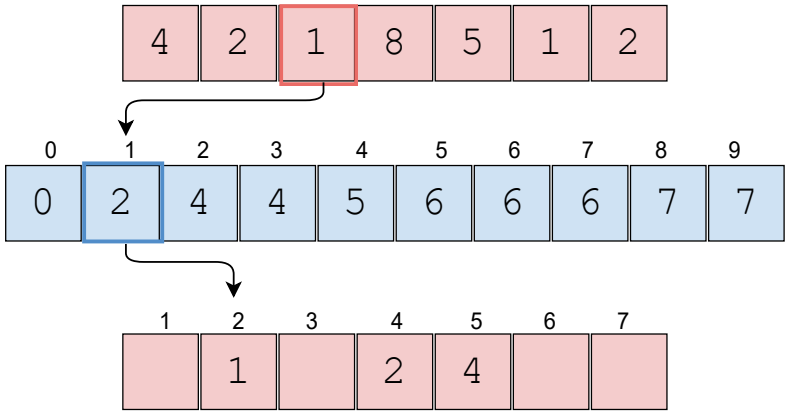


It's time to place the items in the right order. We do this by looping over the original array. The value of the element in the original array, is the index in the index array. The value in the index array is the new index in the sorted array. We have our element with the value 4, so we check the index array at index 4. Here, the value is 5. 5 is the new index of the element 4 in the index array, so we put the element with value 4 on index 5. After doing this, we decrement the value of the

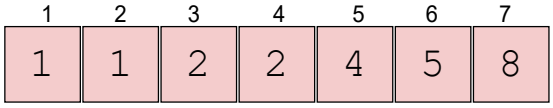
element on index 4 in the index array. In the next example, the elements value on index 4 is decremented by one.



We move to the next element in the array. We place the value of the element on the index with the value of the index array, which is 4 in this case.



We decrement the value again, and move on to the next element. We keep on doing this, until we have the sorted array.



We move to the next element in the array. We place the value of the element on the index with the

We move to the next element in the array. We place the value of the element on the index with the value of the index array, which is 4 in this case.

In the next lesson, I will talk about the implementation of this algorithm.

Counting Sort (Implementation)

(Reading time: 3 minutes)

We create a function that takes 3 arguments: the array, the minimum value, and the maximum value. In the example above, the minimum value was 0, and the maximum value was 9.

```
function countingSort(arr, min, max) {
}
```

First, we need to initialize an empty index array based on the minimum and maximum values that have been passed as parameters. The initial value of all elements in this array is 0.

```
function countingSort(arr, min, max) {
  let count = [];

  for (let i = min; i <= max; i++) {
     count[i] = 0;
  }
}
```

After initializing the index array, we need to fill this array with the occurrences of the elements in the original array. We do this by looping over the original array, and incrementing the value of the element in the index array that corresponds to the value in the original array.

```
for (let i = 0; i < arr.length; i++) {
  count[arr[i]]++;
}
```

Then, we start modifying the original array so that the elements are in the correct position. We do this by checking whether the elements in the index array aren't 0, meaning that they never occur in the original array. If they do, we place them in the correct position in the array. We know the correct

position, as we declare a new variable.

```
let z = 0;

for (let i = min; i <= max; i++) {
  while (count[i]-- > 0) {
    arr[z++] = i;
  }
}
```

We loop over the index array, and check whether the element's value is bigger than 0. If that's the case, then we increment the value of variable z by one, as we go to the next element, and set it equal to the index of the value in the index array.

The entire function looks like:

```
function countingSort(arr, min, max) {
  let count = [];
  let z = 0;

  for (let i = min; i <= max; i++) {
    count[i] = 0;
  }

  for (let i = 0; i < arr.length; i++) {
     count[arr[i]]++;
  }

  for (let i = min; i <= max; i++) {
    while (count[i]-- > 0) {
      arr[z++] = i;
    }
  }
  return arr;
}
```

In the next lesson, I will talk about the time complexity of this algorithm.

Counting Sort (Time Complexity)

best, average and worst case time complexity of the counting sort algorithm (Reading time: under 1 minute)

TIME			SPACE
Best	**Average**	**Worst**	**Worst**
O(n + k)	O(n + k)	O(n + k)	O(k)

Best, average and worst:

As we have three separate for loops, the time it takes for the entire function to run is dependent on the individual loops. If the first for-loop has *linear* complexity, i.e., **O(n)**, and the second for loop also has *linear* complexity, i.e.,**O(k)**, where **k** is the difference between the highest and lowest value in the array we want to sort. We add them together, which makes **O(n** + **k)**.

Worst space:

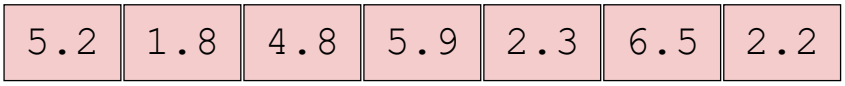
The length of the count array grows with the same amount as the size of the input.

Now, let's move on to the Bucket sort algorithm in the next lesson.

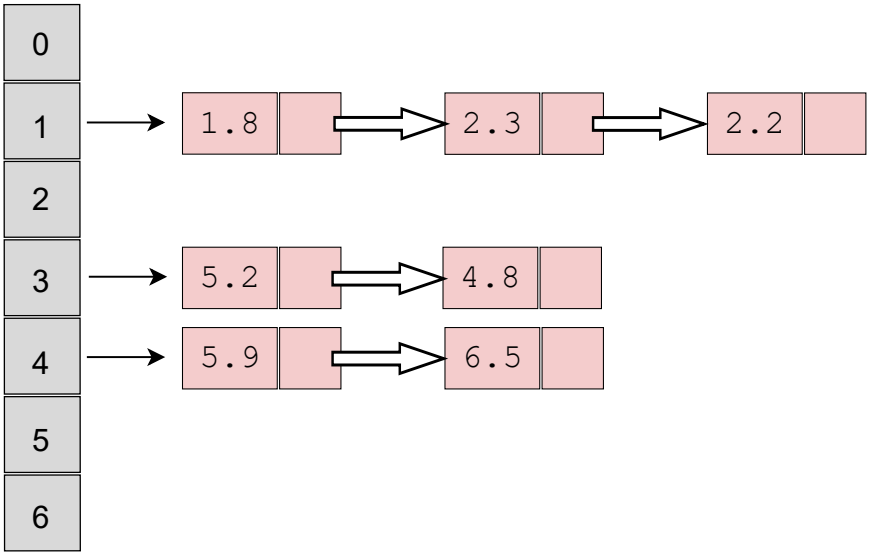
Introduction to Bucket Sort

Bucket sort is a very useful sorting algorithm when working with floating point numbers. (Reading time: under 3 minutes)

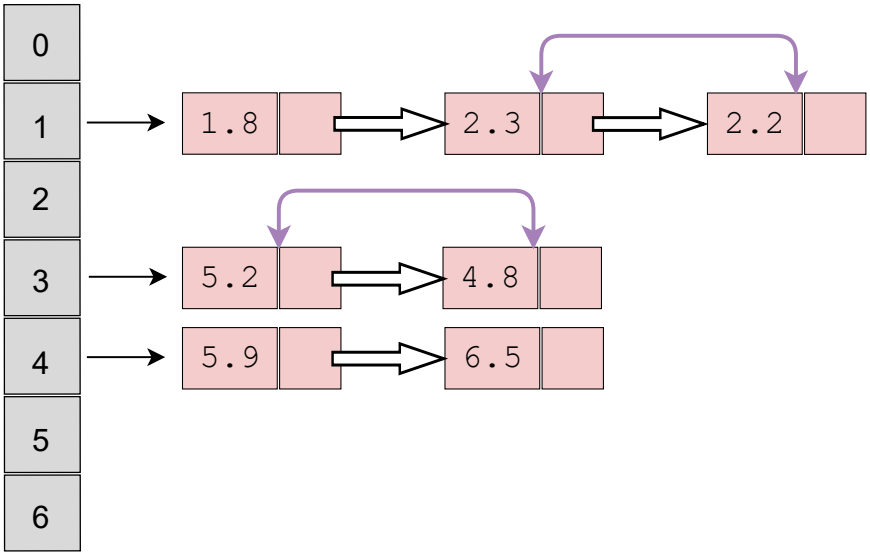
We store the values in buckets, which we then sort using insertion sort, and merge.



Let's say we want to sort the following array, with floating point numbers as values. First, we create as many buckets as the length of the array, 7 in this case. Then, we store the values based on their value, by (for example) using **Math.floor(n * arr[i] / 10)**. This multiplies the length of the array with the value, and divides it by 10.

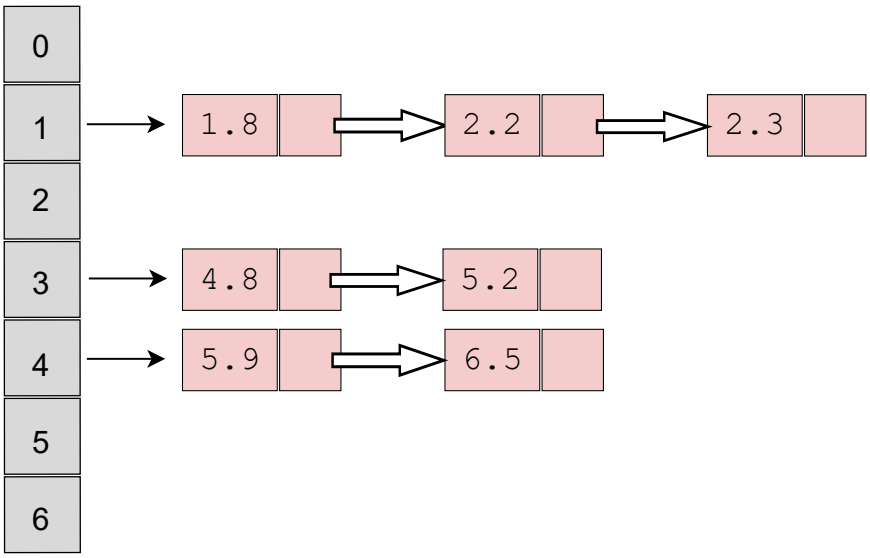


Math.floor(n * arr[i] / 10) is 3, so the element with value 5.2 gets stored in bucket 3. This applies to all elements in the array. The items are connected as a linked list.



Now, we sort the elements in every individual bucket using insertion sort. Then, all items from the lowest bucket to the highest bucket gets pushed to a new array. This means that the lowest values will be pushed first, and the highest values will be pushed last, which results in a sorted array.

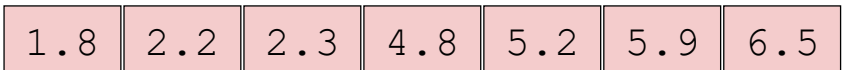
2 of 3



Now, we sort the elements in every individual bucket using insertion sort. Then, all items from the lowest bucket to the highest bucket gets pushed to a new array. This means that the lowest values will be pushed first, and the highest values will be pushed last, which results in a sorted array.

3 of 3

The sorted array becomes:



In the next lesson, I will talk about the implementation of this algorithm.

Bucket Sort (Implementation)

(Reading time: 4 minutes)

We create a function that receives the array that we want to sort as an argument. We can immediately declare 3 variables: n, which is the length of the buckets (which is the length of the array), allBuckets, which is a new array of all the buckets, and the new array sortedArray, to which we will push the elements to in ascending order. If the length of the array is smaller than 2, we don't need to sort it at all. We can then return the array right away.

```
function bucketSort(array) {
  const n = array.length;
  const allBuckets = new Array(n);
  const sortedArray = [];

  if (n < 2) return array;
}
```

For every bucket, we initialize an empty array. We will later push the elements that belong to that bucket to its corresponding array.

```
for (let i = 0; i < n; i++) {
   allBuckets[i] = [];
}
```

In order to push elements to these bucket arrays, we loop over the array we want to sort. For every element, we calculate to which bucket they should be pushed. In this example, I use `Math.floor(n * array[i] / 10)` in order to calculate this, however this function can be different.

```
for (let i = 0; i < n; i++) {
   const index = Math.floor(n * array[i] / 10);
   allBuckets[index].push(array[i]);
};
```

Now, we have all the **buckets**. If we logged the buckets now, it would look like:

```
[
 [],
 [ 1.8, 2.3, 2.2 ],
 [],
 [ 5.2, 4.8 ],
 [ 5.9, 6.5 ],
 [],
 []
]
```

Now, we want to sort every individual array using insertion sort. We loop over the buckets array, and invoke the `insertionSort` function on every array. Then, once the bucket array has been sorted, we push it to the `sortedArray` array! We do this for all the arrays in the `allBuckets` array.

```
allBuckets.forEach(bucket => {
   insertionSort(bucket);
   bucket.forEach(element => sortedArray.push(element))
});
```

The entire function would look like :

```
function bucketSort(array) {
  const n = array.length;
  const allBuckets = new Array(n);
  const sortedArray = [];

  if (n < 2) return array;

  for (let i = 0; i < n; i++) {
    allBuckets[i] = [];
  }

  for (let i = 0; i < n; i++) {
    const index = Math.floor(n * array[i] / 10);
    allBuckets[index].push(array[i]);
  };

  allBuckets.forEach(bucket => {
    insertionSort(bucket);
    bucket.forEach(element => sortedArray.push(element))
  });

  return sortedArray;
}
```

In the next lesson, I will talk about the time complexity of this algorithm.

Bucket Sort (Time Complexity)

best, average and worst case time complexity of the bucket sort algorithm (Reading time: under 1 minute)

TIME			SPACE
Best	**Average**	**Worst**	**Worst**
O(n + k)	O(n + k)	$O(n^2)$	O(n)

Best and average:

As we have two separate for-loops, the time it takes for the entire function to run is dependent on the individual loops.

Worst:

Every element gets allocated at the same bucket. First, we loop over n items, to then loop over n items again in the bucket array. `n * n = n^2`.

Worst space:

The length of the sorted and bucket array grows with the same amount as the size of the input.

Now, let's move on to Radix sort.

Introduction to Radix Sort

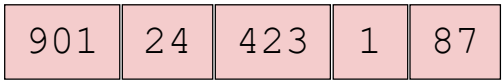
Radix sort is used to sort numbers, and works by sorting the least significant number to the most significant number. (Reading time: under 3 minutes)

A significant number is one that does not have a zero at the beginning.

20: two significant numbers, 2 and 0. The least significant number is 0, the most significant number is 2.

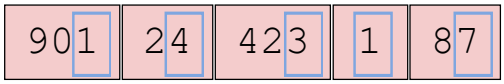
02: one significant number: 2. The least significant number is 2.

12.005: five significant numbers: 2, 2, 0, 0, and 5. The least significant number is 5, the most significant number is 1.



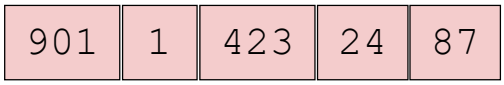
Let's say that we want to sort the array [901, 24, 423, 1, 87].

1 of 6



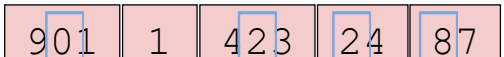
First, we get the least significant numbers of every element in the array. In this case, that results in [1, 4, 3, 1, 7]. Now, the items in the array get sorted based on the value of their least significant number. If there are two items with the same value, like 901 and 1 in this case, they get sorted based on their index. 901 comes before 1, so it stays that way.

2 of 6



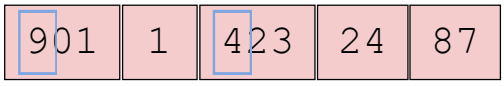
The values are now sorted based on the values of their least significant value. [1, 1, 3, 4, 7].

3 of 6



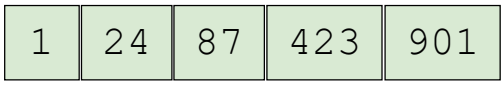
We move one significant value. If the element doesn't contain that many numbers, its value is equal to 0. In this case, our array becomes [0, 0, 2, 2, 8]. However, the numbers are already in the right place! There is no need to sort the array in a different order.

4 of 6



Lastly, we move one more significant value. Only two items in the array have enough numbers, so our array becomes [9, 0, 4, 0, 0]. We first sort the items with the value 0 based on their indexes, and then the items with the higher values. This results in [1, 24, 97, 423, 901].

5 of 6



We have successfully sorted our array using radix sort!

6 of 6

Now, let's move on to the implementation of this algorithm.

Radix Sort (Implementation)

Here, I'll discuss the code of the radix sort algorithm. (Reading time: 4 minutes)

Radix sort uses both counting sort and bucket sort. To implement radix sort, we need to have a `radixSort` function that receives the array we want to sort.

```
function radixSort(array) {
}
```

Right now, we need to store the largest digit of the maximum number in the given array, initialize a digit bucket list where we store the values, and the current index.

```
function radixSort(array) {
  const max = Math.max(...array).toString().length;
  let digitBuckets = [];
  let index = 0;
}
```

Next, we want to initialize a bucket for every digit that's possible. Let's say that we have the array `[8, 23, 12223, 901, 2990, 12]` that we want to sort. Now, **max** would be equal to **5**, as the length of the maximum `12223` value is **5**.

```
for (let i = 0; i < max + 1; i++) {
  digitBuckets = [];
}
```

Now, we want to loop over the numbers, and put the numbers in the buckets they belong, considering the currently active digit! While doing so, we also need to create a function that lets us know what the currently active digit is.

```
for (let j = 0; j < list.length; j++) {
  const digit = getDigit(list[j], i + 1);
}
```

```
function getDigit(num, nth) {
  let value = 0;
  while (nth--) {
    value = num % 10;
    num = Math.floor((num - value) / 10);
  }
  return value;
}
```

To the `getDigit` function, we pass the number `num` and the currently active significant value `nth`. Then, we initialize a default value of **0** as the number on the currently active digit: not all numbers have the same amount of significant numbers! Then, we have to calculate the value of the currently active digit, by `num % 10`. If the number is **1234**, the value would now be **4**. Then, we change the value of `num`, as we just checked one significant value. By subtracting the currently active digit, and then dividing it by **10**, we have our new value of `num`. If it was **1234** previously, `num` is now equal to **123**. This keeps on going, until `nth` returns `false`, when `nth` is **0**. Then, we return the value which is equal to the digit that we currently care about.

Now, we want to store every value in the same array, based on their value. For example, in the first round when we check the least significant digit, we would want the `digitBuckets` array to look like `[[10], [31], [22, 902], [3]]`.

The index of the array they should be pushed to, is equal to the `value` that got returned from the `getDigit` function! If that array doesn't exist yet, we have to initialize it first.

```
digitBuckets[digit] = digitBuckets[digit] || [];
digitBuckets[digit].push(list[j]);
```

Then, we want to loop over the `digitBuckets` array. Before doing so, we set the value of the `index` variable back to **0**.

```
idx = 0;
for (let t = 0; t < digitBuckets.length; t++) {
}
```

Now, we need to check whether the current array has a length. That's not always the case: the `digitBuckets` array could look like: `[[10], [], [52]]`

always the case, the `digitBuckets` array could look like: `[[10], [], [32]]`.

Then, we want to loop over the separate arrays, and place the element in the correct position in the original array.

```
if (digitBuckets[t] && digitBuckets[t].length > 0) {
   for (let j = 0; j < digitBuckets[t].length; j++) {
      list[idx++] = digitBuckets[t][j];
   }
}
```

Right now, we've modified the original list, with the items in the correct order.

The entire `radixSort` function looks like:

```
function radixSort(array) {
  const max = Math.max(...array).toString().length;
  let digitBuckets = [];
  let index = 0;
  for (let i = 0; i < max + 1; i++) {
    digitBuckets = [];
    for (let j = 0; j < array.length; j++) {
       const digit = getDigit(array[j], i + 1);
       digitBuckets[digit] = digitBuckets[digit] || [];
       digitBuckets[digit].push(array[j]);
    }

    idx = 0;
    for (let t = 0; t < digitBuckets.length; t++) {
      if (digitBuckets[t] && digitBuckets[t].length > 0) {
        for (let m = 0; m < digitBuckets[t].length; m++) {
           array[idx++] = digitBuckets[t][m];
        }
      }
    }
  }
  return array;
}
```

Now, let's discuss the time complexity of this algorithm.

Radix Sort (Time Complexity)

best, average and worst case time complexity of the radix sort algorithm. (Reading time: 1 minute)

TIME			SPACE
Best	**Average**	**Worst**	**Worst**
O(nk)	O(nk)	O(nk)	O(n + k)

Best, average and worst:

There are nested for-loops. We iterate over the outer for-loop **n** times, and the inner for loop **k** times. This results in **O(n×k)**.

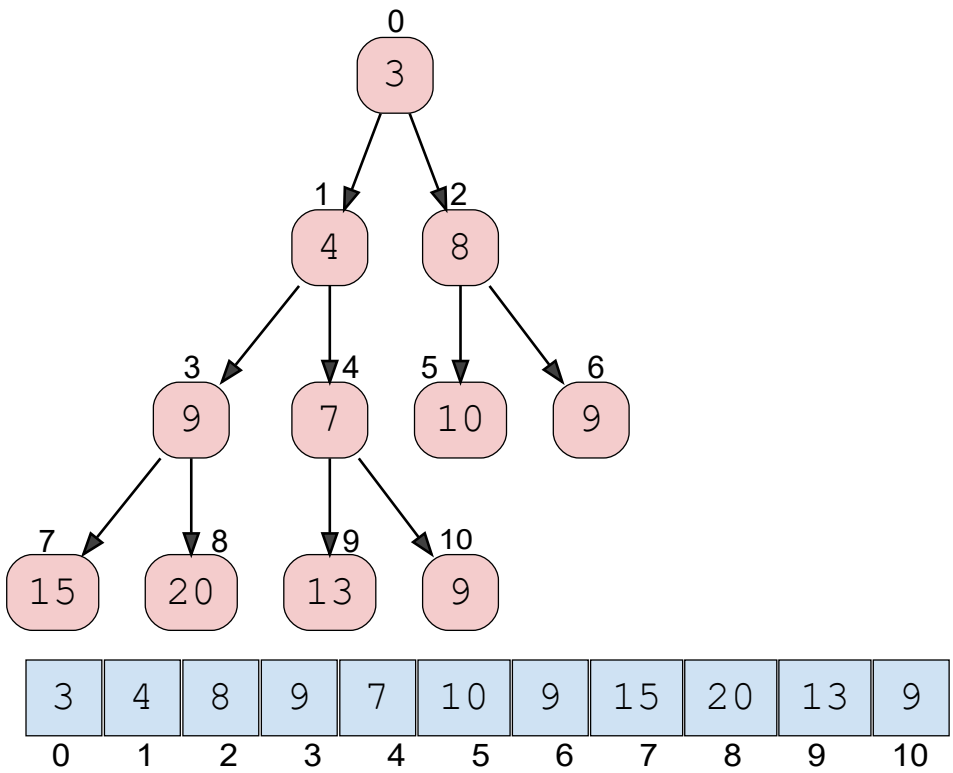
Worst space: Outside the for-loops, there are four constant variables: **O(n)**, and inside the for-loops there are four constant variables: **O(k)**. This results in `O(n) + O(k) = O(n + k)`.

Now, let's move on to the Heapsort algorithm.

Introduction to Heap Sort

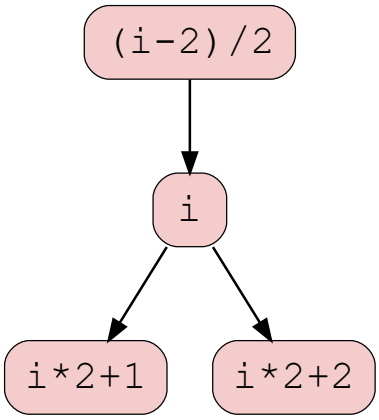
This algorithm creates a heap to sort the elements of an array. (Reading time: under 3 minutes)

In a heap, all nodes are stored based on the value of their parent node.

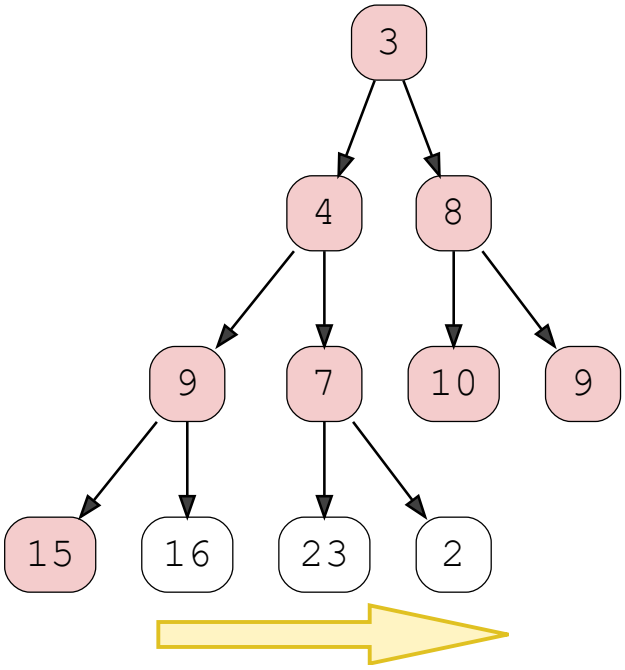


This is a **minimum heap**: the children of a node are always smaller than or equal to their parent. In a maximum heap, the children of a node are always bigger or equal to their parent. It is not sorted yet!

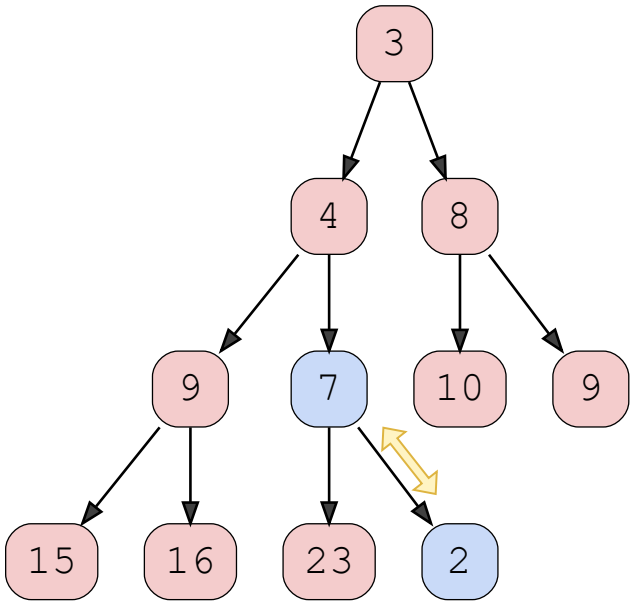
With a heap, you can easily find a node's parent node and children nodes, based on its index in the array.



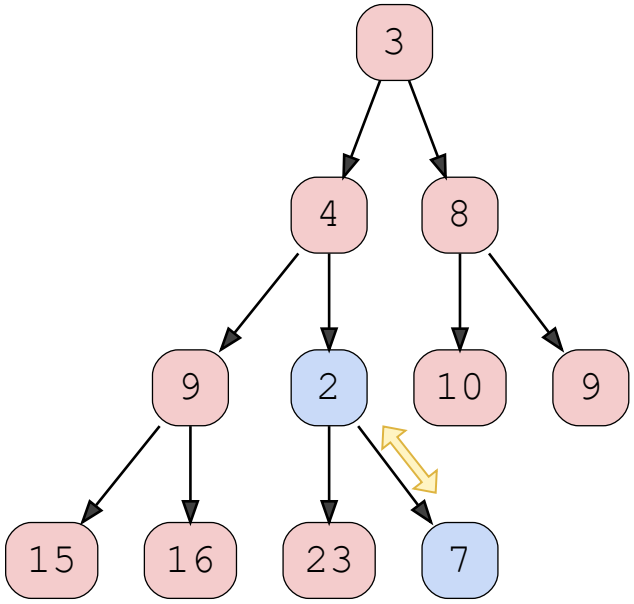
Whenever you add new nodes, they're always added in the same way: they get added to the first free spot, from left to right. You don't specify a node's parent in a heap.



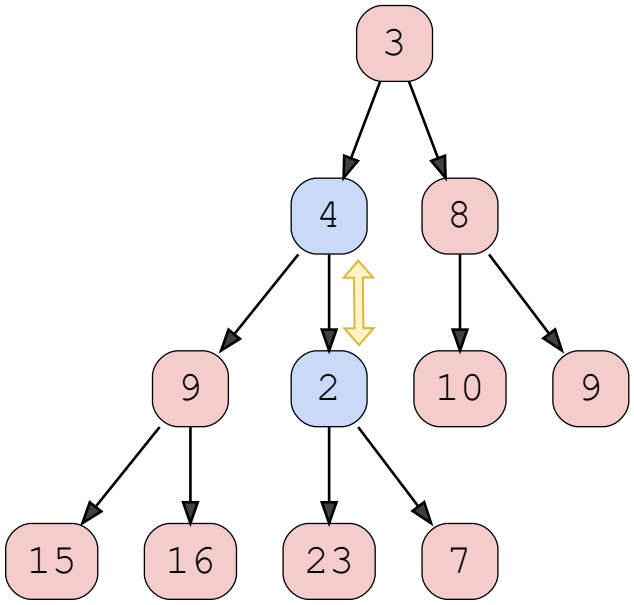
However, this means that the added node might not be in the right place, just like here with the number 2. In order to solve this, we compare the node with its parent node. If the node's value is smaller than that of the parent node, we swap them, until the node is in the right position.



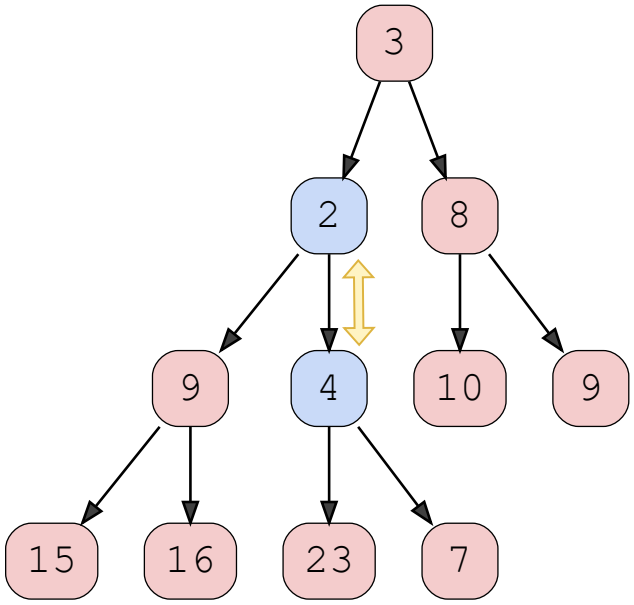
Comparing 2 and 7



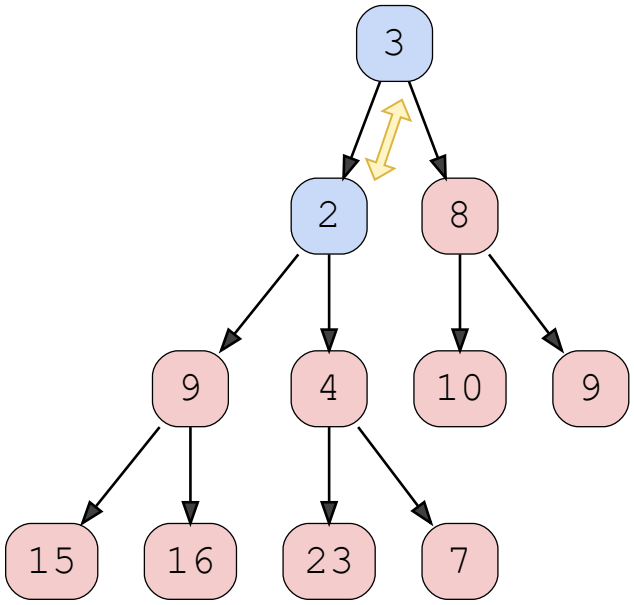
2 < 7, so swap



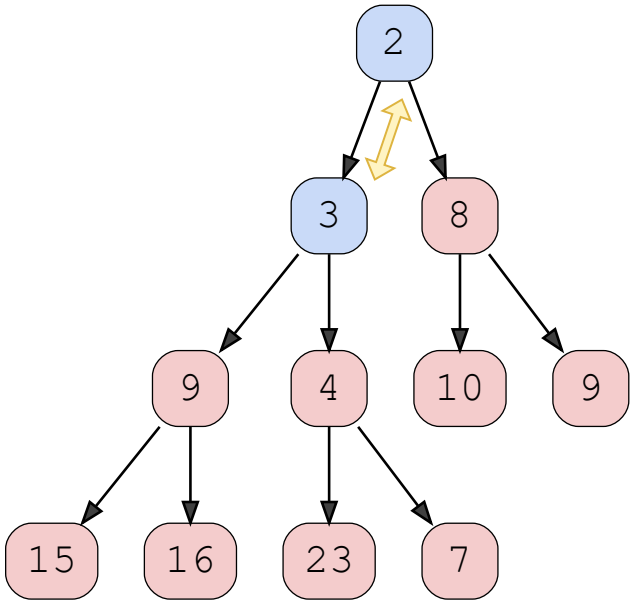
Comparing 2 and 4



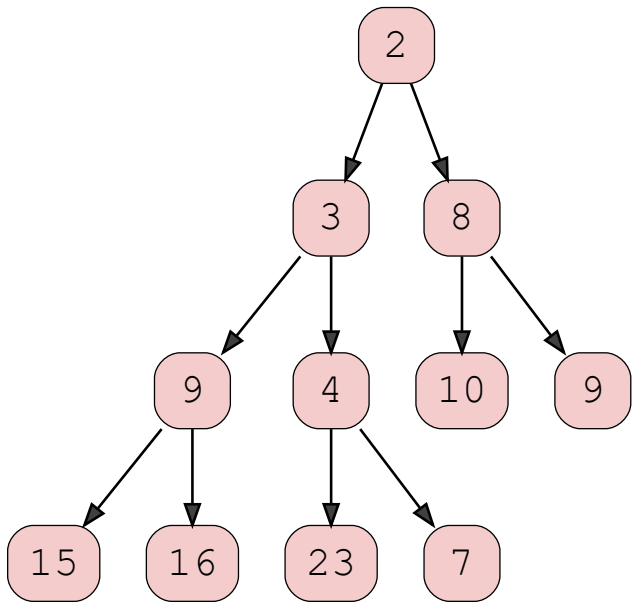
2 < 4, so swap



Comparing 2 and 3



2 < 3, so swap



Sorted!

7 of 7

Now, let’s discuss the implementation of this algorithm.

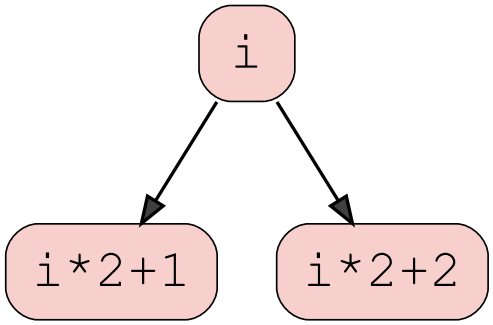
Heap Sort (Implementation)

(Reading time: 3 minutes)

To make a heap out of an array, we first need to go over all the items in the array, from right to left. This is necessary, as we start at the leaves. It receives the array we want to sort, and invokes the function that makes a heap on every element.

```
function makeHeap(arr) {
  const n = arr.length;
  for (let i = n - 1; i >= 0; i--) {
    // Heapify function.
  }
}
```

The heapify function should receive the array, the length of the array, and the current index. In order to make the heap, this order is very important to remember:



If we have a *maximum heap*, the node with the highest value should be on top. Next, we have the left node, and the right node.

```
function maxHeapify(arr, n, i) {
   let max = i;
```

```
   const left = 2 * i + 1;
   const right = 2 * i + 2;
}
```

By default, we make `i` the maximum value. However, if the value of i is not the maximum, we need to swap elements.

If there is a **left** or **right** value, and if the maximum value is smaller than the **left** or **right** value, we make the left or right value the maximum value. However, we're not actually swapping yet! We're just redeclaring the `max` variable! If the value of the `max` variable changed, the last if-statement returns true, and we swap the elements. The `maxHeapify` function gets called again, only this time for the values based on the **maximum** value. This way, we go through the array, and *maxHeapify* the entire array!

```
if (left < n && arr[max] < arr[left]) {
   max = left;
}

if (right < n && arr[max] < arr[right]) {
   max = right;
}

if (max !== i) {
   [arr[i], arr[max]] = [arr[max], arr[i]]
   maxHeapify(arr, n, max)
}
```

The entire function would look like:

```
function maxHeapify(arr, n, i) {
   let max = i;
   const left = 2 * i + 1;
   const right = 2 * i + 2;

   if (left < n && arr[max] < arr[left]) {
      max = left;
   }

   if (right < n && arr[max] < arr[right]) {
      max = right;
   }

   if (max !== i) {
      [arr[i], arr[max]] = [arr[max], arr[i]]
      maxHeapify(arr, n, max)
   }
}

function makeHeap(arr) {
  const n = arr.length;
  for (let i = n - 1; i >= 0; i--) {
     maxHeapify(arr, n, i);
```

```
    maxHeapify(arr, n, i);
  }
  return arr;
}
```

In the next lesson, I will talk about the time complexity of this algorithm.

Heapsort (Time Complexity)

best, average and worst case time complexity of the heapsort algorithm. (Reading time: under 1 minute)

TIME			SPACE
Best	**Average**	**Worst**	**Worst**
O(nk)	O(nk)	O(nk)	O(n + k)

Best, average and worst:

The size of the input array that we want to sort, gets heapified **n** times.

Worst space:

There are **five** constant variables: `n`, `i`, `max`, `left` and `right`.